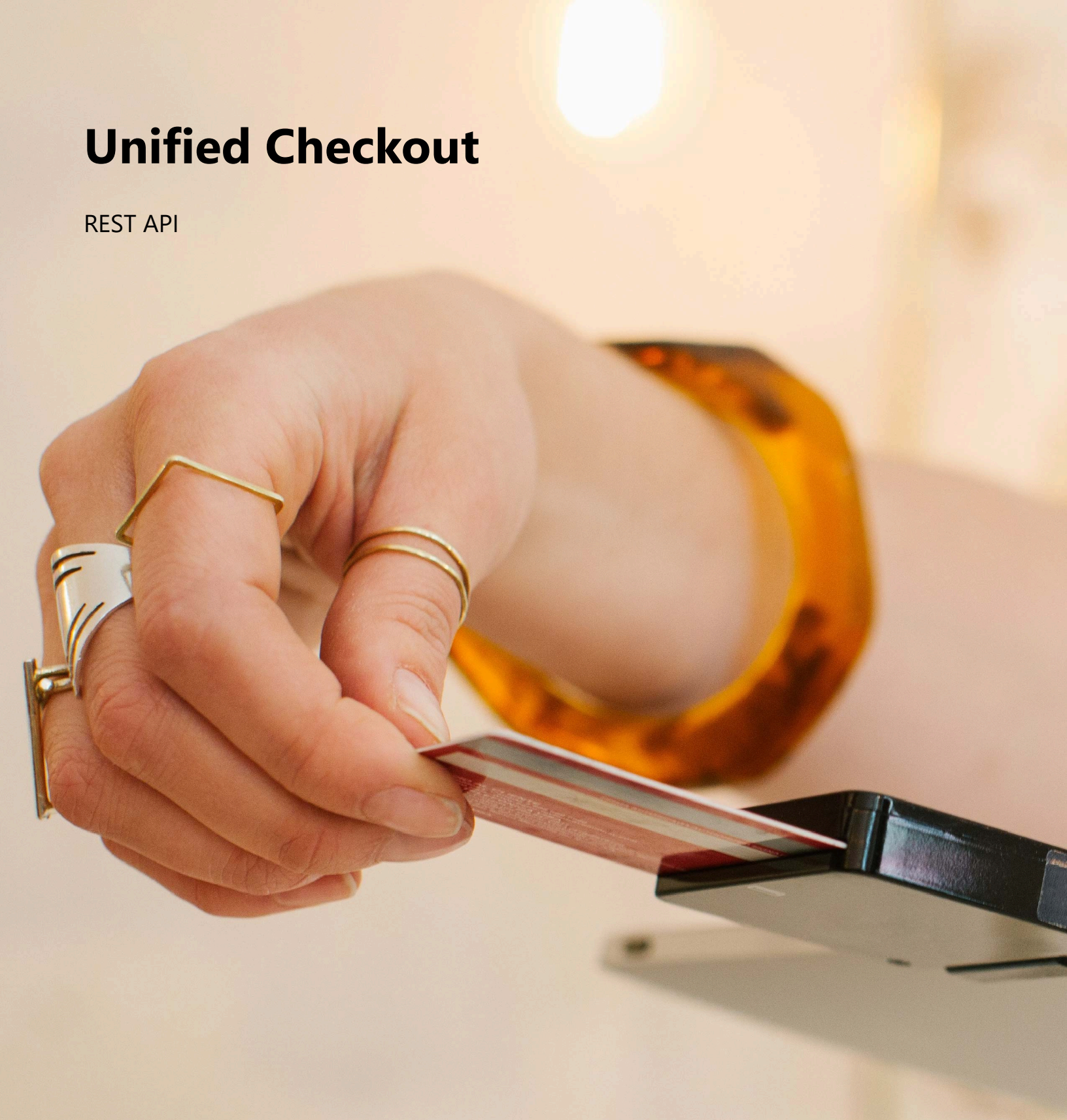
Unified Checkout

REST API



Developer Guide

cybersource
A Visa Solution

© 2025. Cybersource Corporation. All rights reserved.

Cybersource Corporation (Cybersource) furnishes this document and the software described in this document under the applicable agreement between the reader of this document (You) and Cybersource (Agreement). You may use this document and/or software only in accordance with the terms of the Agreement. Except as expressly set forth in the Agreement, the information contained in this document is subject to change without notice and therefore should not be interpreted in any way as a guarantee or warranty by Cybersource. Cybersource assumes no responsibility or liability for any errors that may appear in this document. The copyrighted software that accompanies this document is licensed to You for use only in strict accordance with the Agreement. You should read the Agreement carefully before using the software. Except as permitted by the Agreement, You may not reproduce any part of this document, store this document in a retrieval system, or transmit this document, in any form or by any means, electronic, mechanical, recording, or otherwise, without the prior written consent of Cybersource.

Restricted Rights Legends

For Government or defense agencies: Use, duplication, or disclosure by the Government or defense agencies is subject to restrictions as set forth the Rights in Technical Data and Computer Software clause at DFARS 252.227-7013 and in similar clauses in the FAR and NASA FAR Supplement.

For civilian agencies: Use, reproduction, or disclosure is subject to restrictions set forth in subparagraphs (a) through (d) of the Commercial Computer Software Restricted Rights clause at 52.227-19 and the limitations set forth in Cybersource Corporation's standard commercial agreement for this software. Unpublished rights reserved under the copyright laws of the United States.

Trademarks

Authorize.net and The Power of Payment are registered trademarks of Cybersource Corporation. Cybersource and Cybersource Decision Manager are trademarks and/or service marks of Cybersource Corporation. Visa, Visa International, Cybersource, the Visa logo, the Cybersource logo, and 3-D Secure are the registered trademarks of Visa International in the United States and other countries. All other trademarks, service marks, registered marks, or registered service marks are the property of their respective owners.

Version: 26.08.01

Contents

Unified Checkout Developer Guide **7**
Recent Revisions to This Document 8
Introduction to Unified Checkout **14**
Unified Checkout Quick Start **17**
Step 1: Enable Unified Checkout 18
Step 2: Set Up the Server-Side Component 20
Step 3: Set Up the Client-Side Component 22
Step 4: Configure Unified Checkout 26
Step 5: Test Your Unified Checkout Integration 27
Unified Checkout Flow **29**
Payment Methods **32**
Cards 33
Pay with Token 35
eCheck/ACH Service 39
Enrolling in eCheck/ACH Services 40
Pay with eCheck/ACH Service UI 41
Digital Wallets 43
Apple Pay 45
Google Pay 48
Click to Pay 52
Paze 68
Alternative Payment Methods 71
Online Bank Transfers 72
Buy Now, Pay Later 87
Post-Pay Reference Payments 94
Alternative Payment Wallets 98
Integration Methods **113**
Checkout 114
Button 118
Trigger 121
Components 124
Server-Side Set Up **129**

Capture Context 129
Versioning 132
Client Version History 133
Client-Side Set Up 136
Loading the JavaScript Library 137
JavaScript Example: Initializing the SDK 138
JavaScript Example: Displaying the Button List 138
JavaScript Example: Client-Defined Trigger for Click to Pay or PAN Entry 139
JavaScript Example: Processing a Payment 139
JavaScript Example: Authorization 140
JavaScript Example: Sale 141
JavaScript Example: Sale with Decision Manager 142
JavaScript Example: No Service Orchestration 142
JavaScript Example: Setting Up with Full Sidebar 143
JavaScript Example: Setting Up with the Embedded Component 145
Configuration 147
Configure the Unified Checkout Merchant Experience 147
Enable Unified Checkout 150
Configure Payment Options 152
Customize the Unified Checkout Look and Feel 155
Look and Feel UI Examples 159
Configure Customer Data and Payment Flow 165
Capture Context Fields in the Business Center 167
Manage Permissions 171
Process Payments with Unified Checkout 174
Webhooks Support 176
Sessions API 179
Capture Context 179
Allowed Card Networks 181
Target Origins 183
Allowed Payment Types 183
Client Version 185
Auto-check Enrollment 185
Button Type 186
Customize Button Text 187

Complete Mandate ... 188
Capture Mandate ... 194
Include Card Prefix ... 199
Email Autolookup ... 201
Mobile as Identity for Click to Pay ... 201
Example: Unified Checkout Complete Capture Context ... 202
Validating the Capture Context ... 210
Session Validation ... 212
Transient Tokens ... 214
Transient Token Format ... 214
Example: Transient Token Format ... 215
Token Verification ... 217
Dual-Branded Cards ... 218
Authorizations with a Transient Token ... 220
REST Example: Requesting an Authorization with a Transient Token ... 221
Test Your Configuration ... 223
Unified Checkout Test Cards ... 223
Visa and Mastercard Click to Pay Test Cards ... 225
Test Cards for Authentication by Click to Pay ... 227
Echeck Test Values ... 228
Test Authentication ... 229
Handle Errors ... 231
UnifiedCheckoutError ... 232
Detect Errors ... 232
Error Handling Patterns ... 233
Error Codes ... 234
Reason Codes ... 237
Payment Details API ... 241
REST Example: Retrieving Transient Token Payment Details ... 242
JavaScript API Reference ... 244
VAS.UnifiedCheckout(sessionJWT) ... 244
UnifiedCheckoutInterface ... 245
Checkout ... 249
Trigger ... 252
PaymentButton ... 254

Events ... 255
Update to Unified Checkout Version 1 ... 257
Update Reason Codes ... 263
Version 1 Update Checklist ... 264
Appendix ... 265
JSON Web Tokens ... 266
Supported Countries for Digital Payments ... 268
Supported Countries for Digital Payments A-D ... 268
Supported Countries for Digital Payments E-K ... 270
Supported Countries for Digital Payments L-R ... 274
Supported Countries for Digital Payments S-Z ... 277
Supported Locales ... 280
Security Recommendations ... 282
PCI Compliance ... 285
Client Version History ... 286
Visa Platform Connect: Specifications and Conditions for Resellers/Partners ... 290

Unified Checkout Developer Guide

This section describes how to use this guide and where to find further information.

Audience and Purpose

This document is written for merchants who want to enable Unified Checkout so that they can accept payments on their e-commerce page. Merchants use the Unified Checkout JavaScript SDK to place a digital button widget on their e-commerce site, allowing Cybersource to capture payment data on their behalf.

Conventions

This statement is used in this document:

> **Important:** An *Important* statement contains information essential to successfully completing a task or learning a concept.

Related Documentation

Visit the Cybersource documentation hub to find additional technical documentation.

Customer Support

For support information about any service, visit the Support Center:

http://support.visaacceptance.com

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Recent Revisions to This Document

26.08.01

Payment Methods

Removed support for Tink Pay By Bank in the EU. See Tink Pay By Bank (on page 76).

Added information about domestic card schemes. See Allowed Card Networks (on page 181) and Dual-Branded Cards (on page 218).

26.07.01

Payment Methods

Updated the list of supported card types for Apple Pay. See Cards (on page 33).

Integration Methods

Added information about the Unified Checkout integration methods. See Integration Methods (on page 113).

Configuration

Updated the steps for setting up a webhooks subscriptions. See Webhooks Support (on page 176).

Sessions API

Added information about values greater than or equal to 0.00 in the **data.orderInformation.amountDetails.totalAmount** field in the capture context. See Capture Context (on page 179).

Added information about passkey authentication. See completeMandate.consumerAuthentication (on page 190).

Added information about the removal of the contact details page during Click to Pay autolookup. See Email Autolookup (on page 201).

Test Your Configuration

Added support for testing Mastercard cards. See Visa and Mastercard Click to Pay Test Cards (on page 225) and Test Cards for Authentication by Click to Pay (on page 227).

Appendix

Added support for these locales:

- `ar_JO`
- `ar_QA`
- `en_AE`

- `en_LK`
- `en_QA`
- `si_LK`
- `ta_LK`

Added information about right-to-left Arabic language support. See Supported Locales (on page 280).

26.05.01

Payment Methods

Added information about expanded regional support Tink Pay By Bank. See Tink Pay By Bank (on page 76).

26.04.01

Payment Methods

Added information about using Tink Pay By Bank in the UK. See Tink Pay By Bank (on page 76).

Added support for PayPal and Venmo. See PayPal (on page 101) and Venmo (on page 104).

Test Your Configuration

Updated the Visa Click to Pay test card numbers. See Visa and Mastercard Click to Pay Test Cards (on page 225).

26.03.01

Appendix

Added information about how to update your integration to Version 1. See Update to Unified Checkout Version 1 (on page 257).

Added information about security recommendations and PCI compliance. See Security Recommendations (on page 282).

Capture Context

Added information to the capture context section for Version 1. See Capture Context (on page 179).

Added support for client version `1.0`. See Client Version History (on page 133).

Added information about semantic versioning. See Versioning (on page 132).

Added endpoints for Saudi Arabia. See Sessions API (on page 179) and Validating the Capture Context (on page 210)

Client-Side Setup

Updated the JavaScript examples and reference for Version 1. See Loading the JavaScript Library (on page 137) and JavaScript API Reference (on page 244).

Merchant Experience

Added support for the merchant experience in the Business Center. See Configure the Unified Checkout Merchant Experience (on page 147).

Added information about processing payments with Unified Checkout. See Process Payments with Unified Checkout (on page 174).

Payment Details

Updated the endpoint for retrieving payment details and added endpoints for Saudi Arabia. See Payment Details API (on page 241).

Payment Methods

Added support for paying with tokens. See Pay with Token (on page 35).

Quick Start

Added a quick-start section. See Unified Checkout Quick Start (on page 17).

Test Your Configuration

Added information about handling errors. See Handle Errors (on page 231).

Updated the reason codes to include Version 1 reason codes. See Reason Codes (on page 237).

Transient Tokens

Added endpoints for Saudi Arabia. See Authorizations with a Transient Token (on page 220).

25.12.01

Server-Side Set Up

Added information about supported browsers. See Capture Context (on page 129).

Capture Context

Added support for up to **clientVersion** `0.34`. See Client Version History (on page 133).

Transient Tokens

Added information about Cartes Bancaires dual-branded cards. See Dual-Branded Cards (on page 218).

Payment Methods

Added information about Konbini. See Konbini (on page 95).

Test Your Unified Checkout Configuration

Added test payment information for testing Visa Click to Pay authentication within the Click to Pay flow and outside of the flow. See Test Your Configuration (on page 223).

25.11.02

Capture Context

Added a complete capture context request example with all possible fields. See Example: Unified Checkout Complete Capture Context (on page 202).

Updated the capture context examples for **clientVersion** `0.31`. See Capture Context (on page 129).

Appendix

Added information about testing your authentication method. See Test Authentication (on page 229).

Added information about customizing the Click to Pay UI. See Click to Pay UI Guidelines (on page 60).

25.11.01

Capture Context

Updated the list of allowed card networks. See Sessions API (on page 179).

Payment Methods

Added information about Bancontact, Dragonpay and MyBank. See these topics:

- Bancontact (on page 74)
- DragonPay (on page 75)
- MyBank (on page 80)

25.10.02

Capture Context

Added support for **clientVersion** `0.30`. See Client Version History (on page 133).

Payment Methods

Added information about Tink Pay By Bank. See Tink Pay By Bank (on page 76).

25.10.01

Unified Checkout Appendix

Added missing reason codes. See Reason Codes (on page 237).

Updated the **clientVersion** field value to `0.30` in examples.

Added Pakistan locales. See Supported Locales (on page 280).

Payment Methods

Added information about handling responses for Online Bank Transfers and Buy Now, Pay Later. See Handle Responses for Online Bank Transfers (on page 85) and Handle Responses for Buy Now, Pay Later (on page 92).

25.08.02

Added more information about JSON Web Tokens. See JSON Web Tokens (on page 266).

25.08.01

Client-Side Setup

Added information for creating a trigger. See JavaScript Example: Client-Defined Trigger for Click to Pay or PAN Entry (on page 139).

Capture Context

Added PayPak as a payment method. See Sessions API (on page 179).

Added information for client version 0.28. See Client Version History (on page 133).

Payment Methods

Added information for alternative payment methods. See Payment Methods (on page 32).

25.05.01

Capture Context

Added information for the complete mandate. See Sessions API (on page 179).

Added information on the available features and the fields specific to each feature to the Capture Context API. See Sessions API (on page 179).

Added Paze to the list of allowed payment types. See Sessions API (on page 179).

Added capture context validation. See Validating the Capture Context (on page 210).

Unified Checkout Appendix

Added a client version history and the features included in each version. See Client Version History (on page 133).

Updated the list of supported languages. See Supported Locales (on page 280).

Client-Side Set Up

Updated the examples for loading the JavaScript library:

- JavaScript Example: Initializing the SDK (on page 138)
- JavaScript Example: Displaying the Button List (on page 138)
- JavaScript Example: Processing a Payment (on page 139).

Click to Pay

Updated the list of supported countries to include Bulgaria, Greece, Japan, Romania, Slovenia, Thailand, and Vietnam. See Supported Countries for Digital Payments (on page 268).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Introduction to Unified Checkout

Unified Checkout provides a single interface with which you can accept numerous types of card, digital, and alternative payments. Unified Checkout calls other follow-on services such as Payments, Decision Manager, Payer Authentication, and Token Management Service (TMS).

Unified Checkout consists of a server-side component and a client-side JavaScript library.

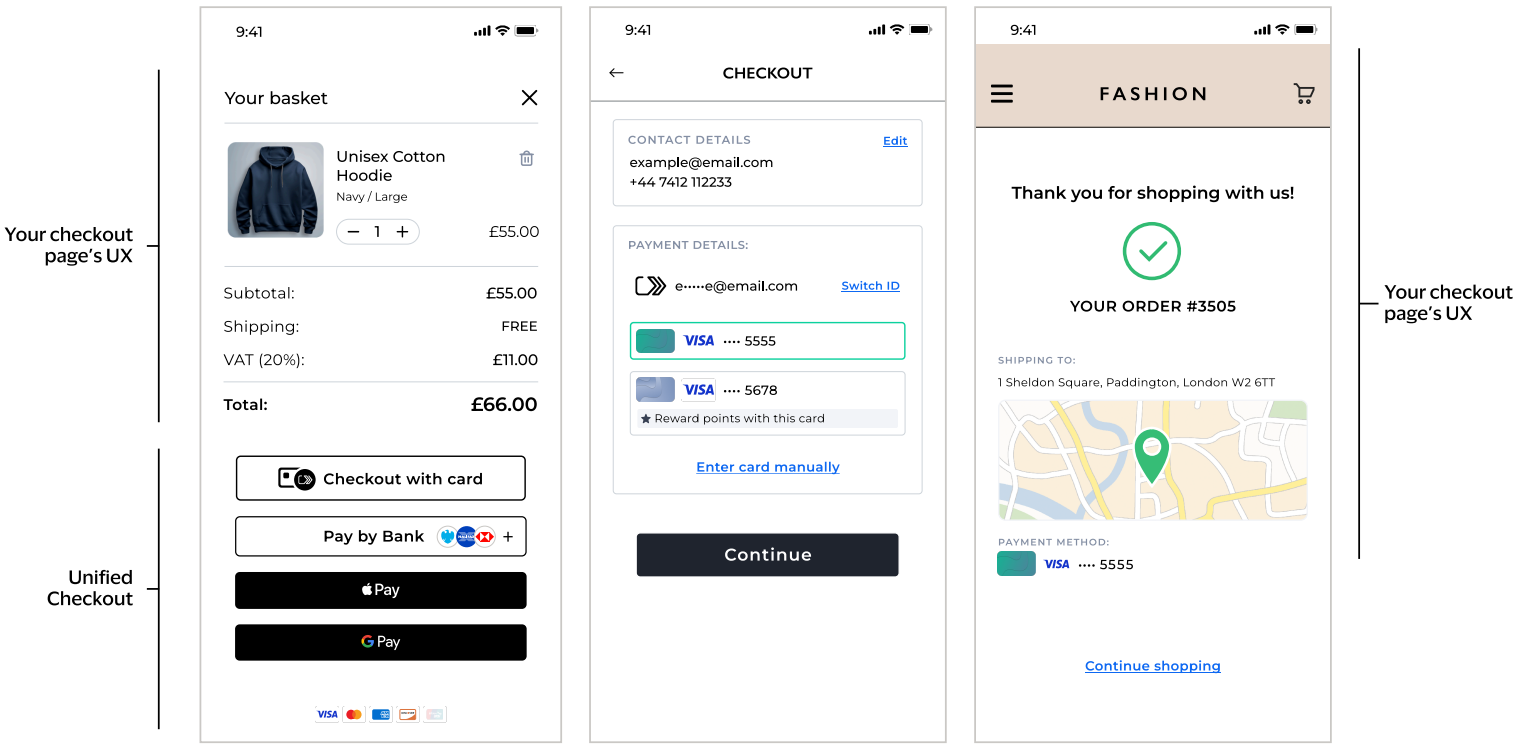
The server-side component authenticates your merchant identity and instructs the system to act within your payment environment. The response contains limited-use public keys. The keys are for end-to-end encryption and contain merchant-specific payment information that drives the interaction of the application. The client-side JavaScript library dynamically and securely places digital payment options onto your e-commerce page.

The provided JavaScript library enables you to securely accept many payment options within your e-commerce environment. Unified Checkout can be embedded seamlessly into your existing webpage, simplifying payment acceptance.

When a customer selects a payment method from the button widget, Unified Checkout handles all interactions with the payment method that was chosen. Unified Checkout is also able to orchestrate requests for to follow-on services such as Payments, Decision Manager, Payer Authentication, and TMS before it provides a response to your e-commerce system.

The figure below shows Unified Checkout with customer checkout payment options.

Button Widget



For examples of different payment method UIs through Unified Checkout, see these topics:

- Click to Pay UI (on page 60)
- Google Pay UI (on page 51)
- Pay with eCheck/ACH Service UI (on page 41)
- Paze UI (on page 69)
- Apple Pay UI (on page 47)

.

Important: Each request that you send to Cybersource requires header information. For information about constructing the headers for your request, see the *Getting Started with REST Developer Guide*.

Key Features



- **Low-code integration**: You can use as few as three lines of JavaScript to accept payments, as well as add or remove payment methods through portal configuration without changing your integration code.
- **PCI SAQ-A compliant**: Payment data never touches your systems.
- **Fully customizable**: You can match the payment experience to your brand with theming, fonts, and layout options. Embed inline or display as a sidebar overlay.
- **Service orchestration**: You can use Decision Manager, Payer Authentication (3-D Secure), and Token Management Service (TMS) throughout the session.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Unified Checkout Quick Start

Unified Checkout is a powerful and flexible payment solution that simplifies the integration process and enhances the customer checkout experience. This guide will help you get up and running with Unified Checkout.

Key Features

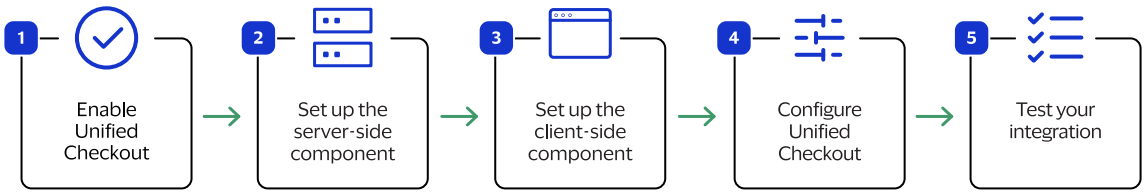
- Seamless integration with your existing e-commerce platform.
- Support for multiple payment methods.
- Customizable checkout flow.
- Enhanced security features.
- Responsive design for mobile and desktop.

Benefits

- Simplified integration process.
- Improved conversion rates.
- Reduced cart abandonment.
- Enhanced customer experience.
- Compliance with industry security standards.

This graphic provides an overview of the steps you must follow to get set up with Unified Checkout:

Unified CheckoutIntegration Overview



Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

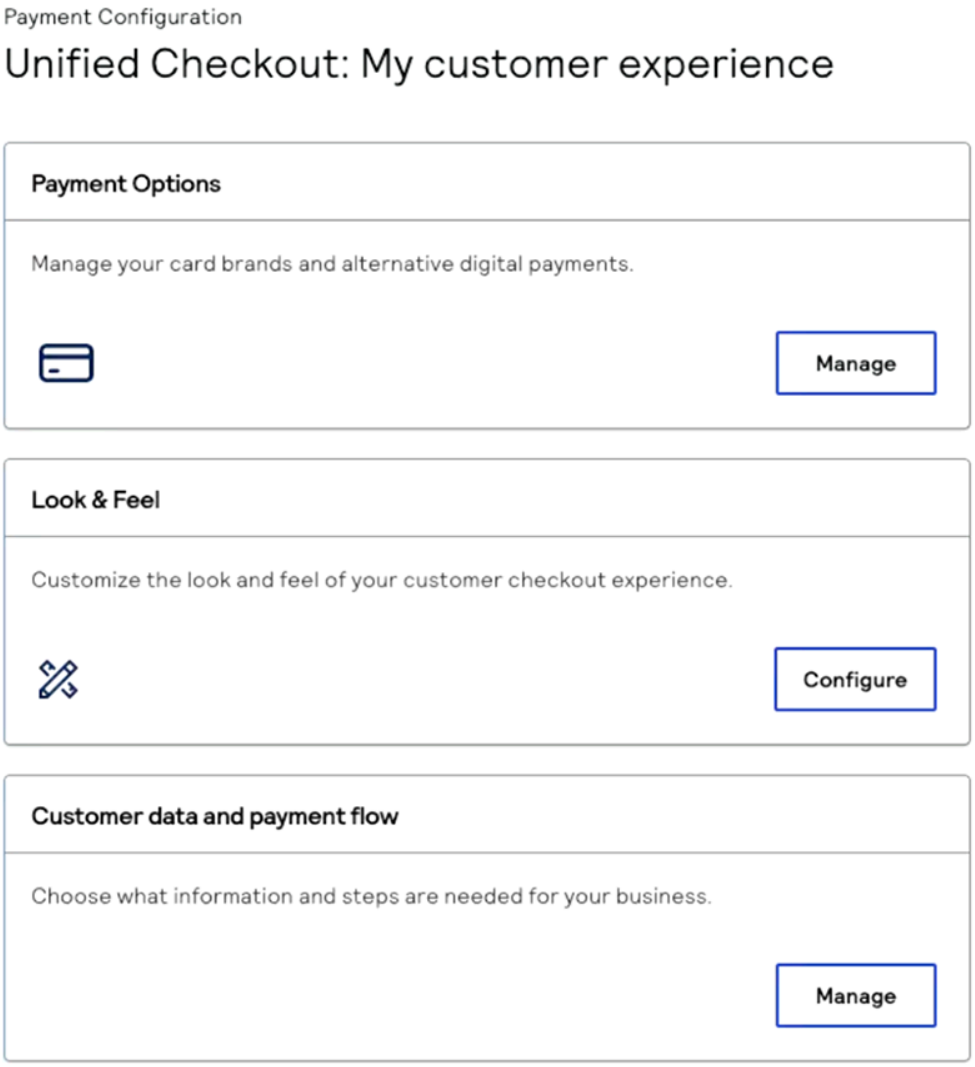
Step 1: Enable Unified Checkout

To begin using Unified Checkout, you must first ensure that your merchant ID (MID) is configured to use the service and that any payment methods you intend to use are properly set up.

1. Log in to the Business Center:
   Test URL: `https://businesscentertest.cybersource.com/ebc2`
   Production URL: `https://businesscenter.cybersource.com`
   If you are unable to access this page, contact your sales representative.

2. In the Business Center, go to the left navigation panel and choose **Payment Configuration** > **Unified Checkout**. The Unified Checkout customer experience page appears:

Unified Checkout Customer Experience



3. The Unified Checkout configuration interface provides complete low-code control over your checkout experience by using dedicated configuration screens accessible through the Business Center. The configuration interface is organized into separate screens, each accessible from the *My customer experience* page. Configure each of these components of the checkout experience:

    - **Payment Options**: Add, remove, and arrange payment methods. For information about configuring payment methods, see Configure Payment Options (on page 152).
    - **Look & feel**: Configure visual appearance and branding. For information about configuring the look and feel, see Customize the Unified Checkout Look and Feel (on page 155).
    - **Customer information and payment flow**: Control data collection and manage payment processing service. For information about configuring customer data, see Configure Customer Data and Payment Flow (on page 165).

Proceed to Step 2: Set Up the Server-Side Component (on page 20).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Step 2: Set Up the Server-Side Component

To initialize Unified Checkout within your webpage, you need to set up the server-side component. This task involves generating a capture context. A capture context is a signed JSON Web Token (JWT) that contains your merchant configuration, one-time encryption keys, and payment parameters.

Follow these steps to make a server-to-server call to the Sessions API to authenticate your merchant credentials and establish how the Unified Checkout front-end components will function:

1. Implement a server-to-server call to the Sessions API.
   This call should include parameters that define how Unified Checkout performs.

2. Handle the response from the Sessions API.
   The response will contain:

   - A transaction-specific public key for securing the transaction in the customer's browser.

   - An authenticated context description package that manages the payment experience on the client side, including available payment options, interface styling, and payment methods.

3. Store and manage the JSON Web Token (JWT) object, referred to as the *capture context*.
   This JWT contains all the functions compiled from the Sessions API response:

```
{
  "targetOrigins": ["https://merchant.com", "https://reseller.com:8443"],
  "locale": "en_US",
  "country": "US",
    "data": {
        "orderInformation": {
          "amountDetails": {
            "totalAmount": "21.00",
            "currency": "USD"
      }
    }
  }
}
```

   The **targetOrigins** array must include every origin that will host the SDK. The response JWT is passed to the client-side library.

   This capture context contains only the minimum required fields. For information about the components of the capture context and how to create one using the Sessions API, see Sessions API (on page 179). For a complete capture context with all available fields, see Example: Unified Checkout Complete Capture Context (on page 202).

Proceed to Step 3: Set Up the Client-Side Component (on page 22).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Step 3: Set Up the Client-Side Component

To add the payment interface to your e-commerce site, you need to set up the client-side component using the Unified Checkout JavaScript library. This setup involves two primary components:

- The button widget, which lists available payment methods for the customer.
- The payment acceptance page, which captures payment information from the cardholder. This can be integrated with your webpage or added as a sidebar.

This example shows a complete client-side integration:

```
async function launchCheckout() {
  try {
    const client = await VAS.UnifiedCheckout(sessionJWT);
    const checkout = await client.createCheckout();
    const result = await checkout.mount('#payment-buttons');

    // result contains the completed payment result JWT
    // Send result to your server for verification
    sendToServer(result);
  } catch (error) {
    if (error.name === 'UnifiedCheckoutError') {
      handleError(error.reason, error.message);
    }
  } finally {
    checkout.destroy();
    client.destroy();
  }
}

launchCheckout();
```

Follow these steps to create the client-side integration:

1. Load the Unified Checkout JavaScript library:

   ```
   <script
    src="https://apitest.cybersource.com/uc/v1/assets/1.0.0/UnifiedCheckout.js"
   ></script>
   ```

   Replace the domain with the production URL for live environments:

   - **Test**: `https://apitest.cybersource.com`
   - **Production**: `https://api.cybersource.com`

You must include the library in your webpage's HTML.

2. Initialize the SDK by calling `VAS.UnifiedCheckout()`. This returns a client instance:

```
const client = await VAS.UnifiedCheckout(sessionJWT);
```

Use the JWT obtained from the server-side setup in Step 2: Set Up the Server-Side Component (on page 20).

Initialization validates the JWT signature, checks that the current page origin matches `targetOrigins`, and prepares the SDK for use. If the JWT is invalid or expired, a `UnifiedCheckoutError` is returned.

3. Create a checkout to render a list of available payment methods and handles the payment flow:

```
const checkout = await client.createCheckout();
```

When you include `autoProcessing` and set it to `true`, `mount()` returns the completed payment result:

```
// Explicit auto-processing
const checkout = await client.createCheckout({ autoProcessing: true });
```

When you include it and set it to `false`, `mount()` returns a transient token that you pass to `checkout.complete()`:

```
// Explicit manual processing
const checkout = await client.createCheckout({ autoProcessing: false });
```

The default value of `autoProcessing` us `true` when a **completeMandate** is included in the session.

4. Mount the checkout by calling `mount()`. This attaches the payment UI to your page. The argument determines the display mode:

Sidebar Mode

The payment screen appears as an overlay sidebar. Pass a CSS selector for the payment button list, or omit it entirely:

```
// Full sidebar — buttons and payment screen both in sidebar
const result = await checkout.mount();

// Buttons embedded, payment screen in sidebar
const result = await checkout.mount('#payment-buttons');
```

Embedded Mode

Both the button list and payment screen render inline within your page layout:

```
const result = await checkout.mount({
  paymentSelection: '#payment-buttons',
  paymentScreen: '#payment-form'
});
```

Mount Result

When `autoProcessing` is enabled, `mount()` resolves with the completed payment result JWT once the customer finishes the payment flow.

When `autoProcessing` is disabled, `mount()` resolves with a transient token JWT. You then call **checkout.complete()** to finish the payment:

```
const checkout = await client.createCheckout({ autoProcessing:
 false });
const transientToken = await checkout.mount('#payment-buttons');

// Later, complete the payment
const result = await checkout.complete(transientToken);
```

Unmount

You can remove the payment UI from the page without destroying the checkout:

```
checkout.unmount();

// Later, mount again
const result = await checkout.mount('#payment-buttons');
```

For more information about setting up the client side, see Client-Side Set Up (on page 136). For information about handling errors on the client side, see Handle Errors (on page 231). Proceed to Step 4: Configure Unified Checkout (on page 26).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Step 4: Configure Unified Checkout

Proper configuration ensures that your checkout process aligns with your business needs and provides a smooth experience for your customers. You can configure the checkout process in the Business Center:

Select and configure the payment methods you support:

a. Log in to the Business Center and navigate to the Unified Checkout configuration section.

b. Select the payment methods you want to use:

- Credit and debit cards. See Cards (on page 33).
- eCheck/ACH Services. See eCheck/ACH Service (on page 39).
- Digital wallets such as Apple Pay, Click to Pay, and Google Pay. See Digital Wallets (on page 43).
- Alternative payment methods. See Alternative Payment Methods (on page 71).

> **Important:**
>
> You must configure the payment methods you want to use for each transacting MID.
>
> For information about which payment methods are supported on Unified Checkout, see Payment Methods (on page 32).

c. Configure the settings for each selected payment method.

For information about configuring Unified Checkout in the Business Center, see Configure the Unified Checkout Merchant Experience (on page 147).

Proceed to Step 5: Test Your Unified Checkout Integration (on page 27).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Step 5: Test Your Unified Checkout Integration

After configuring Unified Checkout, it's crucial to thoroughly test your integration to ensure it works correctly and provides a smooth checkout experience for your customers. This section outlines the steps to test your Unified Checkout integration.

1. Set up your test environment:
    a. Log in to the Business Center account using your test credentials.
    b. Switch to the test environment if not already in test mode.
    c. Set up a test website or application that integrates Unified Checkout.
2. Use test card numbers to simulate different payment scenarios.

    For more test payment data, see Unified Checkout Test Cards (on page 223).
3. Test different payment scenarios:
    - Successful transactions
    - Declined transactions
    - 3-D Secure authentication, if applicable
    - Different card brands
    - Digital wallet payments, if configured
4. Verify that the capture context object that you get from the sessions API is correct and that your integration can handle tokens.

    Ensure that your integration correctly handles the capture context and transient tokens throughout the payment process.
5. Test error handling and edge cases.

    Simulate various error scenarios to ensure your integration gracefully handles and reports errors to the user.
6. Verify webhook notifications, if configured.

    If you set up webhook notifications, ensure that your system correctly receives and processes them for various transaction events. For information about configuring your webhook notifications, see Webhooks Support (on page 176).

After completing these testing steps, you should have confidence in your integration. Remember to test in both the test and production environments before going live.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

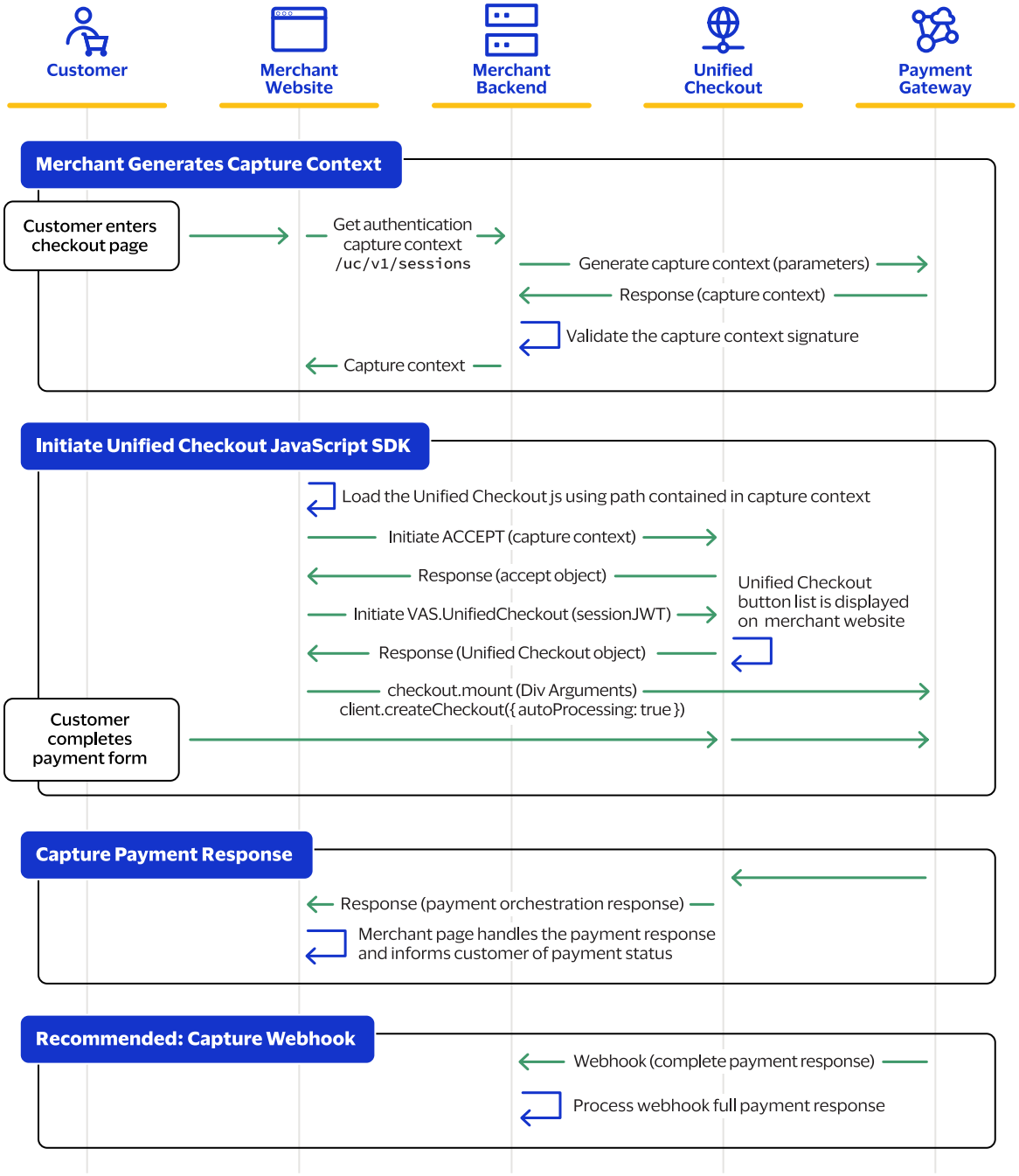
Unified Checkout Flow

To integrate Unified Checkout into your platform, you must follow several integration steps. This section gives a high-level overview of how to integrate and launch Unified Checkout on your webpage and process a transaction. You can find the detailed specifications of the APIs later in this document.

Information that is captured by Unified Checkout, including the billing and shipping address, can be retrieved using the Payment Details API.

The figure below shows the Unified Checkout payment flow using the Sessions API to generate the capture context:

Unified Checkout Payment Flow



For more information on the specific APIs referenced, see these topics:

- Sessions API (on page 179): This generates the capture context and determines what fields are displayed to the customer in the UI during checkout.
- Payment Details API (on page 241): This API can be used to retrieve personally identifiable information that is associated with a Unified Checkout transient token, such as the cardholder name and billing and shipping details, without retrieving payment credentials.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Payment Methods

This section describes the payment methods you can use in your Unified Checkout integration. After you successfully integrate one payment method, you can add another from the same category with minimal adjustments to your existing configuration.

These payment methods are available:

- Cards (on page 33)
- eCheck/ACH Service (on page 39)
- Apple Pay (on page 45)
- Google Pay (on page 48)
- Click to Pay (on page 52)
- Paze (on page 68)
- Online Bank Transfers (on page 72)
- Buy Now, Pay Later (on page 87)
- Post-Pay Reference Payments (on page 94)
- PayPal (on page 101)
- Venmo (on page 104)

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Cards

Unified Checkout accepts multiple card types including global networks such as Visa, Mastercard, and American Express. Unified Checkout also accepts local schemes such as Cartes Bancaires in France, EFTPOS in Australia, and PayPak in Pakistan.

Card Support

Support for card brands varies based on the payment method for these services:

- Payments
- Decision Manager
- Payer Authentication

This table shows which card types are accepted for each payment method and which region:

Card Brand by Region and Payment Method

Region	Card Brand	Manual Card Entry	Apple Pay	Click to Pay	Google Pay	Paze
Asia Pacific	China UnionPay	✓	✗	✗	✗	✗
Asia Pacific	EFTPOS	✓	✗	✗	✗	✗
Asia Pacific	JCB	✓	✗	✗	✗	✗
CEMEA	mada	✓	✓	✗	✗	✗
CEMEA	Meeza	✓	✓	✗	✗	✗
CEMEA	Jaywan	✓	✓	✗	✗	✗
CEMEA	PayPak	✓	✗	✗	✗	✗
Europe	Cartes Bancaires	✓	✓	✗	✗	✗
Global	American Express	✓	✓	✓	✓	✗
Global	Diners Club	✓	✗	✗	✗	✗
Global	Mastercard	✓	✓	✓	✓	✓
Global	Visa	✓	✓	✓	✓	✓

Card Brand by Region and Payment Method (continued)

Region	Card Brand	Manual Card Entry	Apple Pay	Click to Pay	Google Pay	Paze
Global and Europe	Maestro	✓	✗	✗	✗	✗
Latin America	Carnet	✓	✗	✗	✗	✗
Latin America	ELO	✓	✗	✗	✗	✗
US and Canada	Discover	✓	✗	✗	✗	✗
US and Canada	JCrew	✓	✗	✗	✗	✗

This table shows which card types are supported for each complete mandate feature by region.

Card Support for the Complete Mandate

Region	Card Brand	Authorization	Payer Authentication	Decision Manager	Token Create by Token Management Service
Asia Pacific	China UnionPay	✓	✓	✓	✓
Asia Pacific	EFTPOS	✓	✓	✓	✓
Asia Pacific	JCB	✓	✓	✓	✗
CEMEA	mada	✓	✓	✓	✓
CEMEA	Meeza	✓	✓	✓	✗
CEMEA	Jaywan	✓	✓	✓	✗
CEMEA	PayPak	✓	✓	✓	✗
Europe	Cartes Bancaires	✓	✓	✓	✓
Global	American Express	✓	✓	✓	✓
Global	Diners Club	✓	✓	✓	✓

Card Support for the Complete Mandate (continued)

Region	Card Brand	Authorization	Payer Authentication	Decision Manager	Token Create by Token Management Service
Global	Mastercard	✓	✓	✓	✓
Global	Visa	✓	✓	✓	✓
Global and Europe	Maestro	✓	✓	✓	✓
Latin America	Carnet	✓	✗	✓	✓
Latin America	ELO	✓	✓	✓	✓
US and Canada	Discover	✓	✓	✓	✓
US and Canada	JCrew	✓	✗	✓	✓

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Pay with Token

You can use Unified Checkout to pass through a single token ID to be shown within the Unified Checkout UI. To display a payment instrument in the Unified Checkout UI, you must include `TMS_TOKEN` as an allowed payment type in the **allowedPaymentTypes** field object and the details of the Token Management Service (TMS) token in the **paymentConfigurations** field object in the capture context request:

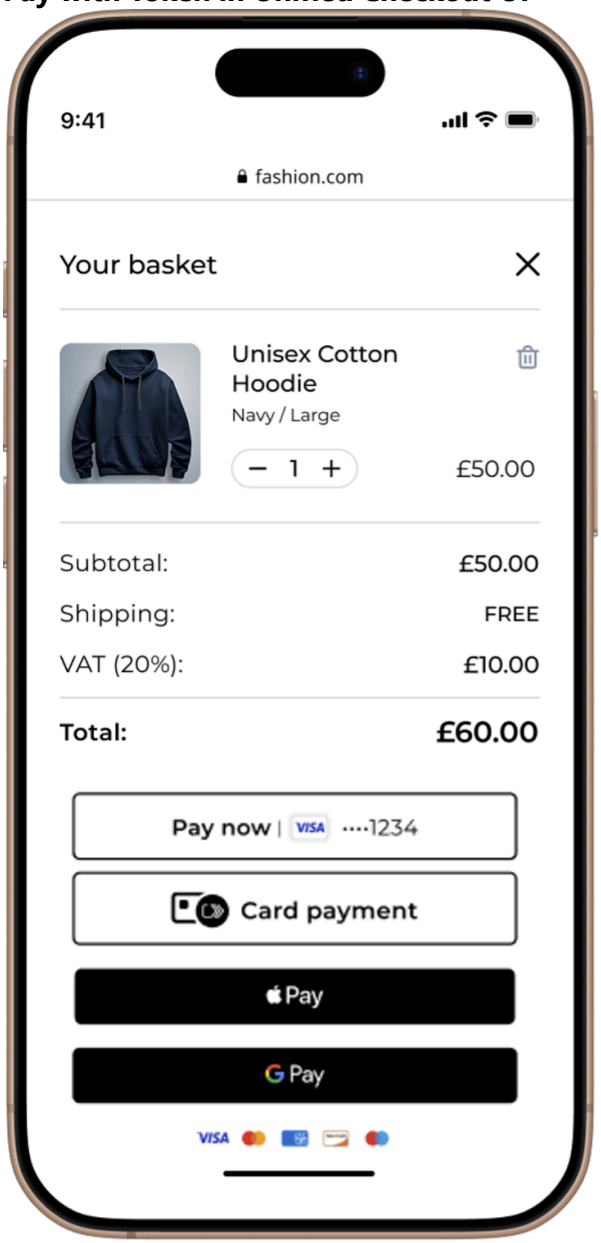
```
{
"allowedPaymentTypes": [

    "PANENTRY",
    "TMS_TOKEN"
 ]
```

```
},
"paymentConfigurations": {
    "TMS_TOKEN": {
      "instrumentIdentifiers": [
        {
          "id": "4B1BCB328D52ED86E063AF598E0A99A5"
        }
      ]
    }
}
```

This is an example UI with a payment instrument:

Pay with Token in Unified Checkout UI



You can use these token types to pay with a token in Unified Checkout:

Customer Tokens

When you include the customer token, your UI displays the default payment instrument that is inked to a customer. To display a customer token, you must include the **paymentConfigurations.TMS_TOKEN.customer.id** field in your Sessions API request.

Important: When you include a customer token ID here with **tokenCreate** for a **paymentInstrument** or **instrumentIdentifier**, the complete mandate creates a new payment instrument or instrument identifier within the level of the customer token that you provide.

```
{
"allowedPaymentTypes": [
    "PANENTRY",
    "TMS_TOKEN"
 ]
},
"paymentConfigurations": {

    "TMS_TOKEN": {
      "customer": [
        {
          "id": "4B1BCB328D52ED86E063AF598E0A99A5"
        }
      ]
    }
}
```

Instrument Identifier Tokens

When you include an instrument identifier token, your UI displays the payment credential that is associated with the specified instrument identifier. To display an instrument identifier token, you must include the **paymentConfigurations.TMS_TOKEN.instrumentIdentifiers.id** field in your Sessions API request.

```
{
"allowedPaymentTypes": [
    "PANENTRY",
    "TMS_TOKEN"
 ]
},
"paymentConfigurations": {

    "TMS_TOKEN": {
```

```
      "instrumentIdentifiers": [
        {
          "id": "4B1BCB328D52ED86E063AF598E0A99A5"
        }
      ]
    }
}
```

Payment Instruments

When you include a payment instrument, your UI displays the payment instrument that is associated with the specified payment instrument token identifier. To display a payment instrument, you must include the **paymentConfigurations.TMS_TOKEN.paymentInstruments.id** field in your Sessions API request.

```
{
"allowedPaymentTypes": [
    "PANENTRY",
    "TMS_TOKEN"
 ]
},
"paymentConfigurations": {

    "TMS_TOKEN": {
      "paymentInstruments": [
        {
          "id": "4B1BCB328D52ED86E063AF598E0A99A5"
        }
      ]
    }
}
```

Important: To make a new payment instrument or instrument identifier under an existing customer during the complete mandate, you must meet these requirements:

- You must include the customer token ID as the **TMS_TOKEN.customer.id** field in the **paymentConfigurations** field object.
- `TMS_TOKEN` must be included in the **allowedPaymentTypes** field object.
- **tokenCreate** must be set to `true` and `paymentInstrument` and `instrumentIdentifier` must be included as values in the **tms.tokenTypes** field array. For example:

```
"tms": {
      "tokenCreate": true,
      "tokenTypes": [
```



```
            "paymentInstrument",
            "instrumentIdentifier",
        ]
    }
```

When you meet these requirements, a new payment instrument or instrument identifier is created under the specified customer token.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

eCheck/ACH Service

Unified Checkout supports the acceptance of eCheck information. Sensitive eCheck data is securely captured and replaced with a token. Acceptance of eCheck information enables merchants to collect funds from a customer's bank account through both the ACH service and eCheck service (US only) for either of these flows:

- ACH services are a set of connections composed of the legacy gateway solutions where Cybersource serves as the gateway.
- eCheck, the new service on Payments 2.0, is the acquirer solution where Cybersource is the acquirer.

Unified Checkout replaces these eCheck information fields in your payment input form:

- Routing number
- Account number
- Account type (non-sensitive)

Related information

Getting Started with REST
Response Codes
API Field Reference Guide

API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Enrolling in eCheck/ACH Services

Unified Checkout can accept bank account payments using the eCheck product. To accept eCheck payments through Unified Checkout, you must have the eCheck processing service enabled. To request access to eCheck processing and enable eCheck, you must submit an application in the Business Center. Once your application is approved, you can accept eCheck payments.

For step-by-step instructions on enrolling and enabling eCheck, see the "Getting Started with the eCheck Service" section of the eCheck Merchant Guide. If eCheck is not listed in the Available Products section in the Business Center, you must contact your portfolio owner to enable your account to apply for eCheck.

Important: If you have a business account or a financial relationship with Bank of America, Wells Fargo, or Chase, and you would like them to process your transactions, you must contact our Sales or Support team for more information on our ACH product.

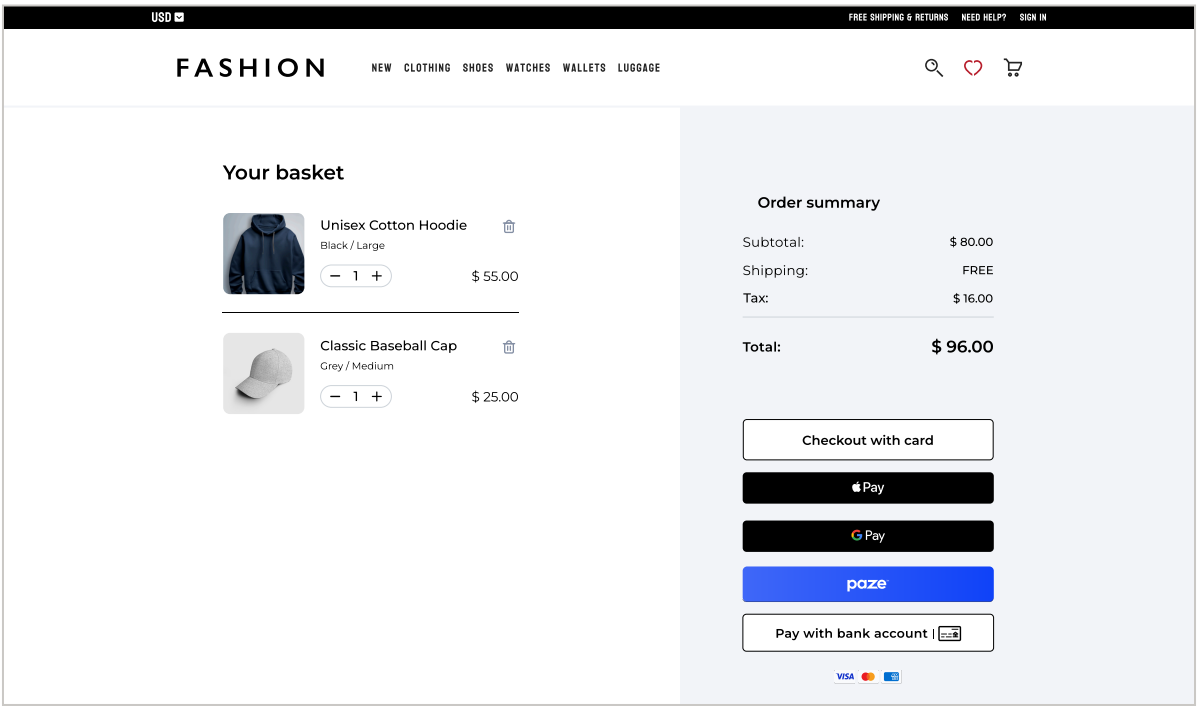
Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

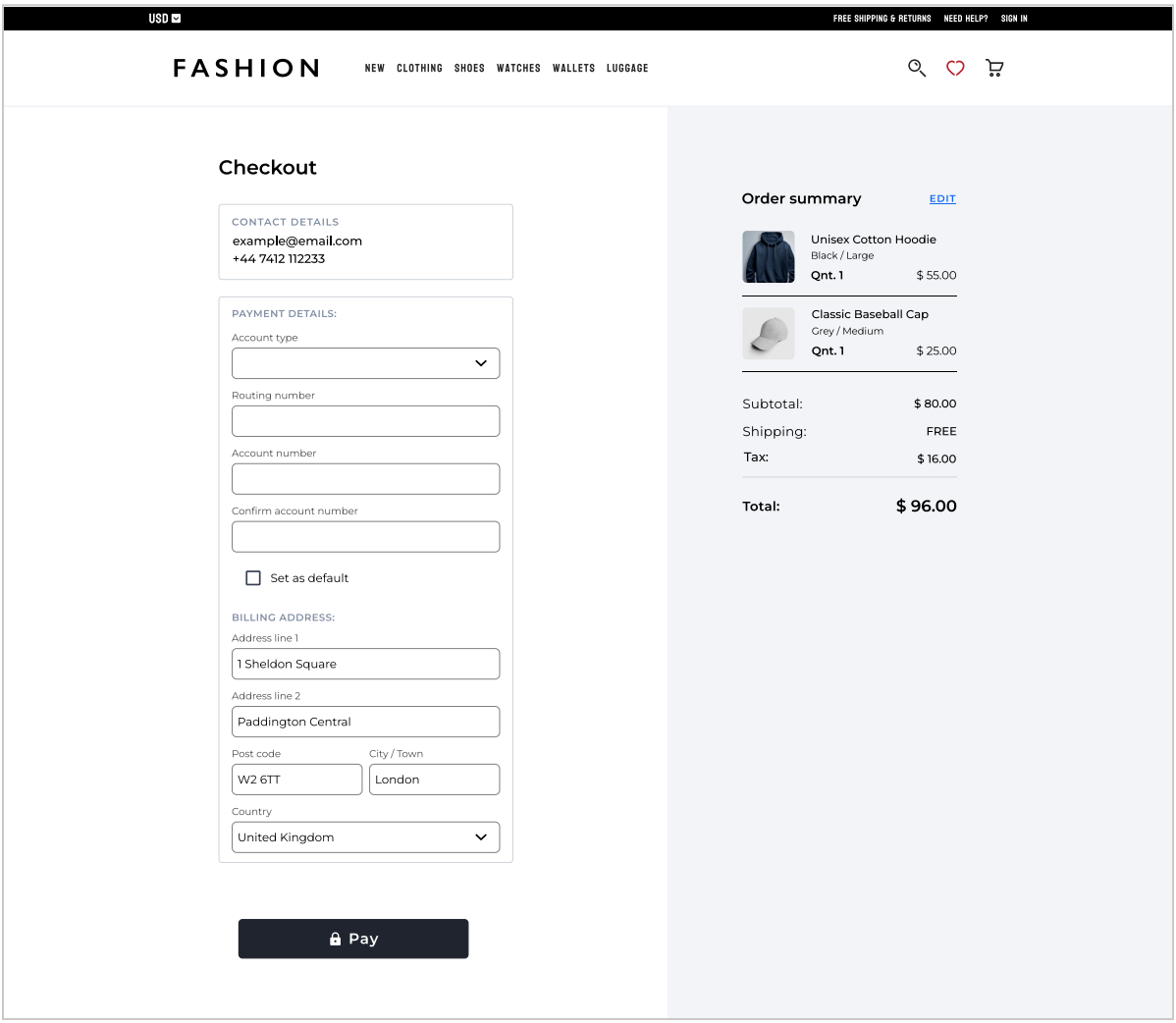
Pay with eCheck/ACH Service UI

These screen captures show the sequence of events your customer can expect when completing a payment with the eCheck/ACH service.

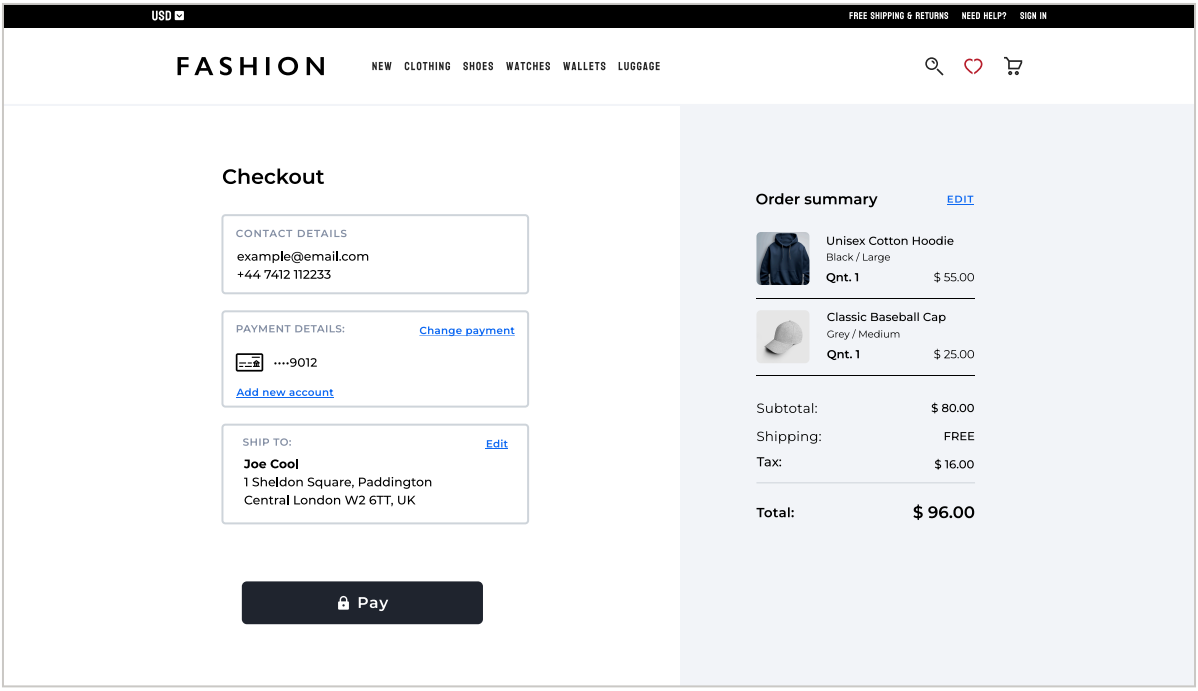
eCheck/ACH Service Account Order Summary



Pay with eCheck/ACH Service Checkout



Pay with eCheck/ACH Service Review and Confirm



Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Digital Wallets

Digital wallets are secure applications or services that enable users to store payment details, such as debit or credit cards electronically.

Digital wallets such as Apple Pay, and Google Pay are accessible with smartphones, computers, or even directly in web browsers. Digital wallets allow customers to pay for goods and services both online and in physical stores without a physical card, often using biometrics, a device passcode, or wallet login, and approve the payment in just a few clicks.

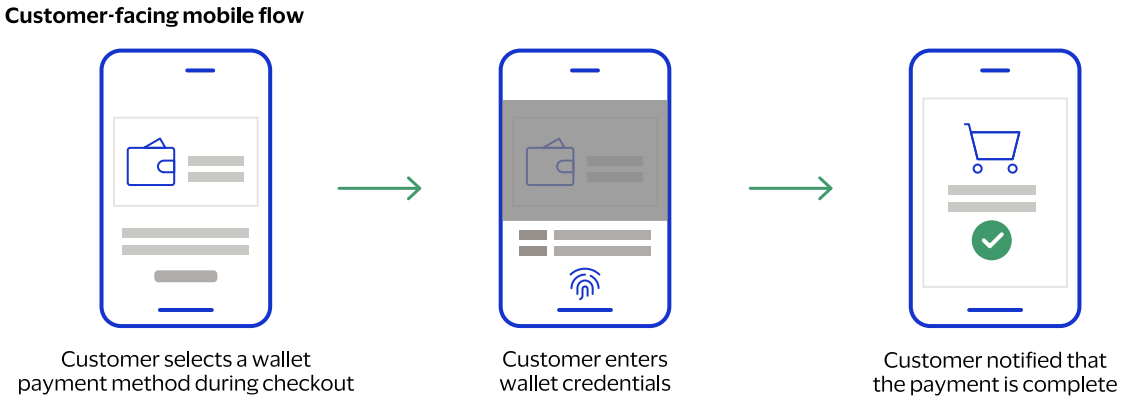
For online transactions, the wallet securely passes a payment token to the merchant or payment processor. This means that you do not handle sensitive customer data directly. This reduces friction at checkout, improves conversion rates, and enhances security through built-in authentication and tokenization.

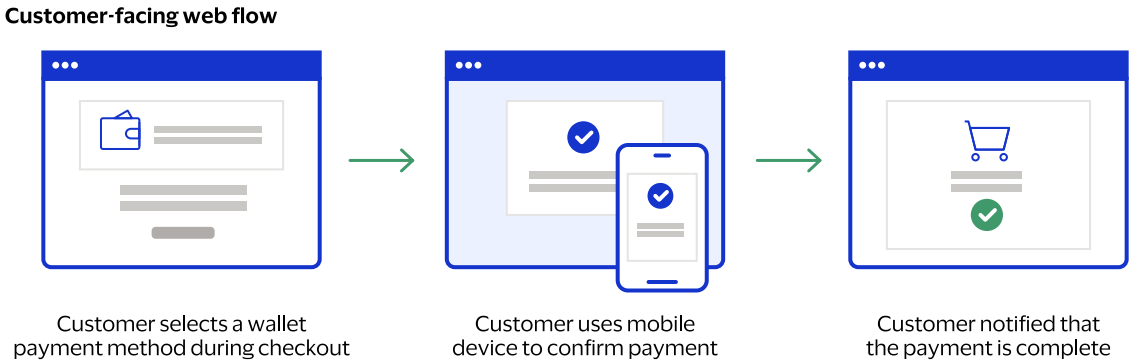
Wallets are best suited for one-time online purchases and express checkout experiences. Support for subscriptions and recurring payments varies by wallet, so you must ensure compatibility if future charges or merchant-initiated transactions are required.

Unified Checkout supports these wallet-based payment options:

- Apple Pay
- Google Pay
- Click to Pay
- Paze

This is how payments with digital wallets work:





Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

Apple Pay

Apple Pay is a digital payment solution that enables your customers to make secure and convenient purchases without requiring them to enter their card details or shipping information. This section includes information about accepting Apple Pay payments with your Unified Checkout integration.

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Enrolling in Apple Pay

Apple Pay is a digital payment service that enables users to make secure and convenient transactions using their Apple devices. Users can add their credit or debit cards to the Wallet app and use them to pay online or in apps in a safe and convenient consumer experience.

To enable Apple Pay you must first host a public certificate on your web page and then pass your merchant name and domain name to Apple. Apple crawls out to your web page to validate the presence of this certificate to ensure the web pages are properly vetted and registered with Apple.

Follow these steps to validate your domain and enroll in Apple Pay:

1. Navigate to **Payment Configuration > Unified Checkout**.
2. In the Apple Pay section, click **Set Up**.
3. Follow the link to download the certificate.
4. Upload the *apple-developer-merchantid-domain-association* certificate file to your web server at:
   `/.well-known/apple-developer-merchantid-domain-association`
   You must verify that the file is accessible through HTTPS. You can validate this by visiting `https://<your-domain>/.well-known/apple-developer-merchantid-domain-association`.

5. Click **Verify Domain**.

6. Enter the domain name where you are hosting Apple Pay. This must be the same domain to which you uploaded the public certificate.
   Your domain is now verified for Apple Pay.

Important: In order to run an end-to-end test of the Apple Pay service on Unified Checkout, you must perform additional setup steps. See Preparing a Device for Testing Apple Pay on Unified Checkout (on page 46).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Preparing a Device for Testing Apple Pay on Unified Checkout

To run an end-to-end test of the Apple Pay service on Unified Checkout, you must prepare an Apple test device by loading Apple Pay test cards onto the device.

Follow these steps to prepare your Apple test device for end-to-end testing:

1. Make sure your Apple Developer account is configured for Apple Pay.

2. Register your Apple Pay test device with Apple.

3. Load Apple Pay test cards onto your Apple test device.
   The Apple Developer center provides the instructions in the Sandbox Testing page for Apple Pay:

   a. Follow the steps described in *Create a Sandbox Tester Account*.

   b. Follow the steps described in *Adding a Test Card Number*.

Related information

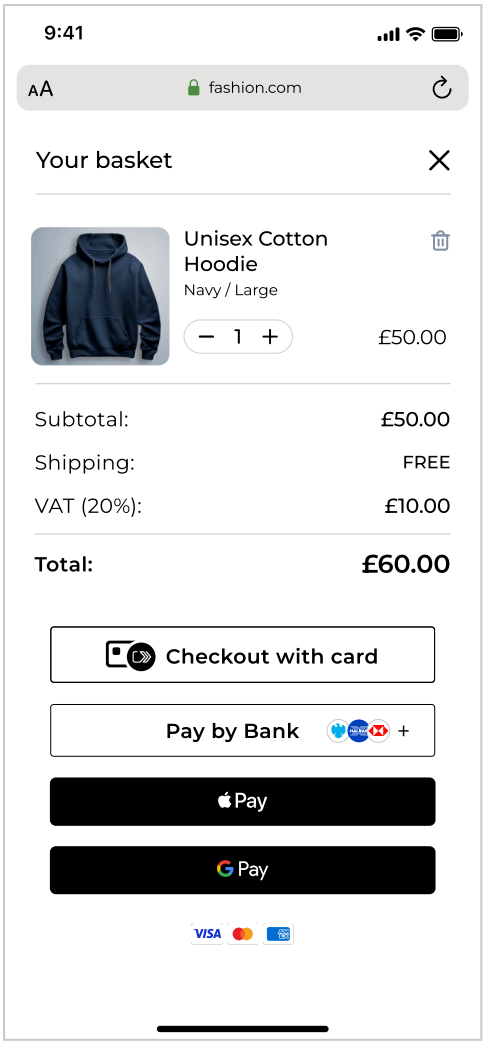
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

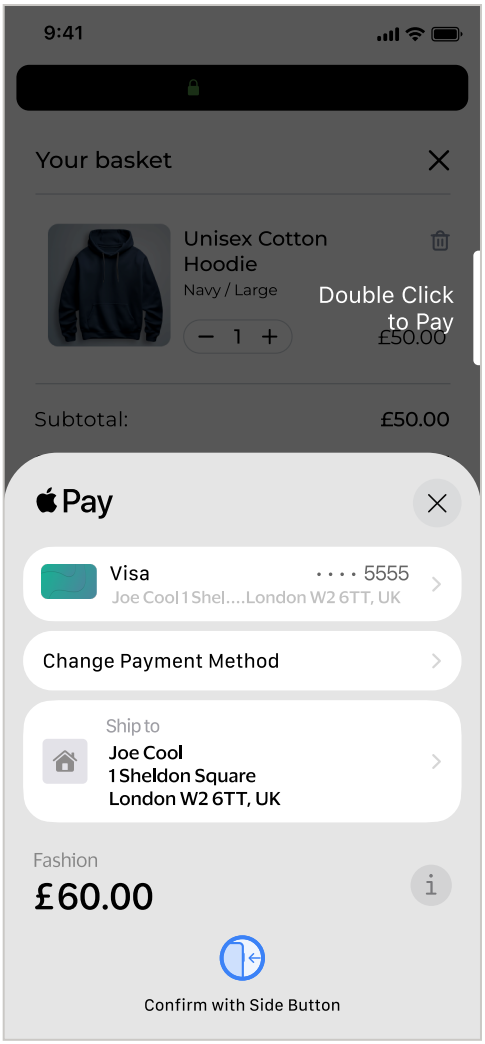
Apple Pay UI

These screen captures show the sequence of events your customer can expect when completing a payment with Apple Pay.

Apple Pay UI



Apple Pay in the payment method list



Apple Pay checkout

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

Google Pay

Google Pay is a simple, secure in-app mobile and Web payment solution. This section includes information about accepting Google Pay payments with your Unified Checkout integration.

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Enrolling in Google Pay

Google Pay is a digital payment product offered by Google through Chrome browsers and Android devices.

Follow these steps to enroll in Google Pay on Unified Checkout:

1. Navigate to **Payment Configuration > Unified Checkout**.
2. In the Google Pay section, click **Set Up**.
3. Enter your business name.
4. Click **Submit**.
   You can now accept digital payments with Google Pay.

Important: When you enable Google Pay on Unified Checkout, you can specify an optional parameter that defines the types of credentials that Google Pay sends you. See Managing Google Pay Authentication Types (on page 49).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Managing Google Pay Authentication Types

Additional controls are available for Google Pay on Unified Checkout. When you enable Google Pay on Unified Checkout, you can specify optional parameters that define the types of card authentication you receive from Google Pay.

To manage the types of credentials that Google Pay sends, use this expanded payment type object within the **allowedPaymentTypes** section of the sessions request:

```
"paymentConfigurations": {
    "GOOGLEPAY": {
      "allowedAuthMethods": "<authentication type>"
    }
```

The expanded payment type object has these parameters:

- **type**: Defines the type of payment option.
- **options**: Contains specific payment types parameters.

  For Google Pay, use the new data element **allowedAuthMethods** within the **options** section of the payment types object to specify the authentication type you will receive from Google Pay. Possible values:

  - `PAN_ONLY`: Google returns primary account number (PAN) values
  - `CRYPTOGRAM_3DS`: Google returns fully authenticated network token values.

  By default, Google sends both authentication types.

  Important: When the complete mandate is used and Google Pay does not authenticate the transaction, then Unified Checkout completes the authentication request as part of the complete mandate.

REST Example: Specify Only PAN Authentication Accepted from Google

This sessions request example specifies that Google Pay is to send only PAN values.

```
"allowedPaymentTypes": [
  "GOOGLEPAY"
],
"paymentConfigurations": {
    "GOOGLEPAY": {
      "allowedAuthMethods": [
        "PAN_ONLY",
        "CRYPTOGRAM_3DS"
      ]
    }
  }
```

REST Example: Simple Google Pay Request

This sessions request example specifies that Google Pay can send all authentication types. This can be enabled and configured in the Business Center. For information about configuring your allowed payment types, see Configure Payment Options (on page 152).

```
"allowedPaymentTypes": [
  "PANENTRY",
  "GOOGLEPAY",
  "CLICKTOPAY",
  "PAZE",
  "CHECK"
]
```

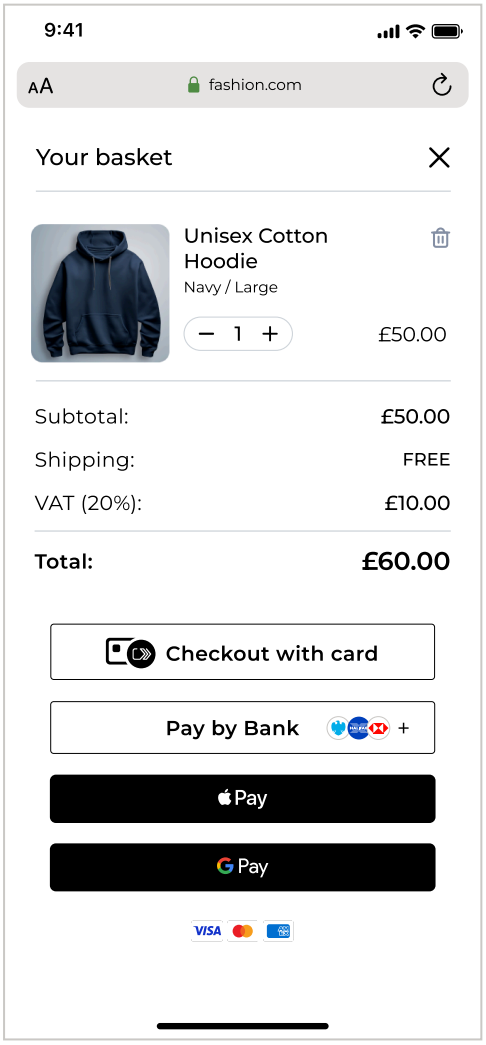
Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

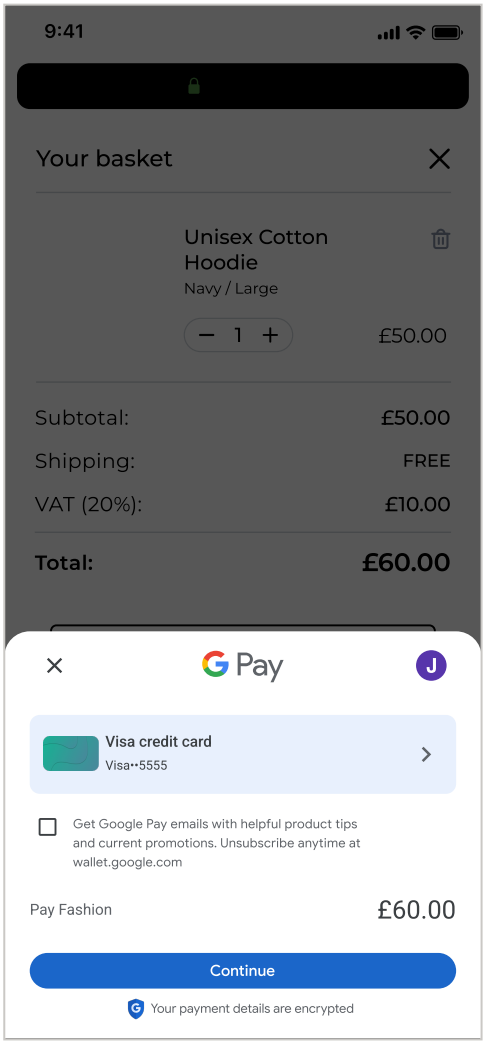
Google Pay UI

These screen captures show the sequence of events your customer can expect when completing a payment with Google Pay.

Google Pay UI



Google Pay in the payment method list



Google Pay checkout

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Click to Pay

Click to Pay is a secure online checkout method that enables customers to make purchases without entering their payment details for every purchase. This section includes information about accepting Click to Pay payments with your Unified Checkout integration.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Enabling Click to Pay in the Business Center

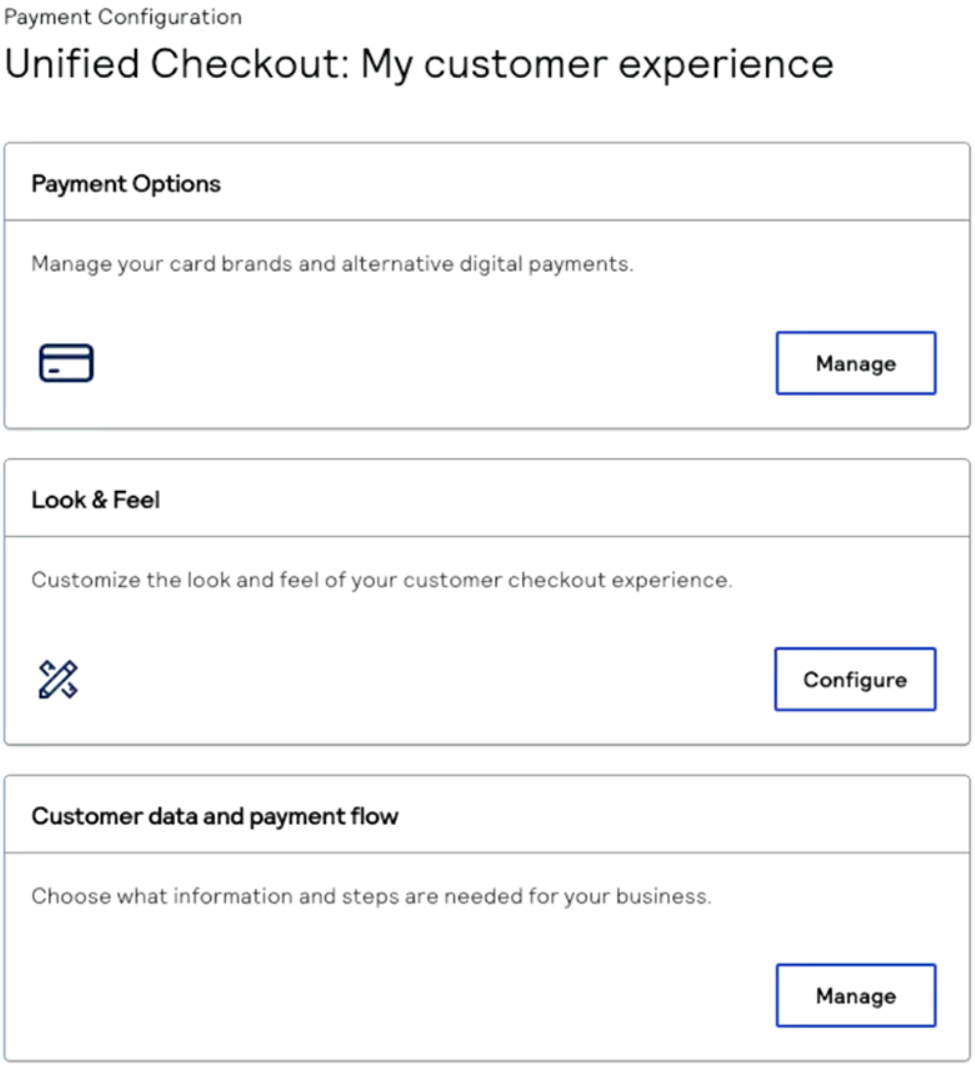
To begin your integration, you must first enable Click to Pay. Click to Pay is a digital payment solution that allows customers to pay with their preferred card network and issuer without entering their card details on every website. Customers can use Visa, Mastercard, and American Express cards to streamline their purchase experience. Click to Pay provides a fast, secure, and consistent checkout experience across devices and browsers.

Follow these steps to enable in Click to Pay on Unified Checkout:

1. Log in to the Business Center:
   Test URL: `https://businesscentertest.cybersource.com/ebc2`
   Production URL: `https://businesscenter.cybersource.com`
   If you are unable to access this page, contact your sales representative.

2. In the Business Center, go to the left navigation panel and choose **Payment Configuration** > **Unified Checkout**. The Unified Checkout customer experience page appears:

Unified Checkout Customer Experience



3. In the Payment Options section, click **Manage**. The Payment Options page appears.

4. Click **Manage** next to Click to Pay. The Click to Pay configuration page appears.
5. Enter your business name and website URL.
6. Click **Submit**.

> **Important:** Click to Pay uses network tokenization for transactions. These network tokens are stored in the vault of the token requestor ID (TRID) for the card scheme.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Set Up Customer Authentication for Click to Pay

Follow these steps to use the Business Center to enable customer authentication through Click to Pay. Authentication methods differ in each region and are dependent on the issuer, the cardholder device, and the Click to Pay configuration. Click to Pay authentication is available for Visa and Mastercard card types.

> **Important:** This authentication method is different from the authentication that is completed entirely in the Cybersource platform. To authenticate within the Cybersource platform, you must include the **consumerAuthentication** field in your request.

These authentication methods are available:

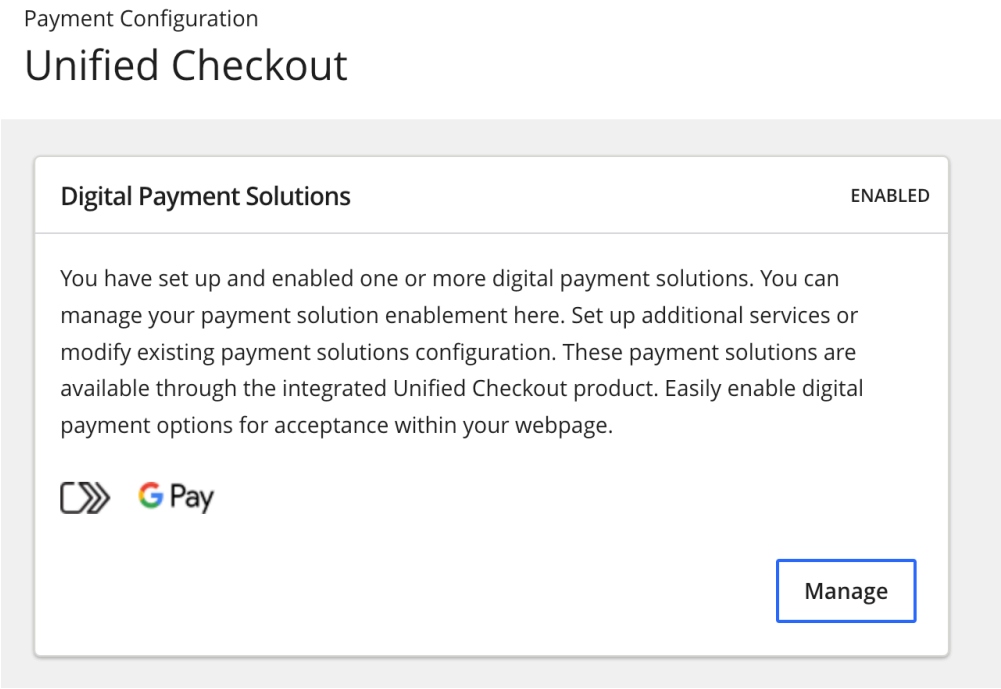
- 3-D Secure
- FIDO/Passkey
- Card verification value (CVV)
- One-time password (OTP)

Important: After you complete these steps, the card issuer determines which authentication method to use. When the issuer determines that they will authenticate, they authenticate each Click to Pay transaction through the appropriate method. This may be a frictionless authentication or the customer may need to provide more information when required by the issuer.

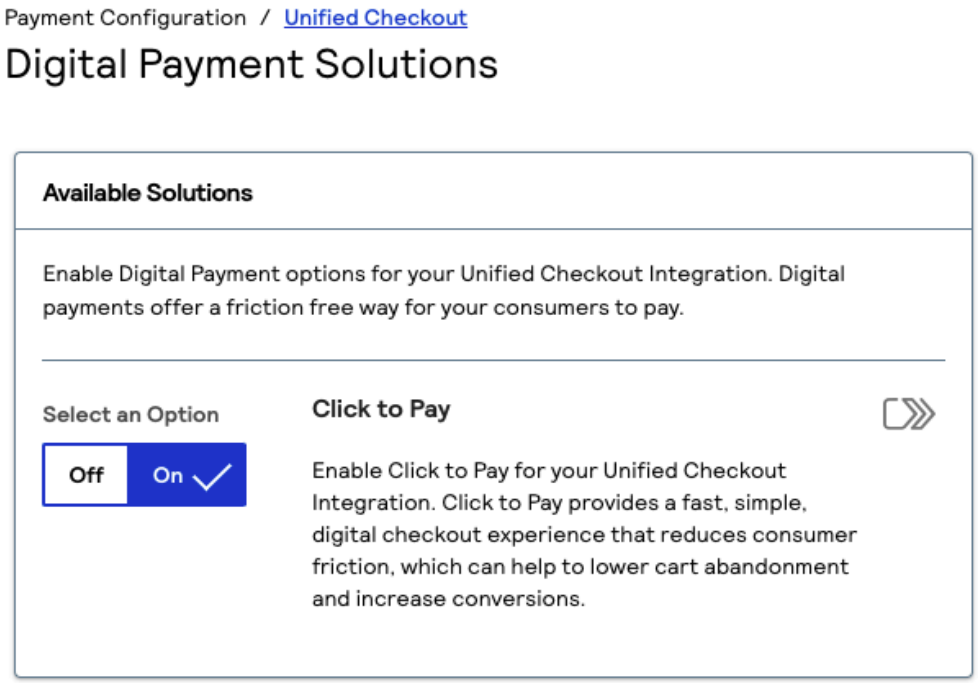
Important: Click to Pay authentication is not the same as consumer authentication using the complete mandate. See Test Authentication (on page 229).

1. Log in to the Business Center:
   Test URL: `https://businesscentertest.cybersource.com/ebc2`
   Production URL: `https://businesscenter.cybersource.com`
   If you are unable to access this page, contact your sales representative.

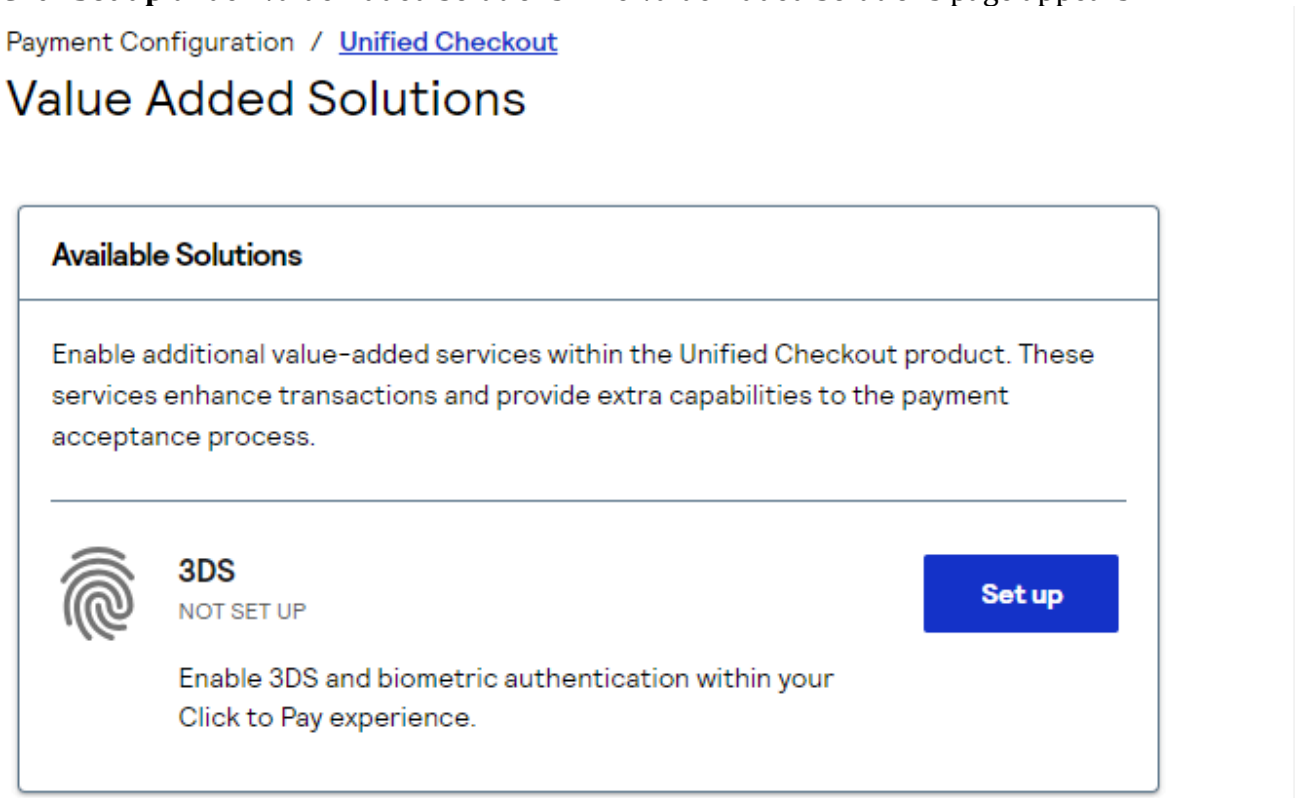
2. In the Business Center, go to the left navigation panel and choose **Payment Configuration** > **Unified Checkout**.
   You must have Click to Pay enabled as a digital payment method in order to use this method of authentication. Click **Manage** to view the digital payment methods that you have enabled.



   If Click to Pay is not enabled, click **On** next to Click to Pay.

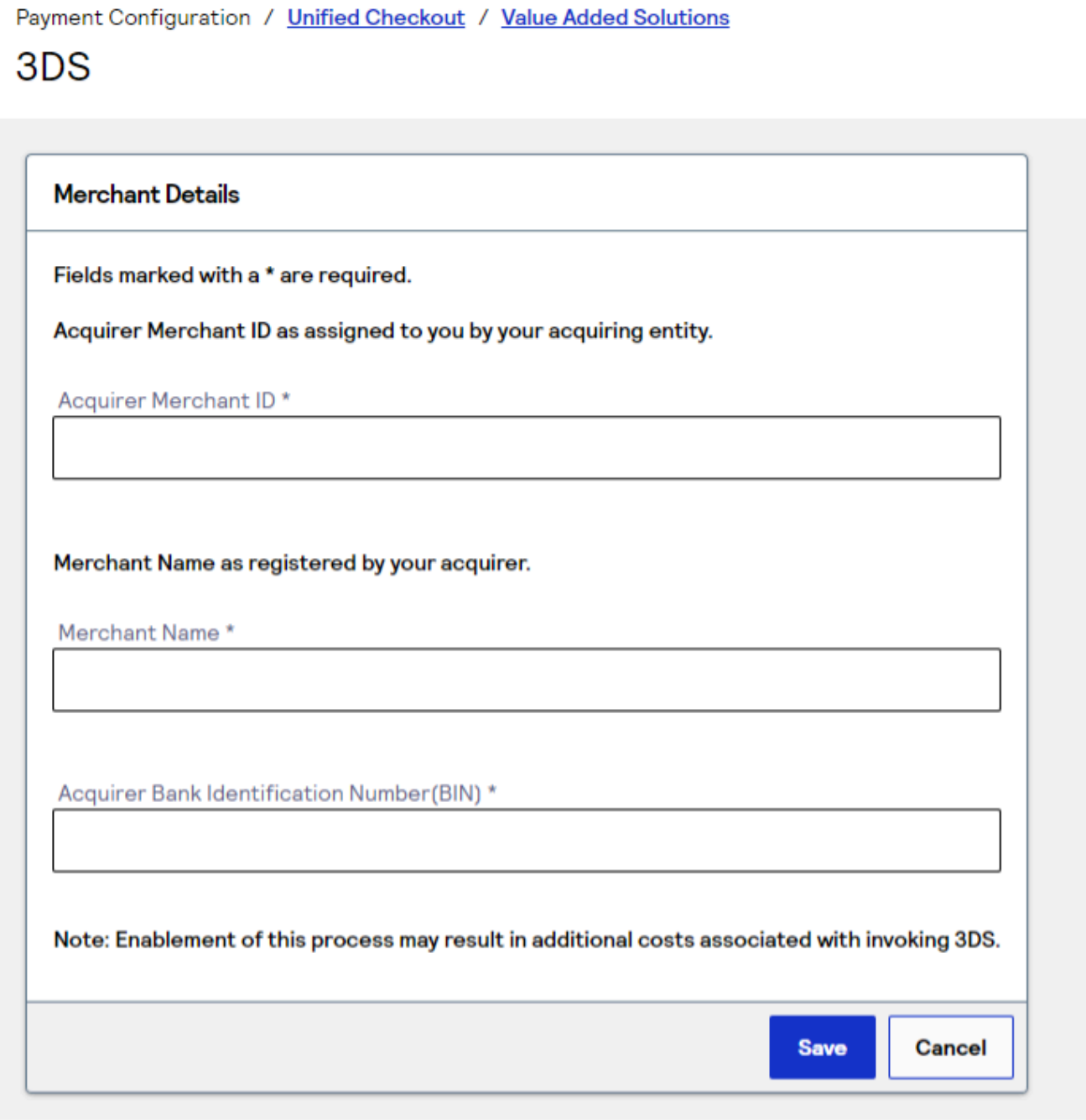


3. Click **Set up** under Value Added Solutions. The Value Added Solutions page appears.



4. Click **Set up** to set up 3-D Secure. The 3DS page appears.

5. Enter the required information in the Merchant Details section. You must enter the information that is provided to you by your acquirer or processor.



This completes the authentication setup for the entered acquirer merchant ID and BIN. If you do not know what these values are, you must contact your acquirer. Completing this information enables Cybersource to send Visa and Mastercard the information that is required for authentication.

Important: Charges for 3-D Secure may apply. You must speak with your acquirer for more information about the charges associated with 3-D Secure.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide

API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

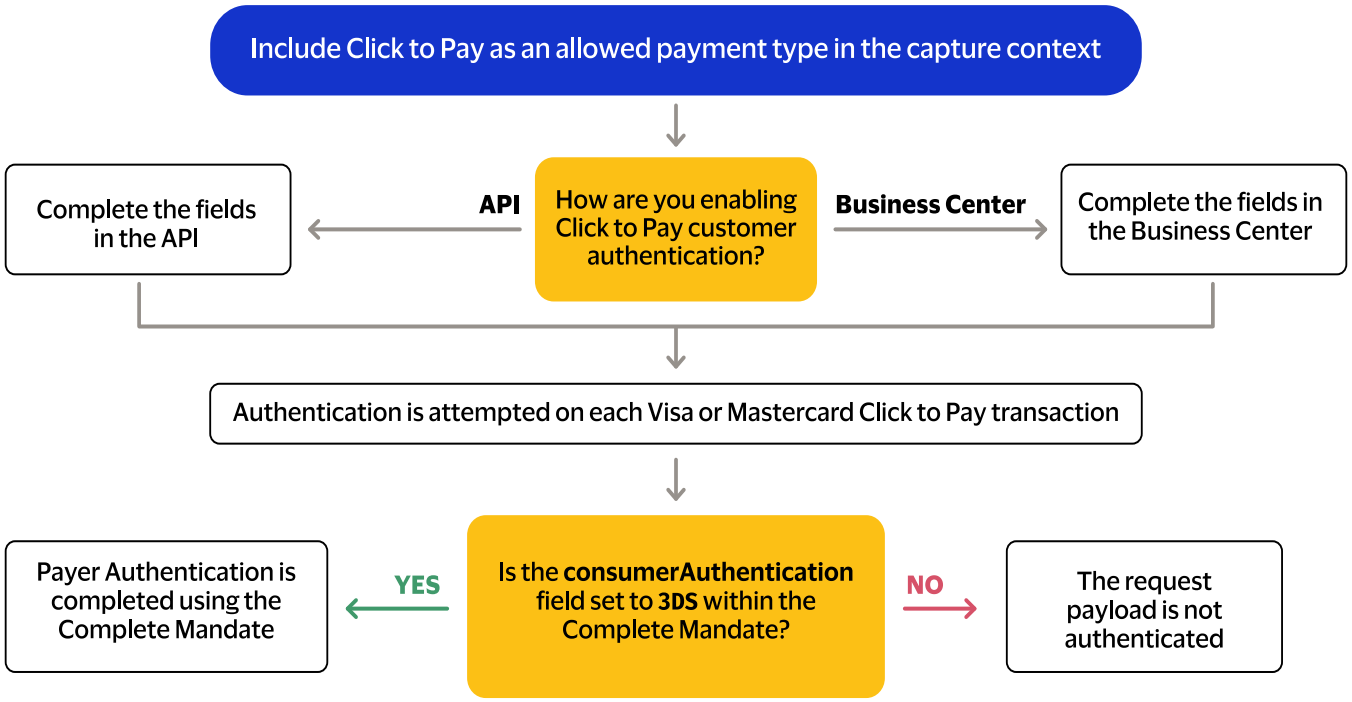
Click to Pay Customer Authentication

When you enable customer authentication through Click to Pay, you give Cybersource permission to send Visa and Mastercard the required authentication information for each transaction. When the customer completes a transaction using a Visa or Mastercard card that is already stored in Click to Pay, authentication is managed within Click to Pay.

Click to Pay authentication is only available for Visa branded cards that are tokenized with Click to Pay. If Click to Pay does not authenticate the transaction, but you are using the complete mandate with the **consumerAuthentication** field set to `3DS`, authentication is attempted as part of this request. When you do not use the complete mandate, you must check the result of the **cardholderAuthenticationStatus** field in the transient token and request Payer Authentication directly when it is required.

Important: American Express card brands cannot be authenticated through Click to Pay customer authentication.

Authentication Flow



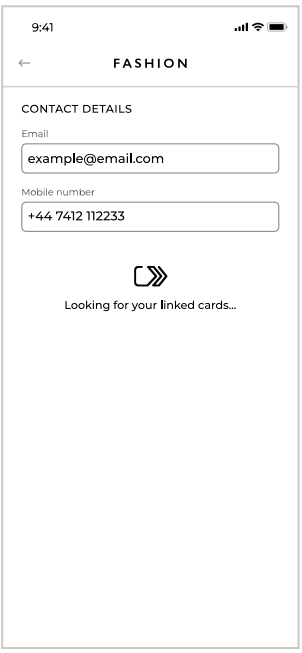
Related information

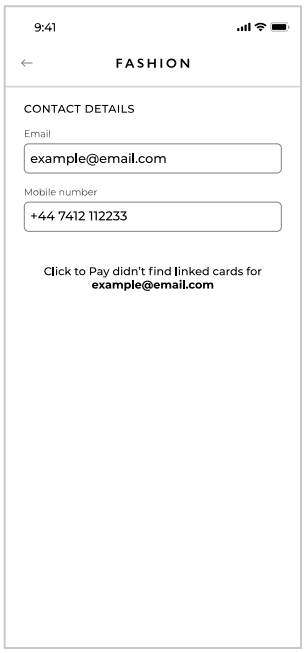
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

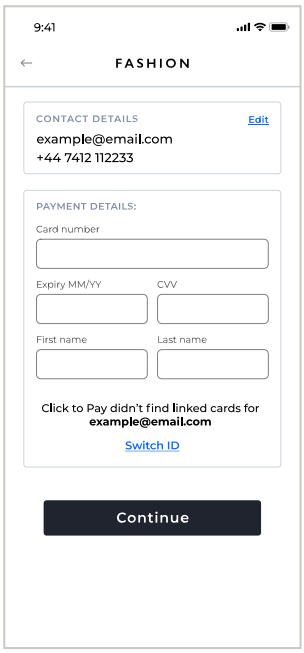
Click to Pay UI

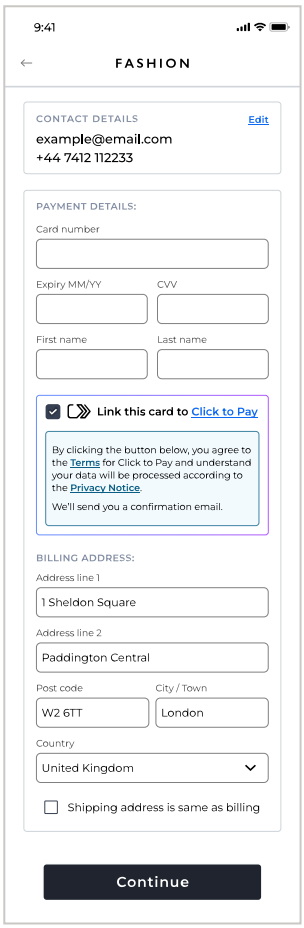
These screen captures show the sequence of events your customer can expect when completing a payment with Click to Pay.

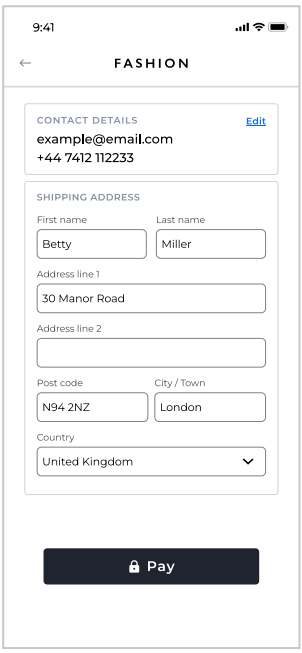
Click to Pay UI











Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Click to Pay UI Guidelines

The UI that is built in Unified Checkout for Click to Pay is built based on the EMV Click to Pay XC Guidelines V1.1. Unified Checkout has simplified the integration of the UI. The only UI work that you must complete is the placement of the payment option.

Important: You must include Click to Pay as one of the presented payment methods and not as a separate payment method.

Unified Checkout captures all card details that are manually entered by the cardholder. This enables the cardholder to enroll in Click to Pay and removes the requirement for the cardholder to manually enter their card details the next time they check out.

Unified Checkout provides a standard payment label in the Unified Checkout JavaScript that is loaded in your checkout page. One of these scenarios occurs when the cardholder selects the button:

- The cardholder is recognized.
- The cardholder is not recognized but has a Click to Pay account.
- The cardholder does not have a Click to Pay account.

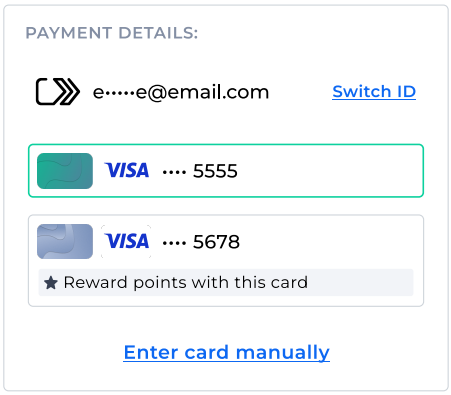
You can also trigger the Unified Checkout flow using a custom button. If you are using your own custom button, your payment button or widget must display the Click to Pay image for the cardholder. For information about a custom button, see JavaScript Example: Client-Defined Trigger for Click to Pay or PAN Entry (on page 139).

Important: Your implementation consultant will ask you for a mock-up of your payment flow for confirmation that it is compliant with the Click to Pay UI design standards.

Recognized Click to Pay Customer

The cardholder is presented with their stored Click to Pay cards in the UI when they are on a recognized device:

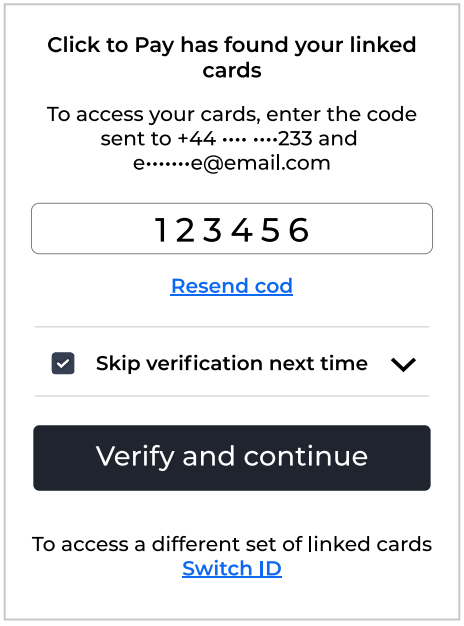
Recognized Click to Pay Customer UI



Unrecognized Click to Pay Customer

When the cardholder has a Click to Pay account but is not on a registered device, they receive a one-time password to their registered email address and phone number to authenticate their identity before their stored Click to Pay credentials are shown:

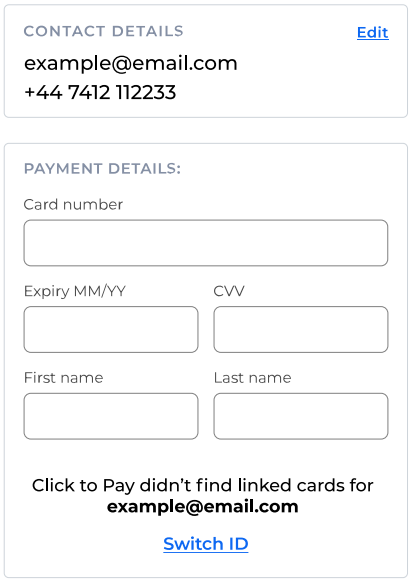
Unrecognized Click to Pay Customer on a Recognized Device UI



No Click to Pay Account

When the cardholder does not have a Click to Pay account, they can provide a new email address to perform a new lookup or they can choose to enter their card details manually. The cardholder can make a one-time payment or complete the payment and choose to create a Click to Pay account for future use:

No Click to Pay Account UI



Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

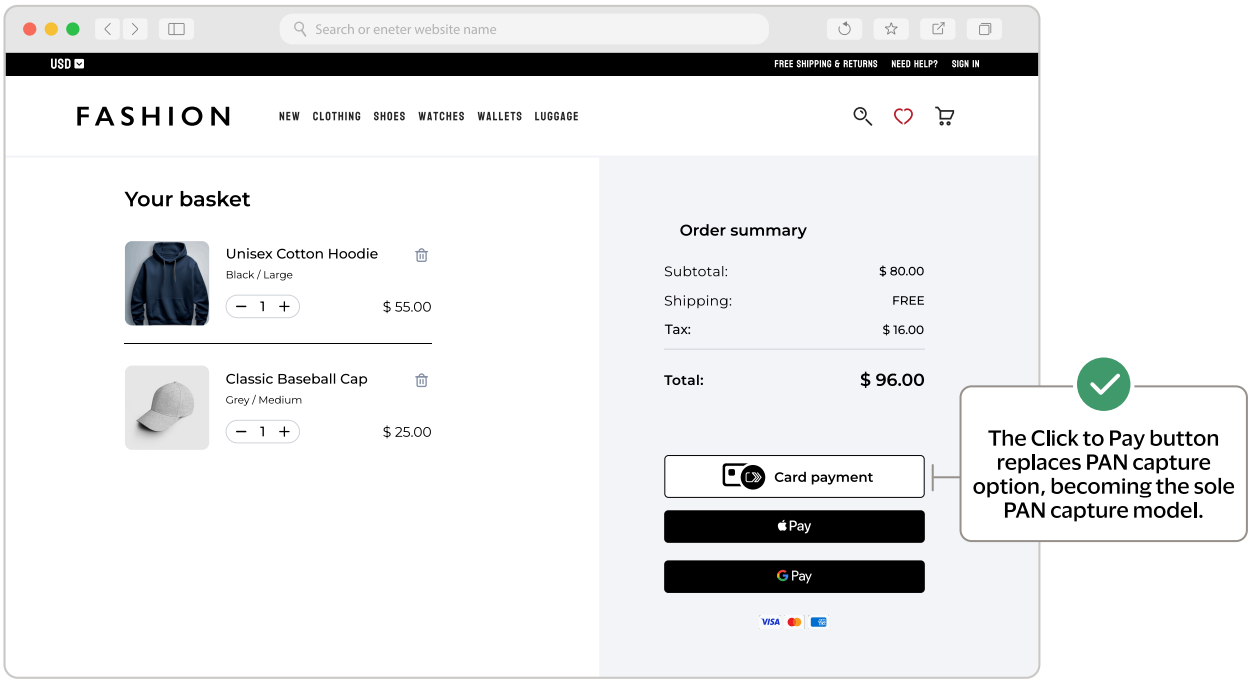
Click to Pay UI Examples

This section contains UI examples of how you should display Click to Pay on your payment page. For information about how to display the UI, see JavaScript API Reference (on page 244).

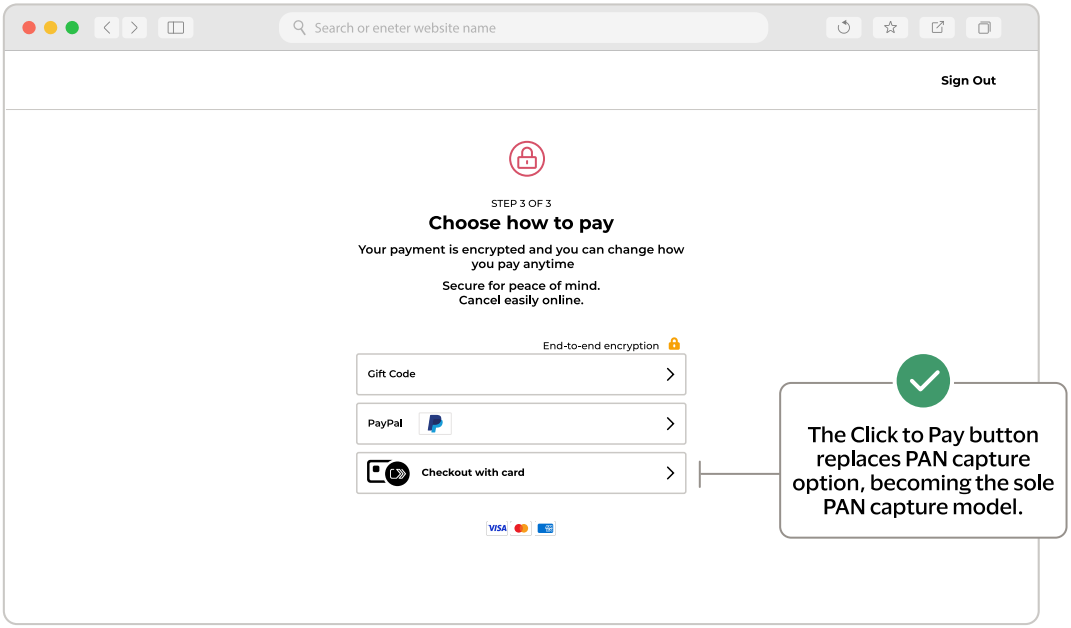
Click to Pay Replaces PAN Capture

Click to Pay is the card entry payment option within your payment page.

Click to Pay Replaces PAN Capture UI Example 1



Click to Pay Replaces PAN Capture UI Example 2

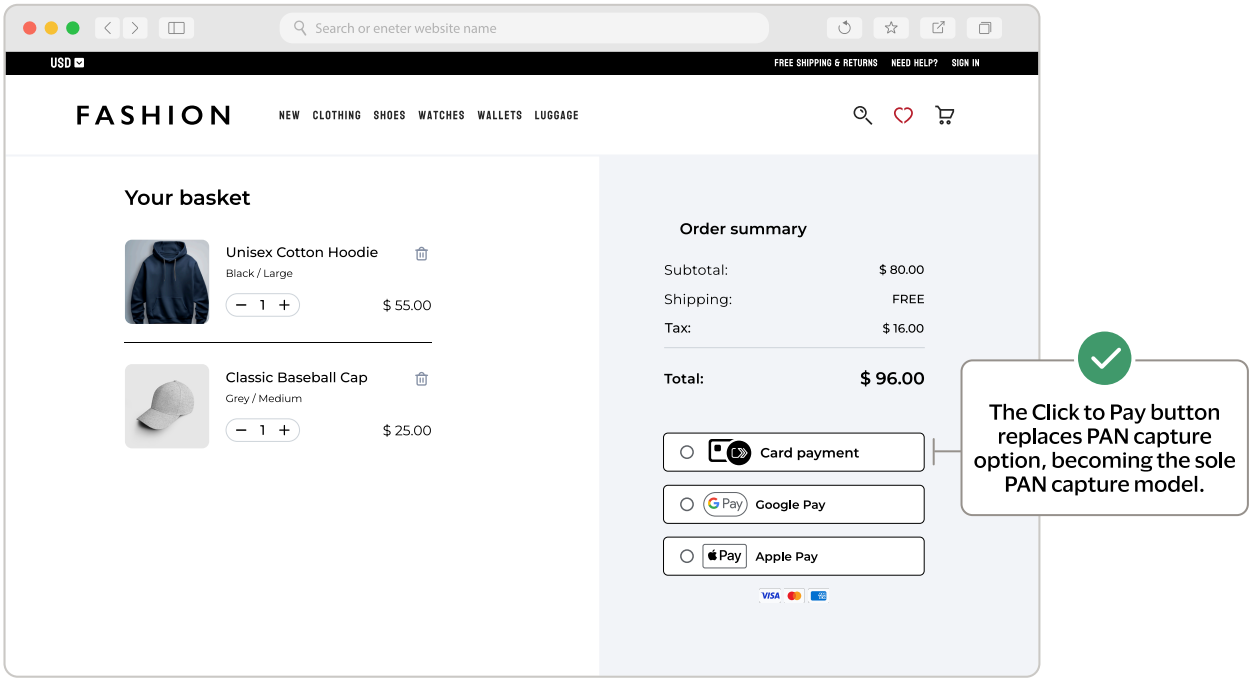


For information about how to configure this UI, see Loading the JavaScript Library (on page 137).

Click to Pay as Radio Button

Click to Pay is a radio button for the card entry payment option within your payment page. When the cardholder selects this option, the Click to Pay payment flow is loaded.

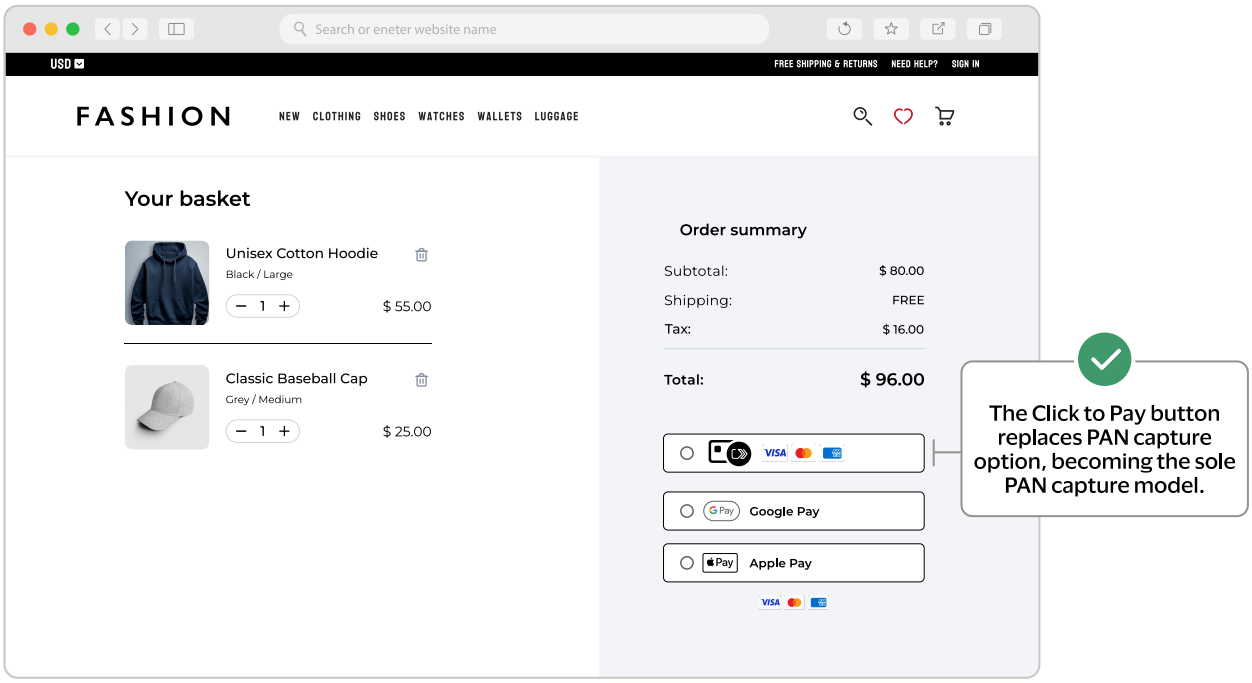
Click to Pay Radio Button Example UI



Click to Pay Icon on Radio Button

You can host the radio selection option for card payment with the Click to Pay icon displayed on the payment label. The Unified Checkout flow loads when the cardholder selects this option. For information about customizing how to trigger Unified Checkout, see Trigger (on page 121) and Components (on page 124).

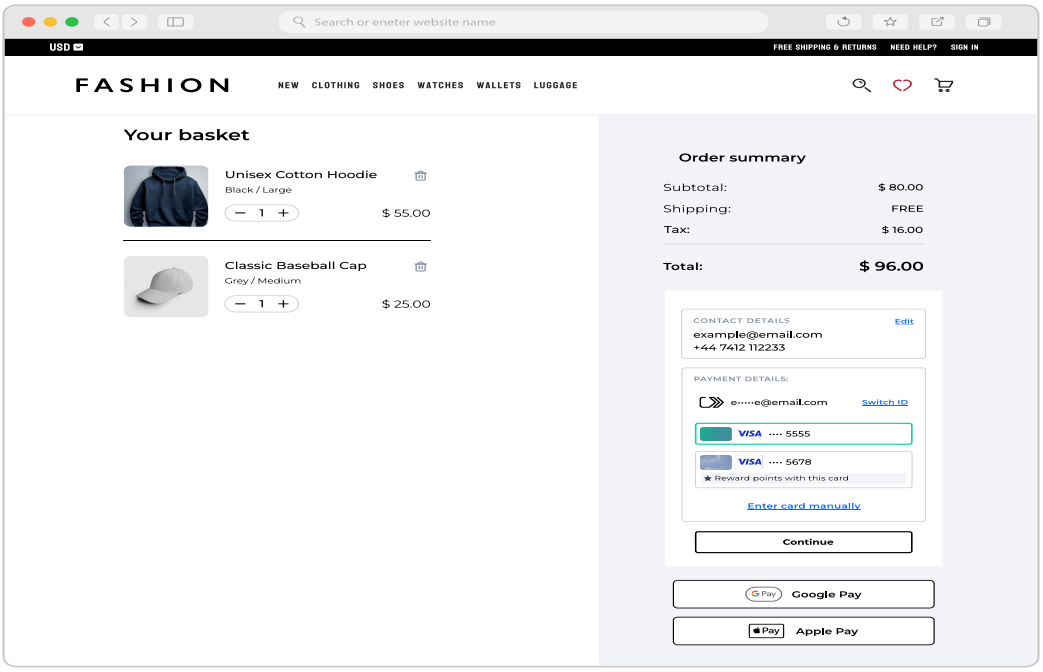
Click to Pay Icon on Radio Button Example UI



Load Click to Pay Automatically From Trigger or Components

You can load the Unified Checkout JavaScript flow within your own payment button without requiring the cardholder to select a card payment option. This example shows a recognized user payment flow where the cardholder's information is shown automatically next to the other payment methods hosted within your payment page. For information about customizing how to trigger Unified Checkout, see Trigger (on page 121) and Components (on page 124).

Click to Pay Loaded Automatically From Trigger or Components UI

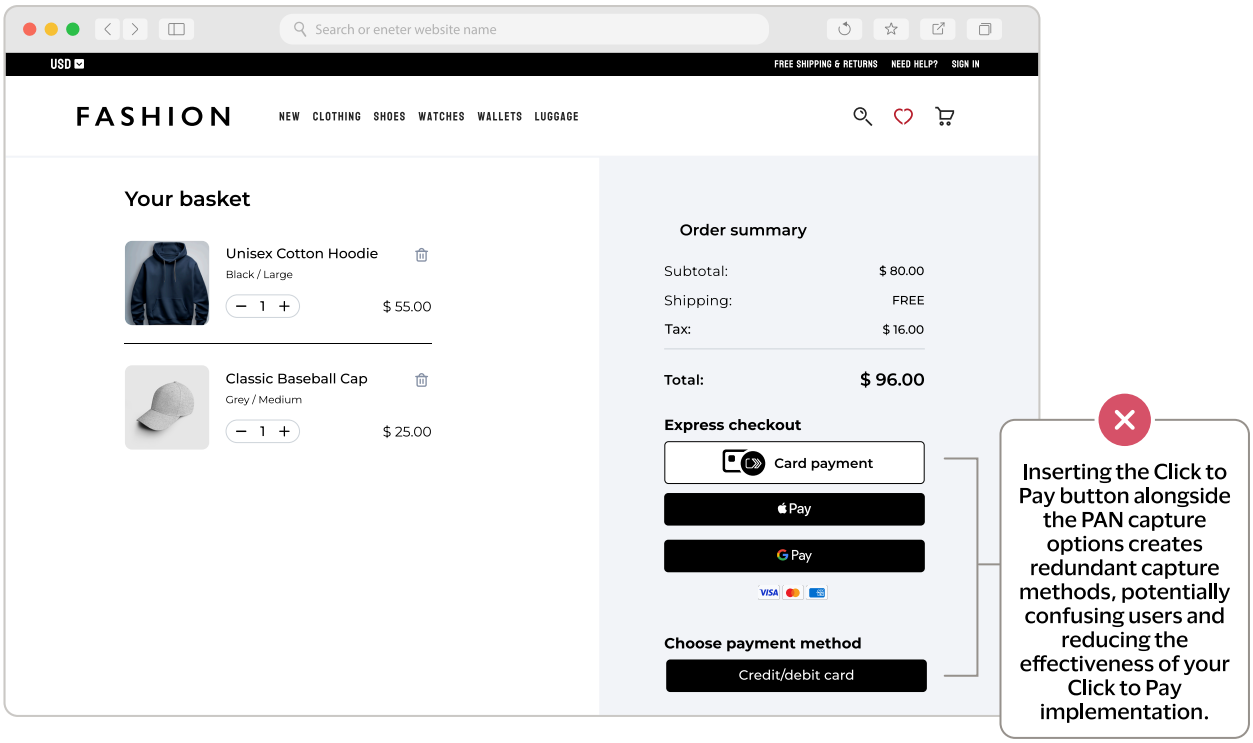


Card Payment Options with Click to Pay in UI

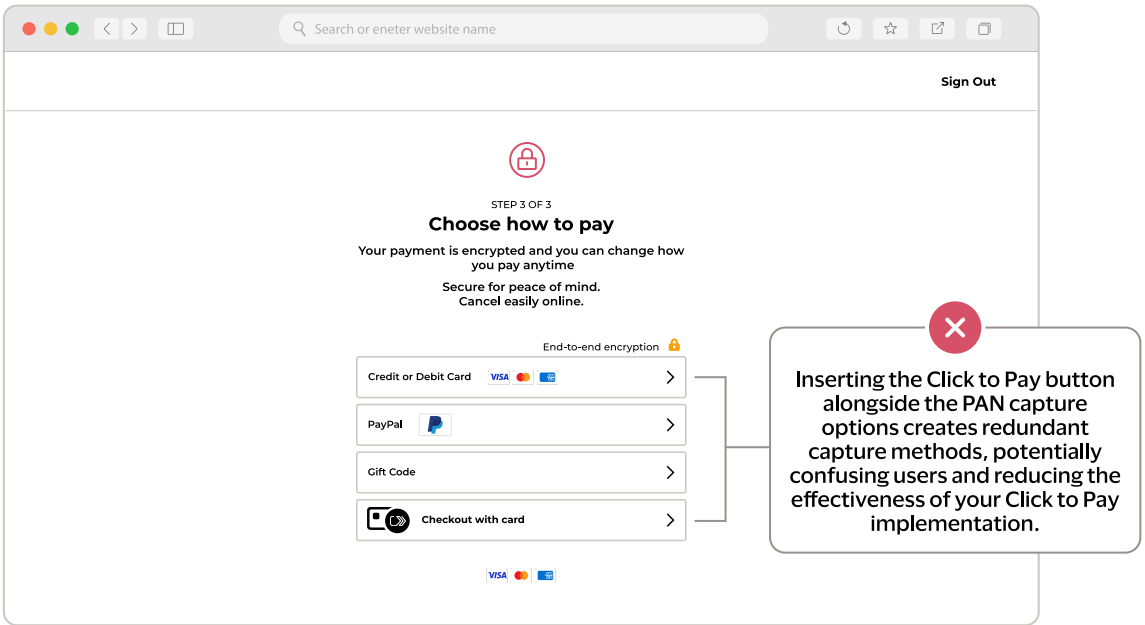
Do not present the Unified Checkout payment button as a separate payment method from the card payment button. If you do this, the cardholder is not prompted with their Click to Pay cards and must manually enter their payment details. They will also not have the option to store their card within Click to Pay for future use.

These examples show multiple card payment options and Click to Pay in a UI:

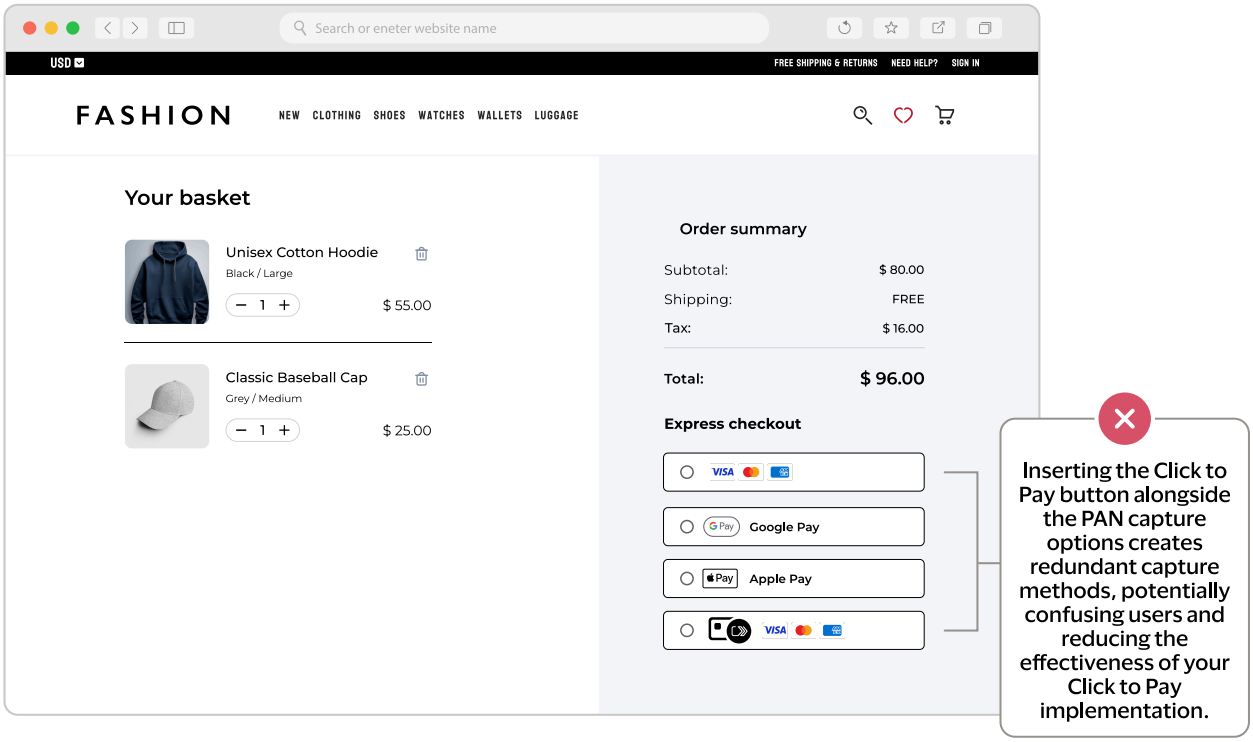
Multiple Card Payment Options in UI Example 1



Multiple Card Payment Options in UI Example 2



Multiple Card Payment Options in UI Example 3



Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Paze

Paze is an online checkout option, or *digital wallet*, that enables you to offer customers a fast and secure way to make purchases online. This section includes information about accepting Paze payments with your Unified Checkout integration.

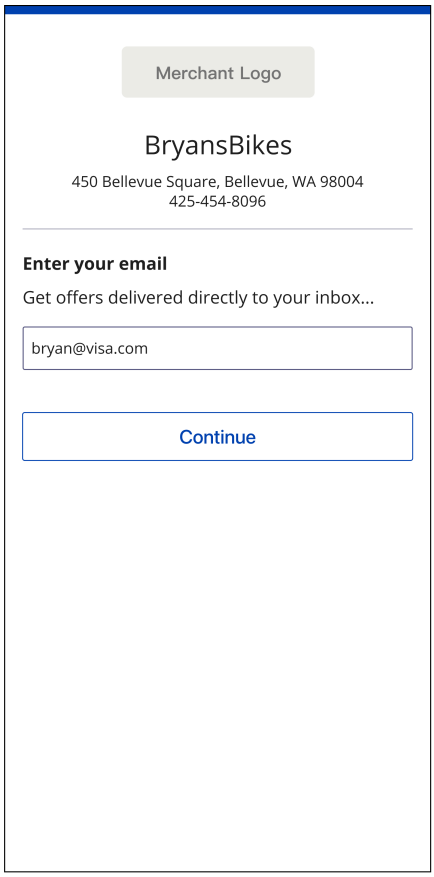
Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

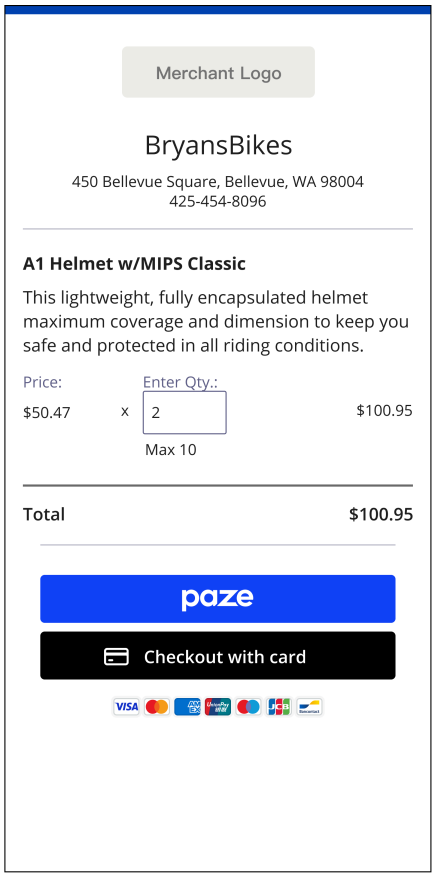
Paze UI

These screen captures show the sequence of events your customer can expect when completing a payment with Paze.

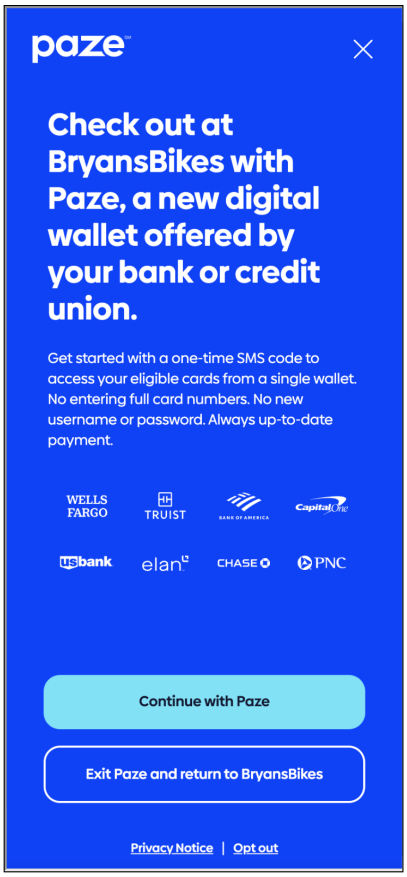
Paze UI



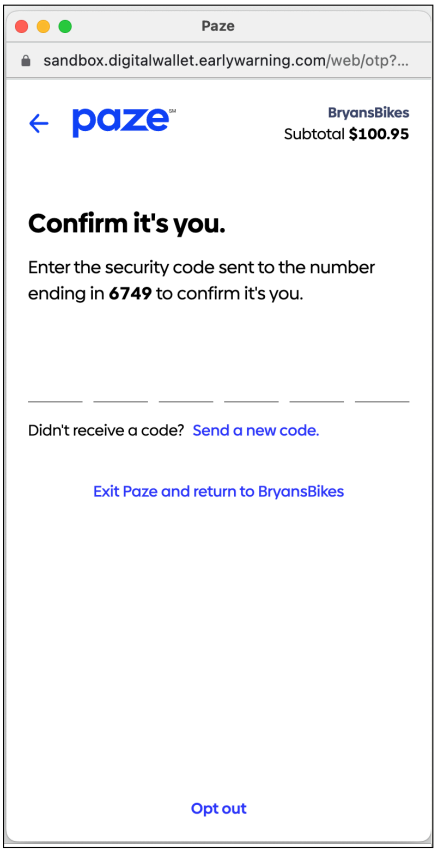
Collecting an email for Paze lookup



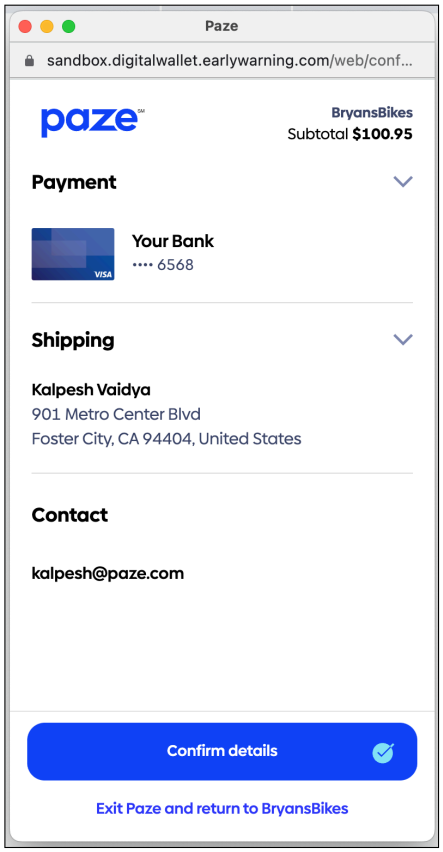
Paze in the payment method list



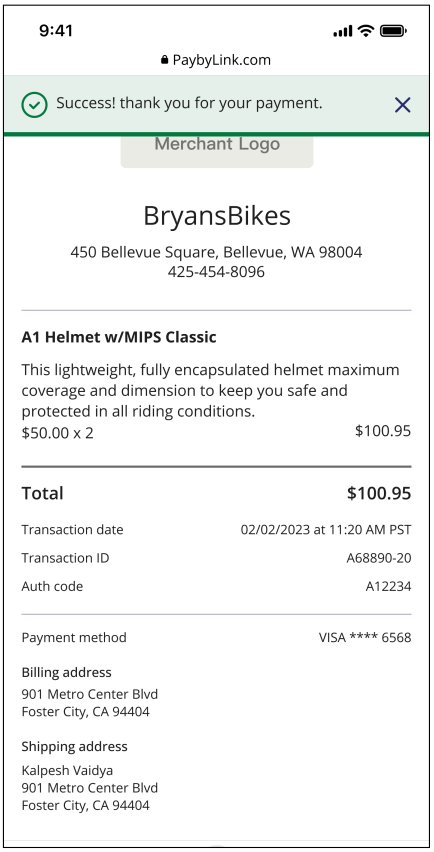
Paze explainer



Paze one-time password



Paze payment method selection



Paze payment confirmation

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Alternative Payment Methods

This section describes the alternative payment methods you can use in your Unified Checkout integration. After you successfully integrate one payment method, you can add another from the same category with minimal adjustments to your existing configuration.

- Online Bank Transfers (on page 72)
- Buy Now, Pay Later (on page 87)
- Post-Pay Reference Payments (on page 94)
- Alternative Payment Wallets (on page 98)

Related information

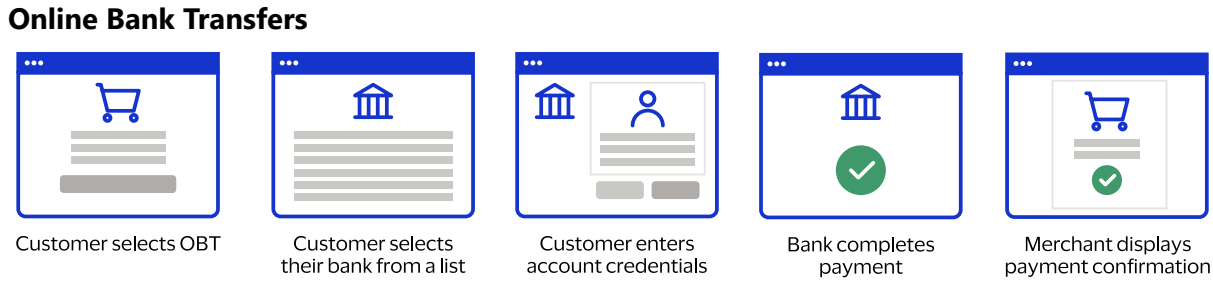
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Online Bank Transfers

Online bank transfers enable customers to complete their purchase by securely logging into their online banking environment. This method is secure, trusted, and widely used in many European countries.

Important: Before you can enroll in these alternative payment method on Unified Checkout, you must first be enabled for the alternative payment platform. Contact your portfolio administrator for more information.

This is how online bank transfers work:



1. The customer chooses online bank transfer as their payment method during checkout.
2. The customer chooses their bank from the list of available banks and is redirected to their bank's website or application where they are prompted to enter their account credentials.
3. The customer confirms their payment and completes the authorization process.
4. The customer is notified that the payment is complete.
5. The customer returns to your website for payment confirmation.

Unified Checkout supports these online bank transfer payment methods:

- Bancontact
- DragonPay
- iDeal
- Multibanco
- MyBank
- Przelewy24|P24
- Tink Pay By Bank

Online Bank Transfer Payment Methods

Payment Method	Capture Context allowedPaymentTypes	Capture Context completeMandate.type	Separate Capture?	Payment Confirmation	Customer Country (Country Code)	Customer ISO Currency Code
iDEAL	`IDEAL`	`CAPTURE` or `PREFER_AUTH`	No	Immediate	Netherlands (NL)	EUR
Multibanco	`MULTIBANCO`	`CAPTURE` or `PREFER_AUTH`	No	Immediate	Portugal (PT)	EUR
Przelewy24	`PRZELEWY24`	`CAPTURE` or `PREFER_AUTH`	No	Immediate	Poland (PL)	PLN
Bancontact	`BANCONTACT`	`CAPTURE` or `PREFER_AUTH`	No	Immediate	Belgium (BE)	EUR
MyBank	`MYBANK`	`CAPTURE` or `PREFER_AUTH`	No	Immediate	Italy (IT)	EUR
					Belgium (BE)	
					Portugal (PT)	
					Spain (ES)	
DragonPay	`DRAGONPAY`	`CAPTURE` or `PREFER_AUTH`	No	Immediate	Philippines (PH)	PHP
Tink Pay by Bank	`TINKPAYBYBANK`	`CAPTURE` or `PREFER_AUTH`	No	Immediate	France (FR)	EUR
					Germany (DE)	EUR
					Ireland (IE)	EUR
					Netherlands (NL)	EUR
					Spain (ES)	EUR
					United Kingdom (GB)	GBP

Related information

Getting Started with REST

Response Codes

API Field Reference Guide

API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

Bancontact

Bancontact enables customers to make secure online and in-store purchases directly from their bank accounts. Bancontact is a leading payment method in Belgium.

When the total amount of the order is outside the range of accepted transaction amounts, the Bancontact payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount**: EUR 0.01
- **Maximum transaction amount**: Not applicable

Opt in to Bancontact on Unified Checkout

Follow these steps to opt in to Bancontact on Unified Checkout:

1. Add Bancontact to your integration by adding `BANCONTACT` to the **allowedPaymentTypes** field object within the capture context request.
2. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

   You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

   If you support more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. The funds will be captured immediately for the online bank transfer.
3. Include these required fields for online bank transfers in the capture context request:
   - **data.orderInformation.billTo.country**
   - **data.orderInformation.billTo.firstName**
   - **data.orderInformation.billTo.lastName**
4. Include this optional field for online bank transfers in the capture context request:
   - **data.orderInformation.billTo.address1**
   - **data.orderInformation.billTo.email**

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

DragonPay

DragonPay provides Filipino customers and businesses with a secure payment channel that does not require customers to be banked or have a credit card. Customers can make purchases online and pay by bank transfer.

When the total amount of the order is outside the range of accepted transaction amounts, the DragonPay payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount**: PHP 50.01
- **Maximum transaction amount**: Not applicable

Opt in to DragonPay on Unified Checkout

Follow these steps to opt in to Multibanco on Unified Checkout:

1. Add DragonPay to your integration by adding `DRAGONPAY` to the **allowedPaymentTypes** field object within the capture context request.
2. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

   You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

   If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. The funds are captured immediately for the online bank transfer.
3. Include these required fields for online bank transfers in the capture context request:
   - **data.orderInformation.billTo.country**
   - **data.orderInformation.billTo.firstName**
   - **data.orderInformation.billTo.lastName**

4. Include this optional field for online bank transfers in the capture context request:
    - **data.orderInformation.billTo.address1**
    - **data.orderInformation.billTo.email**

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Tink Pay By Bank

Tink is an alternative payment method that uses the *pay by bank* payment method. Tink Pay By Bank enables customers to make payments directly from their bank account to the seller's account and bypasses traditional payment methods such as credit cards.

When the total amount of the order is outside the range of accepted transaction amounts, Tink Pay By Bank is not displayed in Unified Checkout.

These are the accepted transaction amounts for the United Kingdom (GB):

- **Minimum transaction amount**: Not applicable
- **Maximum transaction amount**: GBP 8,500

Opt in to Tink Pay By Bank on Unified Checkout

You can enable Tink Pay By Bank from the Unified Checkout merchant experience section in the Business Center. For information about how to enable Tink Pay By Bank using the Business Center, see Configure Payment Options (on page 152).

You can also enable Tink Pay By Bank using the Unified Checkout `v1/sessions` API. Follow these steps to opt in to the Tink Pay By Bank payment method using the API:

1. Add Tink Pay by Bank to your integration by adding `TINKPAYBYBANK` to the **allowedPaymentTypes** field object within the capture context request.

2. To use Tink Pay By Bank in your selected country, you must set the **country** field value to the correct value in the capture context request:

   Tink Pay By Bank country Field Values

Country	country Field Value
United Kingdom	`GB`

3. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

   You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

   If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. The funds are captured immediately for the online bank transfer.

4. To use Tink Pay By Bank in your selected country, you must set the **data.orderInformation.billTo.country** field value to the correct value in the capture context request:

   Tink Pay By Bank data.orderInformation.billTo.country Field Values

Country	data.orderInformation.billTo.country Field Value
United Kingdom	`GB`

5. To use Tink Pay By Bank in your selected country, you must set the **data.orderInformation.amountDetails.currency** field value to the correct value in the capture context request:

   Tink Pay By Bank data.orderInformation.amountDetails.currency Field Values

Country	data.orderInformation.amountDetails.currency Field Value
United Kingdom	`GBP`

6. Include this optional field for online bank transfers in the capture context request:

   - **data.orderInformation.billTo.firstName**
   - **data.orderInformation.billTo.lastName**
   - **data.orderInformation.shipTo.address1**
   - **data.orderInformation.shipTo.address2**
   - **data.orderInformation.shipTo.country**

- **data.orderInformation.shipTo.district**
- **data.orderInformation.shipTo.firstName**
- **data.orderInformation.shipTo.lastName**
- **data.orderInformation.shipTo.locailty**
- **data.orderInformation.shipTo.postalCode**

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

iDeal

iDEAL enables customers to pay online through their mobile banking app or online bank account and provides you with a payment guarantee. iDEAL supports these banks:

- ABN AMRO
- ASN Bank
- bunq
- ING
- Knab
- Rabobank
- RegioBank
- Revolut
- SNS
- Svenska Handelsbanken
- Triodos Bank
- Van Lanschot

When the total amount of the order is outside the range of accepted transaction amounts, the iDeal payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount**: EUR 0.01
- **Maximum transaction amount**: Subject to transaction approval from the customer's account.

Opt in to iDeal on Unified Checkout

Follow these steps to opt in to iDeal on Unified Checkout:

1. Add iDeal to your integration by adding `IDEAL` to the **allowedPaymentTypes** field object within the capture context request.
2. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

   You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

   If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. The funds are captured immediately for the online bank transfer.
3. Include these required fields for online bank transfers in the capture context request:
   - **data.orderInformation.billTo.country**
   - **data.orderInformation.billTo.firstName**
   - **data.orderInformation.billTo.lastName**
4. Include this optional field for online bank transfers in the capture context request:
   - **data.orderInformation.billTo.address1**
   - **data.orderInformation.billTo.email**

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

MyBank

MyBank enables customers to pay for their online purchases in an easy and safe way using real-time bank transfers. MyBank customers complete payments by selecting their bank and logging in with their online banking credentials.

When the total amount of the order is outside the range of accepted transaction amounts, the MyBank payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount**: EUR 0.01
- **Maximum transaction amount**: EUR 999,999,999.99

Opt in to MyBank on Unified Checkout

Follow these steps to opt in to MyBank on Unified Checkout:

1. Add MyBank to your integration by adding `MYBANK` to the **allowedPaymentTypes** field object within the capture context request.
2. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

   You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

   If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. The funds are captured immediately for the online bank transfer.
3. Include these required fields for online bank transfers in the capture context request:
   - **data.orderInformation.billTo.country**
   - **data.orderInformation.billTo.firstName**
   - **data.orderInformation.billTo.lastName**
4. Include this optional field for online bank transfers in the capture context request:
   - **data.orderInformation.billTo.address1**
   - **data.orderInformation.billTo.email**

Related information

Getting Started with REST
Response Codes
API Field Reference Guide

API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Multibanco

Multibanco enables customers to pay for a range of goods and services by bank transfer. These services include e-commerce, licenses, and taxes post-purchase. Multibanco is supported by all banks in Portugal.

When the total amount of the order is outside the range of accepted transaction amounts, the Multibanco payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount**: No minimum
- **Maximum transaction amount**: EUR 99,999

Opt in to Multibanco on Unified Checkout

Follow these steps to opt in to Multibanco on Unified Checkout:

1. Add Multibanco to your integration by adding `MULTIBANCO` to the **allowedPaymentTypes** field object within the capture context request.
2. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

   You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

   If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. The funds are captured immediately for the online bank transfer.
3. Include these required fields for online bank transfers in the capture context request:
   - **data.orderInformation.billTo.country**
   - **data.orderInformation.billTo.firstName**
   - **data.orderInformation.billTo.lastName**

4. Include this optional field for online bank transfers in the capture context request:
    - **data.orderInformation.billTo.address1**
    - **data.orderInformation.billTo.email**

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Przelewy24|P24

Przelewy24, or P24, is a Poland-based real-time online bank transfer payment method. P24 is one of the most popular payment methods in Poland covering all major consumer banks.

When the total amount of the order is outside the range of accepted transaction amounts, the P24 payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount**: PLN 0.01, EUR 0.01
- **Maximum transaction amount**: PLN 55,000.00, EUR 12,500.00

Opt in to Przelewy24|P24 on Unified Checkout

Follow these steps to opt in to Przelewy24|P24 on Unified Checkout:

1. Add Przelewy24|P24 to your integration by adding `PRZELEWY24` to the **allowedPaymentTypes** field object within the capture context request.
2. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

   You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

   If you support more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. The funds will be captured immediately for the online bank transfer.
3. Include these required fields for online bank transfers in the capture context request:

- **data.orderInformation.billTo.country**
- **data.orderInformation.billTo.email**
- **data.orderInformation.billTo.firstName**
- **data.orderInformation.billTo.lastName**

4. Include this optional field for online bank transfers in the capture context request:
   - **data.orderInformation.billTo.address1**

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Verify Status for Online Bank Transfers

When the status of your payment request is `PENDING`, you can verify the status by using the URL method and the payload that is included in the **transactionStatus.url** field in the webhook response:

Webhook Response

```
{
  "payload": {
    "transactionResult": {
      "id": "755775333723635790 4806",
      "rootId": "755775333723635790 4806",
      "reconciliationId": "XFZ40EJPGL5K",
      "submitTimeUTC": "2025-08-21T11:22:13Z",
      "merchantId": "uc_apm_tester004"
    },
    "transactionStatus": {
      "url":
 "https://apitest.cybersource.com/tss/v2/transactions/755775333723635790 4806",
      "method": "GET"
    }
  }
}
```

You can also send a request to this endpoint to verify the status:

Production: `GET https://api.cybersource.com/tss/v2/transactions/{id}`

Test: `GET https://apitest.cybersource.com/tss/v2/transactions/{id}`

The *{id}* is the ID that is returned in the webhook response. For more information, see Webhooks Support (on page 176).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Verify Status for Online Bank Transfers (Tink Pay By Bank)

When the status of your payment request is `PENDING`, you can verify the status by using the URL method and the payload that is included in the **transactionStatus.url** field in the webhook response:

Webhook Response


```
"payload": {
    "transactionResult": {
      "submitTimeUtc": "2025-07-22T08:16:24Z",
      "reconciliationId": "KPUJHD4X2G31",
      "processorInformation": {
        "responseCode": "00004"
      },
      "id": "7531173918516064204807",
      "message": "Request was processed successfully.",
      "status": "SETTLED"
    },
    "transactionStatus": {
      "url":
 "https://api.cybersource.com/pts/v2/refresh-payment-status/753117391851606420480
7",
      "method": "POST",
      "payload": {
        "clientReferenceInformation": {
          "applicationName": "unifiedCheckout"
        },
```

```
      "processingInformation": "processingInformation",
      "paymentInformation": {
        "paymentType": {
          "method": {
            "name": "tinkPayByBank"
          },
          "name": "bankTransfer"
        }
      }
    }
  }
  }
```

You can also send a request to this endpoint to verify the status:

Production: `POST https://api.cybersource.com/pts/v2/refresh-payment-status/{id}`

Test: `POST https://apitest.cybersource.com/pts/v2/refresh-payment-status/{id}`

The `{id}` is the ID that is returned in the webhook response. For more information, see Webhooks Support (on page 176).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Handle Responses

When Unified Checkout automatically processes a payment with `autoProcessing` is set to `true` or you have set `autoProcessing` to `false` and are using **checkout.Complete()**, you must handle both successful responses and various errors. After the payment is complete, the **completeResponse** field object contains information about the transaction outcome.

When a payment is processed successfully, you must parse the response to confirm the payment status, update their order records, and trigger any post-payment workflows. Post-payment workflows include sending confirmation emails or updating inventory. See JavaScript Example: Processing a Payment (on page 139).

Your error handling should account for specific cases such as `COMPLETE_TRANSACTION_CANCELED` and `COMPLETE_TRANSACTION_FAILED`. `COMPLETE_TRANSACTION_CANCELED` occurs when the user cancels the transaction and `COMPLETE_TRANSACTION_FAILED` indicates that the consumer's transaction failed.

For PPRO-enabled online bank transfers, only cancellation errors are returned, and Tink Pay By Bank returns failure and cancellation errors. For information about possible errors that can occur when calling the complete API see UnifiedCheckoutError (on page 232) in Handle Errors (on page 231).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Buy Now, Pay Later

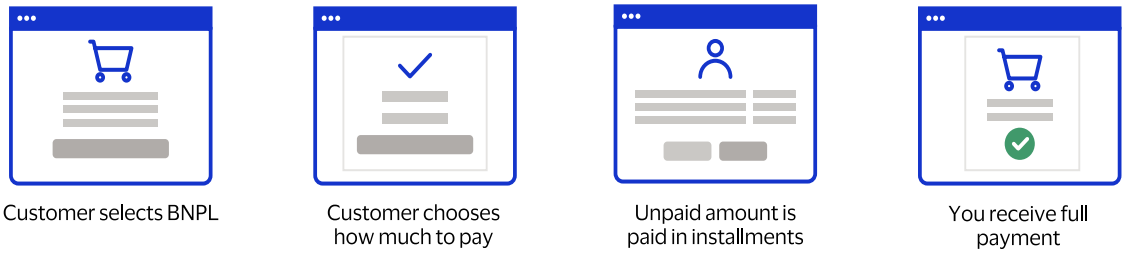
Buy Now, Pay Later payment methods enable customers to purchase goods or services immediately and pay in installments over time. With Buy Now, Pay Later, you are paid immediately and in full, while your customers pay nothing or only a portion of the total at the time of purchase. The remaining balance is typically spread over equal, often interest-free, payments.

Buy Now, Pay Later is increasingly popular for both online and in-store purchases.

Important: Before you can enroll in these alternative payment method on Unified Checkout, you must first be enabled for the alternative payment platform. Contact your portfolio administrator for more information.

This is how Buy Now, Pay Later works:

Buy Now, Pay Later



1. The customer chooses their Buy Now, Pay Later payment method during checkout.
2. The customer chooses how much they want to pay, such as nothing, installments, or the total amount.
3. The unpaid amount is divided into equal installments that are paid over a fixed amount of time.
4. You receive the full payment after the customer completes checkout, and the Buy Now, Pay Later provider collects the installment payments from your customer.

Unified Checkout supports the Afterpay/Clearpay and Paypal Buy Now, Pay Later payment methods.

Buy Now, Pay Later Payment Method Support

Payment Method	Capture Context allowedPaymentTypes	Capture Context completeMandate.type	Separate Capture?	Payment Confirmation	Customer Country (Country Code)	Customer ISO Currency Code
Afterpay	`AFTERPAY`	`CAPTURE`	No	Immediate	Canada (CA)	CAD
		`AUTH` or `PREFER_AUTH`	Yes	Delayed		

Buy Now, Pay Later Payment Method Support (continued)

Payment Method	Capture Context allowedPaymentTypes	Capture Context completeMandate.type	Separate Capture?	Payment Confirmation	Customer Country (Country Code)	Customer ISO Currency Code
		`CAPTURE`	No	Immediate	Australia (AU)	AUD
		`AUTH` or `PREFER_AUTH`	Yes	Delayed		
		`CAPTURE`	No	Immediate	New Zealand (NZ)	NZD
		`AUTH` or `PREFER_AUTH`	Yes	Delayed		
Cash App Afterpay		`CAPTURE`	No	Immediate	United States (US)	USD
		`AUTH` or `PREFER_AUTH`	Yes	Delayed		
Clearpay		`CAPTURE`	No	Immediate	Great Britain (GB)	GBP
		`AUTH` or `PREFER_AUTH`	Yes	Delayed		

For information on ISO country codes, see ISO Standard Country Codes.

For information on ISO currency codes, see ISO Standard Currency Codes.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Afterpay

Afterpay is a Buy Now, Pay Later service that allows customers to purchase items immediately and pay for them in four interest-free installments over a period of 6 weeks. Afterpay is also known as Clearpay in the UK, and Cash App Afterpay in the US. For more information, see the *Afterpay and Clearpay Developer Guide*.

When the total amount of the order is outside the range of accepted transaction amounts, the Afterpay/Clearpay payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount**: 1 (CAD, AUD, NZD, USD, and GBP)
- **Maximum transaction amount**: Not applicable

Opt in to Afterpay on Unified Checkout

Follow these steps to opt in to the Afterpay/Clearpay payment method in Unified Checkout:

1. Add Afterpay to your integration by adding `AFTERPAY` to the **allowedPaymentTypes** field within the capture context request. The default field value is `AFTERPAY` even if you want to support Cash App Afterpay in the US or Clear Pay in the UK.
2. Set the **completeMandate.type** field value to `AUTH`, `CAPTURE` or `PREFER_AUTH`.

   You can perform a sale and capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

   You can capture the funds later if you include the **completeMandate.type** field in the capture context request and set the value to `AUTH`. When you capture the funds later, you must perform a capture using the payments API. See Captures (on page 92).

   If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. You must perform a capture using the payments API when an authorization is performed. A capture is performed automatically if an authorization is not allowed by the payment type.
3. Include these required fields in the capture context request:
   - **data.orderInformation.billTo.email**
   - **data.orderInformation.billTo.firstName**
   - **data.orderInformation.billTo.lastName**
   - **data.orderInformation.billTo.address1**
   - **data.orderInformation.billTo.locality**
   - **data.orderInformation.billTo.postalCode**
   - **data.orderInformation.billTo.administrativeArea**
   - **data.orderInformation.billTo.country**
4. Include these optional fields in the capture context request:

   Important: These fields are required when the **requestShipping** field is set to `true`.

- **data.orderInformation.shipTo.firstName**
- **data.orderInformation.shipTo.lastName**
- **data.orderInformation.shipTo.address1**
- **data.orderInformation.shipTo.locality**
- **data.orderInformation.shipTo.postalCode**
- **data.orderInformation.shipTo.administrativeArea**
- **data.orderInformation.shipTo.country**

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Verify Status for Afterpay

When the status of your payment request is `PENDING`, you can verify the status by sending a POST request to the URL that is included in the **transactionStatus.url** field in the webhook response.

Webhook Response


```
{
  "payload": {
    "transactionResult": {
      "submitTimeUtc": "2025-07-22T08:16:24Z",
      "reconciliationId": "KPUJHD4X2G31",
      "processorInformation": {
        "responseCode": "00004"
      },
      "id": "753117391851606420480 7",
      "message": "Request was processed successfully.",
      "status": "SETTLED"
    },
    "transactionStatus": {
```

```
      "url":
 "https://apitest.cybersource.com/pts/v2/refresh-payment-status/7531173918516064204807",
      "method": "POST",
      "payload": {
        "clientReferenceInformation": {
          "applicationName": "unifiedCheckout"
        },
        "processingInformation": {
          "actionList": [
            "AP_STATUS"
          ]
        },
        "paymentInformation": {
          "paymentType": {
            "method": {
              "name": "AFTERPAY"
            },
            "name": "INVOICE"
          }
        }
      }
    }
  }
}
```

You can also send a request to this endpoint to verify the status:

Production: `POST https://api.cybersource.com/pts/v2/refresh-payment-status/{id}`

Test: `POST https://apitest.cybersource.com/pts/v2/refresh-payment-status/{id}`

The *{id}* is ID that is returned in the webhook response. For more information, see Webhooks Support (on page 176).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Handle Responses

When Unified Checkout automatically processes a payment with `autoProcessing` is set to `true` or you have set `autoProcessing` to `false` and are using **checkout.Complete()**, you must handle both successful responses and various errors. After the payment is complete, the **completeResponse** field object contains information about the transaction outcome.

When a payment is processed successfully, you must parse the response to confirm the payment status, update their order records, and trigger any post-payment workflows. Post-payment workflows include sending confirmation emails or updating inventory. See JavaScript Example: Processing a Payment (on page 139).

Your error handling should account for specific cases such as `COMPLETE_TRANSACTION_CANCELED` and `COMPLETE_TRANSACTION_FAILED`. `COMPLETE_TRANSACTION_CANCELED` occurs when the user cancels the transaction and `COMPLETE_TRANSACTION_FAILED` indicates that the consumer's transaction failed.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Captures

When you set the **completeMandate.type** field value to `AUTH` or `PREFER_AUTH`, you must send a request to capture an authorized payment. Full and partial captures are supported.

Endpoint

Production: `POST https://api.cybersource.com/pts/v2/payments/{id}/captures`

Test: `POST https://apitest.cybersource.com/pts/v2/payments/{id}/captures`

The *{id}* is the transaction ID returned in the authorization response.

Example: Authorization Response from Unified Checkout

```
{
  "details": {
    "clientReferenceInformation": {
```

```
      "code": "1753351101383"
    },
    "orderInformation": {
      "amountDetails": {
        "currency": "USD",
        "totalAmount": "21.00"
      }
    },
    "processorInformation": {
      "approvalCode": "AUTH456789",
      "responseCode": "00003",
      "responseDetails": "00003",
      "transactionId": "201601180815391001180815 3AUTH"
    },
    "reconciliationId": "04RYADD29YRO",
    "submitTimeUtc": "2025-07-24T09:58:21Z"
  },
  "id": "753351101428697 1803092",
  "message": "Request processed successfully.",
  "outcome": "AUTHORIZED",
  "status": "AUTHORIZED"
}
```


Response Status

Cybersource responds to your capture request with one of these statuses:

- `FAILED`: The capture request failed.
- `PENDING`: The capture request is accepted but not captured. Send a request to the check status service to retrieve status updates.
- `SETTLED`: The capture request is settled for the amount requested.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

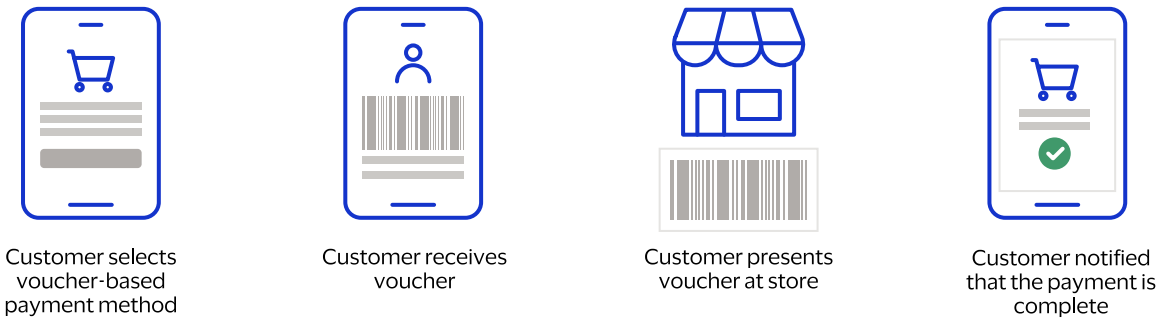
Post-Pay Reference Payments

Post-pay reference payments provide an alternative way for your customers to complete online purchases using cash. During checkout, customers select a voucher payment option and receive a unique code or payment slip. To finalize their purchase, they visit a designated offline location, such as a convenience store or payment kiosk, and pay the required amount in person. Once the payment is made, the merchant receives a notification that they can process the order.

Post-pay reference payments are widely used in countries where cash transactions are common or where many customers don't have access to credit or debit cards.

Important: Before you can enroll in these alternative payment method on Unified Checkout, you must first be enabled for the alternative payment platform. Contact your portfolio administrator for more information.

This is how post-pay reference payments work:



1. The customer selects a convenience store/voucher payment method during checkout.
2. The customer receives their payment code/voucher or QR code.
3. The customer presents the payment code/voucher or QR in-store to complete the payment.
4. The customer receives a notification that the payment is complete.

Unified Checkout supports the Konbini voucher-based payment option:

Post-Pay Reference Payment Support

Payment Method	Capture Context allowedPaymentTypes	Capture Context completeMandate.type	Separate Capture?	Payment Confirmation	Customer Country (Country Code)	Customer ISO Currency Code
Konbini	`KONBINI`	`CAPTURE` or `PREFER_AUTH`	No	Immediate	Japan (JP)	JPY

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Konbini

Konbini is used to make cash payments in Japan. Konbini payments enable your customers to pay for bills and online purchases at convenience stores in Japan.To complete a transaction, your customers receive payment codes for specific convenience stores and a confirmation number. Customers must then bring the information to a convenience store to make a cash payment. Your customers can pay at these convenience stores in Japan:

- 7-Eleven
- Family Mart
- Lawson
- Ministop
- Seicomart

You receive the payment confirmation immediately and the funds are available after 4 business days.

When the total amount of the order is outside the range of accepted transaction amounts, the Konbini payment button is not displayed in Unified Checkout. These are the accepted transaction amounts:

- **Minimum transaction amount**: JPY 1
- **Maximum transaction amount**: Not applicable

Opt in to Konbini on Unified Checkout

Follow these steps to opt in to the Konbini payment method in Unified Checkout:

1. Add Konbini to your integration by adding `KONBINI` to the **allowedPaymentTypes** field within the capture context request.
2. Set the **completeMandate.type** field value to `AUTH`, `CAPTURE` or `PREFER_AUTH`.

   You can perform a sale and capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

   If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. You must perform a capture using the payments API when an authorization is performed. A capture is performed automatically if an authorization is not allowed by the payment type.
3. Include these required fields in the capture context request:
   - **data.orderInformation.billTo.country**
   - **data.orderInformation.billTo.firstName**
   - **data.orderInformation.billTo.lastName**
   - **data.orderInformation.billTo.phoneNumber**
4. Include these optional fields in the capture context request:

   Important: These fields are required when the **requestShipping** field is set to `true`.

   - **data.orderInformation.billTo.address1**
   - **data.orderInformation.billTo.email**

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Verify Status for Post-Pay Reference

When the status of your payment request is `PENDING`, you can verify the status by sending a POST request to the URL that is included in the **transactionStatus.url** field in the webhook response:

Webhook Response


```
{
  "payload": {
    "transactionResult": {
      "id": "755775333723635790 4806",
      "rootId": "755775333723635790 4806",
      "reconciliationId": "XFZ40EJPGL5K",
      "submitTimeUTC": "2025-08-21T11:22:13Z",
      "merchantId": "uc_apm_tester004"
    },
    "transactionStatus": {
      "url":
 "https://apitest.cybersource.com/tss/v2/transactions/755775333723635790 4806",
      "method": "GET"
    }
  }
}
```


You can also send a request to this endpoint to verify the status:

Production: `GET https://api.cybersource.com/tss/v2/transactions/{id}`

Test: `GET https://apitest.cybersource.com/tss/v2/transactions/{id}`

The *{id}* is the ID that is returned in the webhook response. For more information, see Webhooks Support (on page 176).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Handle Responses

When Unified Checkout automatically processes a payment with `autoProcessing` is set to `true` or you have set `autoProcessing` to `false` and are using **checkout.Complete()**, you must handle both successful responses and various errors. After the payment is complete, the **completeResponse** field object contains information about the transaction outcome.

When a payment is processed successfully, you must parse the response to confirm the payment status, update their order records, and trigger any post-payment workflows. Post-payment workflows include sending confirmation emails or updating inventory. See JavaScript Example: Processing a Payment (on page 139).

Your error handling should account for specific cases such as `COMPLETE_TRANSACTION_CANCELED` which is returned when the user cancels the transaction.

For PPRO-enabled online bank transfers, only cancellation errors are returned. For information about possible errors that can occur when calling the complete API see UnifiedCheckoutError (on page 232) in Handle Errors (on page 231).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Alternative Payment Wallets

Digital wallets are secure applications or services that enable users to store payment details, such as debit or credit cards electronically.

For online transactions, the wallet securely passes a payment token to the merchant or payment processor. This means that you do not handle sensitive customer data directly. This reduces friction at checkout, improves conversion rates, and enhances security through built-in authentication and tokenization.

Wallets are best suited for one-time online purchases and express checkout experiences. Support for subscriptions and recurring payments varies by wallet, so you must ensure compatibility if future charges or merchant-initiated transactions are required.

Unified Checkout supports these wallet-based payment options:

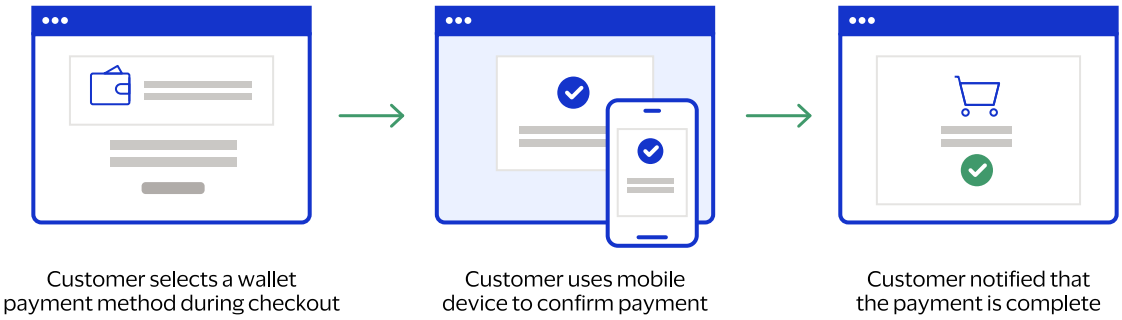
- PayPal
- Venmo

This is how payments with digital wallets work:

Customer-facing mobile flow



Customer-facing web flow



Digital Wallet Payment Support

Payment Method	Capture Context allowedPaymentTypes	Capture Context completeMandate.type	Separate Capture?	Payment Confirmation	Customer Country (Country Code)	Customer ISO Currency Code
PayPal	`PAYPAL`	`AUTH` `PREFER_AUTH`	Yes	Delayed	Global	AUD, CAD, CHF, CZK, DKK, EUR, GBP, HKD, NOK, NZD, PLN, SEK, SGD, and USD
PayPal		`CAPTURE`	No	Immediate		
Venmo	`VENMO`	`AUTH` `PREFER_AUTH`	Yes	Delayed	United States (US)	USD
Venmo		`CAPTURE`	No	Immediate		

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

PayPal

PayPal is a secure and convenient payment service that your customers can use to make payments without directly using their bank accounts or credit cards. PayPal gives yours customers the option to pay in installments over time with PayPal Pay Later while also giving you the full payment immediately. These installments are available:

- **Pay in 3**: Pay in three installments. Available in the UK.
- **Pay in 4**: Pay in four installments. Available in the US.
- **Pay Monthly**: Pay in monthly recurring installments.

Opt in to Paypal on Unified Checkout

You can enable Paypal from the Unified Checkout merchant experience section in the Business Center. For information about how to enable Paypal using the Business Center, see Configure Payment Options (on page 152).

You can also enable Paypal using the Unified Checkout `v1/sessions` API. Follow these steps to opt in to the Paypal payment method using the API:

1. Add Paypal to your integration by adding `PAYPAL` to the **allowedPaymentTypes** field within the capture context request.
2. Set the **completeMandate.type** field value to `AUTH`, `CAPTURE` or `PREFER_AUTH`.

   You can perform a sale and capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

   You can capture the funds later if you include the **completeMandate.type** field in the capture context request and set the value to `AUTH`. When you capture the funds later, you must perform a capture using the payments API. See Captures (on page 110).

   If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. You must perform a capture using the payments API when an authorization is performed. A capture is performed automatically if an authorization is not allowed by the payment type.
3. Include these required fields in the capture context request:
   - **orderInformation.amountDetails.currency**
   - **orderInformation.amountDetails.taxAmount**
     - This field is the sum of all **orderInformation.lineItems[].taxAmount** values.
   - **orderInformation.amountDetails.totalAmount**

        - This field is the sum of (**orderInformation.lineItems[].quantity** × **orderInformation.lineItems[].unitPrice**) + **orderInformation.lineItems[].taxAmount** for all line items.

    - **orderInformation.lineItems[].productDescription**
    - **orderInformation.lineItems[].productName**
    - **orderInformation.lineItems[].productSKU**
    - **orderInformation.lineItems[].quantity**
    - **orderInformation.lineItems[].taxAmount**

      Important: This field is required for all line items when you include **orderInformation.lineItems[].taxAmount** for one line item.

        - You can set this field value to `0.00`.

    - **orderInformation.lineItems[].typeOfSupply**
    - **orderInformation.lineItems[].unitPrice**

4. Include these optional fields in the capture context request:

    - **buyerInformation.dateOfBirth**
    - **buyerInformation.language**
    - **buyerInformation.personalIdentification[].id**
    - **buyerInformation.personalIdentification[].type**
    - **clientReferenceInformation.reconciliationId**
    - **merchantInformation.merchantDescriptor.name**

        - This field affects the name that the customers see on their card statement. If the merchant is Candy Shop and passes this as their name - customer will see `PAYPAL * Candy Shop` in their bank statement.

    - **orderInformation.amountDetails.taxDetails.taxId**
    - **orderInformation.amountDetails.taxDetails.type**
    - **data.orderInformation.billTo.address1**
    - **data.orderInformation.billTo.company.name**
    - **data.orderInformation.billTo.country**

- **data.orderInformation.billTo.email**
- **data.orderInformation.billTo.firstName**
- **data.orderInformation.billTo.lastName**
- **data.orderInformation.billTo.locality**
- **data.orderInformation.billTo.phoneNumber**
- **orderInformation.invoiceDetails.invoiceNumber**
- **orderInformation.invoiceDetails.productDescription**
- **orderInformation.lineItems[].totalAmount**
- **data.orderInformation.shipTo.country**
- **data.orderInformation.shipTo.locality**
- **data.orderInformation.shipTo.postalCode**
- **paymentInformation.customer.customerid**
- **processingInformation.processingInstruction**
  - Set this field to `ORDER_SAVED_EXPLICITLY` to save the customer's payment credentials using the save an order follow-on request.

Example: Multiple Line-Items

This example includes multiple line items in the **amountDetails** field object:

```
{
  "amountDetails": {
    "totalAmount": "221.91",
    "currency": "USD",
    "taxAmount": "14.42"
  },
  "lineItems": [
    {
      "productName": "501 Original Fit Jeans Blues",
      "productSku": "00501019403432",
      "quantity": 1,
      "unitPrice": "100.0",
      "totalAmount": "100.00",
      "taxAmount": "8.63"
    },
    {
```

```
      "productName": "Levi Blue shirt Plaid",
      "productSku": "0094784142",
      "quantity": 1,
      "unitPrice": "100.00",
      "totalAmount": "100.00",
      "taxAmount": "5.79"
    },
    {
      "productName": "shipping_and_handling",
      "productSku": "shipping_and_handling",
      "quantity": 1,
      "unitPrice": "7.49",
      "totalAmount": "7.49",
      "taxAmount": "0.00"
    }
  ]
}
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Venmo

Venmo is a mobile payment service that is owned by PayPal and enables your customers to make payments from their Venmo mobile application. Customers link their Venmo accounts to their bank accounts, debit cards, and credit cards to send and receive payments. Venmo is a secure and convenient payment service for your customers to make payments without directly using their bank accounts or credit cards.

Opt in to Venmo on Unified Checkout

You can enable Venmo from the Unified Checkout merchant experience section in the Business Center. For information about how to enable Venmo using the Business Center, see Configure Payment Options (on page 152).

You can also enable Venmo using the Unified Checkout `v1/sessions` API. Follow these steps to opt in to the Venmo payment method using the API:

1. Add Venmo to your integration by adding `VENMO` to the **allowedPaymentTypes** field object within the capture context request:

2. Set the **completeMandate.type** field value to `CAPTURE` or `PREFER_AUTH`.

   You can capture the funds immediately if you include the **completeMandate.type** field in the capture context request and set the value to `CAPTURE`.

   If you accept more than one payment type and must perform an authorization where funds are collected at a later time, set the **completeMandate.type** field to `PREFER_AUTH`. The funds are captured immediately for the online bank transfer.

3. Include these required fields for online bank transfers in the capture context request:

   - **orderInformation.amountDetails.currency**
   - **orderInformation.amountDetails.taxAmount**
     - This field is the sum of all **orderInformation.lineItems[].taxAmount** values.
   - **orderInformation.amountDetails.totalAmount**
     - This field is the sum of (**orderInformation.lineItems[].quantity** × **orderInformation.lineItems[].unitPrice**) + **orderInformation.lineItems[].taxAmount** for all line items.
   - **orderInformation.lineItems[].productDescription**
   - **orderInformation.lineItems[].productName**
   - **orderInformation.lineItems[].productSKU**
   - **orderInformation.lineItems[].quantity**
   - **orderInformation.lineItems[].taxAmount**

     Important: This field is required for all line items when you include **orderInformation.lineItems[].taxAmount** for one line item.

     - You can set this field value to `0.00`.
   - **orderInformation.lineItems[].unitPrice**

4. Include these optional fields in the capture context request:

   - **buyerInformation.dateOfBirth**
   - **buyerInformation.language**
   - **buyerInformation.personalIdentification[].id**

- **buyerInformation.personalIdentification[].type**
- **clientReferenceInformation.reconciliationId**
- **merchantInformation.merchantDescriptor.name**
    - This field affects the name that the customers see on their card statement. If the merchant is Candy Shop and passes this as their name - customer will see `PAYPAL * Candy Shop` in their bank statement.
- **orderInformation.amountDetails.taxDetails.taxId**
- **orderInformation.amountDetails.taxDetails.type**
- **data.orderInformation.billTo.address1**
- **data.orderInformation.billTo.company.name**
- **data.orderInformation.billTo.country**
- **data.orderInformation.billTo.email**
- **data.orderInformation.billTo.firstName**
- **data.orderInformation.billTo.lastName**
- **data.orderInformation.billTo.locality**
- **data.orderInformation.billTo.phoneNumber**
- **orderInformation.invoiceDetails.invoiceNumber**
- **orderInformation.invoiceDetails.productDescription**
- **orderInformation.lineItems[].totalAmount**
- **orderInformation.lineItems[].typeOfSupply.type**
- **data.orderInformation.shipTo.country**
- **data.orderInformation.shipTo.locality**
- **data.orderInformation.shipTo.postalCode**
- **paymentInformation.customer.customerid**
- **processingInformation.processingInstruction**
    - Set this field to `ORDER_SAVED_EXPLICITLY` to save the customer's payment credentials using the save an order follow-on request.

Example: Multiple Line-Items

This example includes multiple line items in the **amountDetails** field object:

```
{
  "amountDetails": {
    "totalAmount": "221.91",
    "currency": "USD",
    "taxAmount": "14.42"
  },
  "lineItems": [
    {
      "productName": "501 Original Fit Jeans Blues",
      "productSku": "00501019403432",
      "quantity": 1,
      "unitPrice": "100.0",
      "totalAmount": "100.00",
      "taxAmount": "8.63"
    },
    {
      "productName": "Levi Blue shirt Plaid",
      "productSku": "0094784142",
      "quantity": 1,
      "unitPrice": "100.00",
      "totalAmount": "100.00",
      "taxAmount": "5.79"
    },
    {
      "productName": "shipping_and_handling",
      "productSku": "shipping_and_handling",
      "quantity": 1,
      "unitPrice": "7.49",
      "totalAmount": "7.49",
      "taxAmount": "0.00"
    }
  ]
}
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Verify Status for PayPal and Venmo

When the status of your payment request is `PENDING`, you can verify the status by sending a POST request to the URL that is included in the **transactionStatus.url** field in the webhook response:

Webhook Response


```
{
  "transactionStatus": {
    "url":
 "https://apitest.cybersource.com/pts/v2/refresh-payment-status/7640632277149132395
5899488",
    "method": "POST",
    "payload": {
      "clientReferenceInformation": {
        "applicationName": "unifiedCheckout"
      },
      "processingInformation": {
        "actionList": [
          "AP_STATUS"
        ]
      },
      "paymentInformation": {
        "paymentType": {
          "method": {
            "name": "payPal"
          },
          "name": "eWallet"
        }
      }
    }
  },
  "transactionResult": {
    "details": {
      "submitTimeUtc": "2025-11-25T09:31:14Z",
      "processorInformation": {
        "sellerProtection": {
          "eligibility": "ELIGIBLE",
          "disputeCategories": [
            "ITEM_NOT_RECEIVED",
            "UNAUTHORIZED_TRANSACTION"
          ]
        },
        "transactionId": "14815899TN877504A",
        "orderStatus": "COMPLETED",
        "orderId": "74902340T07048209",
        "updateTimeUtc": "2025-11-25T09:33:48Z",
        "expirationTimeUtc": "2025-12-24T09:33:48Z"
```

```
      },
      "orderInformation": {
        "amountDetails": {
          "totalAmount": "0.11",
          "currency": "USD"
        },
        "billTo": {
          "email": "jane.doe@visa.com",
          "lastName": "Doe",
          "firstName": "Jane"
        }
      },
      "updateTimeUtc": "2025-11-25T09:33:48Z",
      "buyerInformation": {
        "merchantCustomerId": "JS4EXZRT68ED8"
      },
      "message": "Successful",
      "createTimeUtc": "2025-11-25T09:33:48Z",
      "clientReferenceInformation": {
        "code": "default"
      },
      "reconciliationId": "76406307371191323955899488",
      "status": "COMPLETED",
      "id": "76406322771491323955899488"
    },
    "id": "76406322771491323955899488",
    "message": "Successful",
    "outcome": "COMPLETED",
    "status": "COMPLETED"
  }
}
```


You can also send a request to this endpoint to verify the status:

Production: `POST https://api.cybersource.com/pts/v2/refresh-payment-status/{id}`

Test: `POST https://apitest.cybersource.com/pts/v2/refresh-payment-status/{id}`

The *{id}* is ID that is returned in the webhook response. For more information, see Webhooks Support (on page 176).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

Handle Responses

When Unified Checkout automatically processes a payment with `autoProcessing` is set to `true` or you have set `autoProcessing` to `false` and are using **checkout.Complete()**, you must handle both successful responses and various errors. After the payment is complete, the **completeResponse** field object contains information about the transaction outcome.

When a payment is processed successfully, you must parse the response to confirm the payment status, update their order records, and trigger any post-payment workflows. Post-payment workflows include sending confirmation emails or updating inventory. See JavaScript Example: Processing a Payment (on page 139).

Your error handling should account for specific cases such as `COMPLETE_TRANSACTION_CANCELED` and `COMPLETE_TRANSACTION_FAILED`. `COMPLETE_TRANSACTION_CANCELED` occurs when the user cancels the transaction and `COMPLETE_TRANSACTION_FAILED` indicates that the consumer's transaction failed.

For wallet based payments, only cancellation errors are returned. For information about possible errors that can occur when calling the complete API, see UnifiedCheckoutError (on page 232) in Handle Errors (on page 231).

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Captures

When you set the **completeMandate.type** field value to `AUTH` or `PREFER_AUTH`, you must send a request to capture an authorized payment. Full and partial captures are supported.

Endpoint

Production: `POST https://api.cybersource.com/pts/v2/payments/{id}/captures`

Test: `POST https://apitest.cybersource.com/pts/v2/payments/{id}/captures`

The `{id}` is the transaction ID returned in the authorization response.

Example: Authorization Response from Unified Checkout


```
{
  "details": {
    "clientReferenceInformation": {
      "code": "1753351101383"
    },
    "orderInformation": {
      "amountDetails": {
        "currency": "USD",
        "totalAmount": "21.00"
      }
    },
    "processorInformation": {
      "approvalCode": "AUTH456789",
      "responseCode": "00003",
      "responseDetails": "00003",
      "transactionId": "201601180815391001180815 3AUTH"
    },
    "reconciliationId": "04RYADD29YRO",
    "submitTimeUtc": "2025-07-24T09:58:21Z"
  },
  "id": "753351101428697180 3092",
  "message": "Request processed successfully.",
  "outcome": "AUTHORIZED",
  "status": "AUTHORIZED"
}
```


Response Status

Cybersource responds to your capture request with one of these statuses:

- `FAILED`: The capture request failed.
- `PENDING`: The capture request is accepted but not captured. Send a request to the check status service to retrieve status updates.
- `SETTLED`: The capture request is settled for the amount requested.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide

API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Integration Methods

Unified Checkout offers flexible integration methods for your page's checkout experience. This section contains information about each integration method and the JavaScript that is required for each one.

This table outlines the different integration methods offered by Unified Checkout:

Unified Checkout Integration Methods

	Checkout	Button	Trigger	Components
What does it do?	Displays all configured payment methods in a single Unified Checkout experience. The display order is determined from the Business CenterMerchant Experience or the Sessions API.	Enables you to place individual Unified Checkout payment buttons on your payment page.	Enables you to use your own button to launch the hosted payment experience, for example card entry or Click to Pay, in a sidebar.	Enables you to embed individual payment form elements, for example, card fields or the Click to Pay experience, directly into your page for full control over the checkout experience.
What is it best for?	When you want a quick out-of- the-box checkout with a list of payment methods.	Merchants that want flexibility in how payment buttons are displayed across their page.	Merchants that want to keep their own checkout button but use Unified Checkout to handle the payment flow.	Merchants that want full control over the UI and want to fully embed the payment form into their own checkout.
Who owns the payment button(s)?	Unified Checkout	Unified Checkout	Merchant	Merchant
Display modes	Sidebar or embedded	Embedded	Sidebar	Embedded
Supported payment methods	All configured methods	All available methods	Card payment and Click to Pay	Card payment, Click to Pay, and eCheck/ACH service

Unified Checkout Integration Methods (continued)

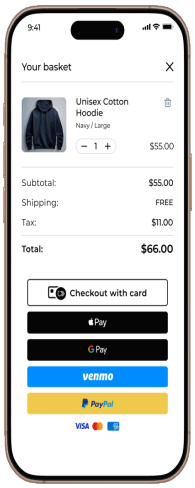
	Checkout	Button	Trigger	Components
Example uses	A merchant that requires a button list of all accepted payment methods in Unified Checkout.	A merchant that wants to host buttons separately on their page. For example, displaying Apple Pay at the top of the page and card entry at the bottom of the page.	A merchant that has an existing pay by card/Click to Paybutton that they want to launch into the payment UI using a sidebar view.	A merchant that wants to embed Unified Checkout payment components directly into their page without the standard header. This integration method is ideal for a user that offers a radio button/drop-down UI.
Summary	**Full out-of-the-box checkout**	**Flexible placement of payment buttons**	**Use your own button to launch the payment UI**	**Fully embedded and customized payment experience**
Documentation	Checkout (on page 114)	Button (on page 118)	Trigger (on page 121)	Components (on page 124)

Checkout

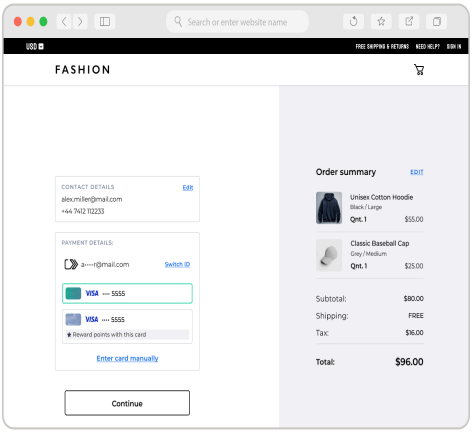
Checkout UI

This image shows an example of the Unified Checkout checkout UI. You can display the checkout UI in embedded or sidebar mode.

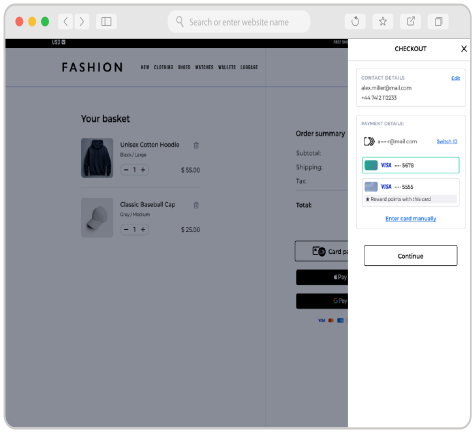
Unified Checkout Checkout UI



Mobile



Embedded Mode



Sidebar Mode

Checkout Integration JavaScript Example

```
// ============================================================
// CHECKOUT API - USAGE GUIDE
// ============================================================

// ============================================================
// FLOW 1: AUTO-PROCESSING (autoProcessing: true)
// Mount returns complete payment result
// Defaults to true when completeMandate is in session
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);

// Create checkout with auto-processing enabled
const checkout = await client.createCheckout({ autoProcessing: true });

// Mount - automatically processes payment and returns complete result
const paymentResult = await checkout.mount('#payment-buttons');
console.log('Payment complete:', paymentResult);

// Clean up
checkout.destroy();
client.destroy();


// ============================================================
// FLOW 2: MANUAL COMPLETE (autoProcessing: false)
// Mount returns transient token, then the merchant can manually complete payment
```

```
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);

// Create checkout with auto-processing disabled
const checkout = await client.createCheckout({ autoProcessing: false });

// Mount - returns transient token only
const token = await checkout.mount('#payment-buttons');
console.log('Received token:', token);

// Manually complete the payment
const paymentResult = await checkout.complete(token);
console.log('Payment complete:', paymentResult);

// Clean up
checkout.destroy();
client.destroy();


// ============================================================
// FLOW 3: TRANSIENT TOKEN ONLY (autoProcessing: false, no complete)
// Mount returns transient token only — server handles payment authorization
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);

// Create checkout with auto-processing disabled
const checkout = await client.createCheckout({ autoProcessing: false });

// Mount - returns transient token only
const transientToken = await checkout.mount('#payment-buttons');
console.log('Received token:', transientToken);

// Pass transient token to your server for payment authorization
sendToServer(transientToken);

// Clean up
checkout.destroy();
client.destroy();


// ============================================================
// MOUNT OPTIONS
// ============================================================

// Sidebar - payment selection and form rendered in the same container
```

```
const result = await checkout.mount('#payment-buttons');

// Embedded - payment selection and form rendered in separate containers
const result = await checkout.mount({
    paymentSelection: '#payment-buttons',
    paymentScreen: '#payment-form'
});


// ============================================================
// LIFECYCLE METHODS (available in all flows)
// ============================================================

// Mount - display the checkout
const result = await checkout.mount('#payment-buttons');

// Destroy checkout instance
checkout.destroy();

// Destroy client instance
client.destroy();


// ============================================================
// ERROR HANDLING
// ============================================================

try {
    const client = await window.VAS.UnifiedCheckout(sessionJWT);
    const checkout = await client.createCheckout();
    const result = await checkout.mount('#payment-buttons');
    sendToServer(result);
} catch (error) {
    if (error.name === 'UnifiedCheckoutError') {
        handleError(error.reason, error.message);
    }
} finally {
    checkout.destroy();
    client.destroy();
}


// ============================================================
// SUPPORTED PAYMENT TYPES
// ============================================================

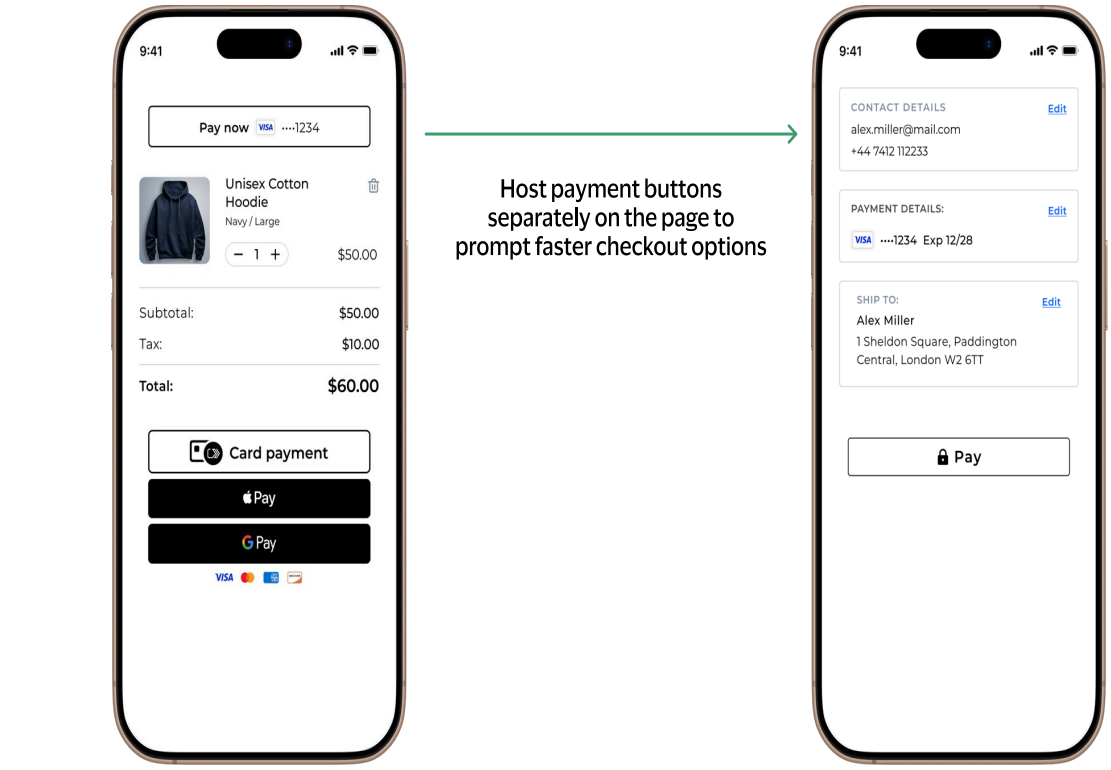
// All payment types configured in the session are displayed
// checkout.mount() renders buttons for every type in the session
```

Button

Button UI

This image shows the Unified Checkout button UI.

Unified Checkout Button UI



Button Integration JavaScript Example

```
// ============================================================
// BUTTON API - USAGE GUIDE
// ============================================================

// ============================================================
// FLOW 1: AUTO-PROCESSING (autoProcessing: true)
// Mount returns complete payment result
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);

// Create button with auto-processing enabled
const button = client.createButton('GOOGLEPAY', {
    autoProcessing: true
```

```
});

// Mount - automatically processes payment and returns complete result
const paymentResult = await button.mount('#payment-container');
console.log('Payment complete:', paymentResult);

// Clean up
button.destroy();


// ============================================================
// FLOW 2: MANUAL COMPLETE (autoProcessing: false)
// Mount returns transient token, then the merchant can manually complete payment
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);

// Create button with auto-processing disabled
const button = client.createButton('GOOGLEPAY', {
    autoProcessing: false
});

// Mount - returns transient token only
const token = await button.mount('#payment-container');
console.log('Received token:', token);

// Manually complete the payment
const paymentResult = await button.complete(token);
console.log('Payment complete:', paymentResult);

// Clean up
button.destroy();


// ============================================================
// FLOW 3: TRANSIENT TOKEN ONLY (autoProcessing: false, no complete)
// Mount returns transient token only.
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);

// Create button with auto-processing disabled
const button = client.createButton('GOOGLEPAY', {
    autoProcessing: false
});

// Mount - returns transient token only
```

```
const token = await button.mount('#payment-container');
console.log('Received token:', token);

// Clean up
button.destroy();


// ============================================================
// LIFECYCLE METHODS (available in all flows)
// ============================================================

// Mount - display the payment button
const result = await button.mount('#payment-container');

// Unmount - hide the button without destroying it
button.unmount();

// Remount - can mount again after unmounting
await button.mount('#payment-container');

// Check mounted state
const mounted = button.isMounted();  // true or false

// Destroy - permanently destroy the button
button.destroy();

// Check destroyed state
const destroyed = button.isDestroyed();  // true or false


// ============================================================
// EVENT HANDLING (optional for all flows)
// ============================================================

// Subscribe to events
button.on('mounted', () => console.log('Button displayed'));
button.on('unmounted', () => console.log('Button hidden'));
button.on('destroyed', () => console.log('Button destroyed'));

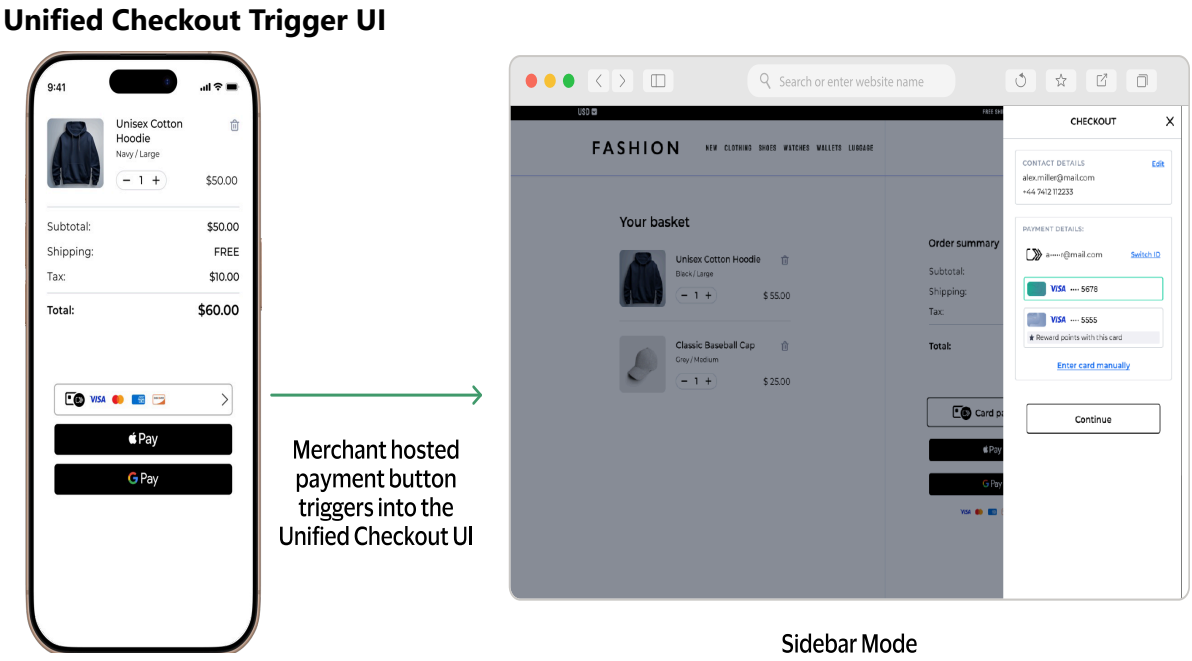

// ============================================================
// SUPPORTED PAYMENT TYPES
// ============================================================

client.createButton('GOOGLEPAY');  // Google Pay
client.createButton('APPLEPAY');   // Apple Pay
```

Trigger

Trigger UI

This image shows the Unified Checkout trigger UI.



Trigger Integration JavaScript Example

```
// ============================================================
// TRIGGER API - USAGE GUIDE V2
// ============================================================

// Triggers load the payment UI without displaying the Unified Checkout
// button. Use when a custom event should initiate the payment flow.
// Supported payment types: PANENTRY and CLICKTOPAY only.

// IMPORTANT: Trigger always opens in SIDEBAR mode only.
// The displayMode: 'sidebar' option must be set when creating the trigger.
// The merchant provides their own button/element to launch the payment flow.

// ============================================================
// FLOW 1: AUTO-PROCESSING (autoProcessing: true)
// Mount returns complete payment result
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);
```

```
// Create trigger — sidebar is the only supported display mode for Trigger
const trigger = client.createTrigger('PANENTRY', {
    displayMode: 'sidebar',
    autoProcessing: true
});

// Mount is called when the merchant's own button is clicked
// This opens the sidebar payment UI and resolves when payment is complete
document.getElementById('your-pay-button').addEventListener('click', async () => {
    const paymentResult = await trigger.mount('#payment-container');
    console.log('Payment complete:', paymentResult);
});

// Clean up
trigger.destroy();


// ============================================================
// FLOW 2: MANUAL COMPLETE (autoProcessing: false)
// Mount returns transient token, then the merchant can manually complete payment
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);

// Create trigger with auto-processing disabled
const trigger = client.createTrigger('PANENTRY', {
    displayMode: 'sidebar',
    autoProcessing: false
});

// Mount is called when the merchant's own button is clicked
document.getElementById('your-pay-button').addEventListener('click', async () => {
    const token = await trigger.mount('#payment-container');
    console.log('Received token:', token);

    // Manually complete the payment
    const paymentResult = await trigger.complete(token);
    console.log('Payment complete:', paymentResult);
});

// Clean up
trigger.destroy();


// ============================================================
// FLOW 3: TRANSIENT TOKEN ONLY (autoProcessing: false, no complete)
// Mount returns transient token only — server handles payment authorisation
```

```
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);

// Create trigger with auto-processing disabled
const trigger = client.createTrigger('PANENTRY', {
    displayMode: 'sidebar',
    autoProcessing: false
});

// Mount is called when the merchant's own button is clicked
document.getElementById('your-pay-button').addEventListener('click', async () => {
    const token = await trigger.mount('#payment-container');
    console.log('Received token:', token);

    // Pass transient token to your server for payment authorisation
    sendToServer(token);
});

// Clean up
trigger.destroy();


// ============================================================
// LIFECYCLE METHODS (available in all flows)
// ============================================================

// Mount - opens the sidebar payment UI (called from merchant's own click handler)
const result = await trigger.mount('#payment-container');

// Unmount - hide the form without destroying it
trigger.unmount();

// Remount - can mount again after unmounting
await trigger.mount('#payment-container');

// Check mounted state
const mounted = trigger.isMounted();  // true or false

// Destroy - permanently destroy the trigger
trigger.destroy();

// Check destroyed state
const destroyed = trigger.isDestroyed();  // true or false


// ============================================================
// EVENT HANDLING (optional for all flows)
```

```
// ============================================================

// Subscribe to events
trigger.on('mounted', () => console.log('Sidebar payment UI displayed'));
trigger.on('unmounted', () => console.log('Sidebar payment UI hidden'));
trigger.on('error', (err) => console.error('Trigger error:', err.code,
 err.message));
trigger.on('destroyed', () => console.log('Trigger destroyed'));


// ============================================================
// SUPPORTED TRIGGER TYPES
// ============================================================

// IMPORTANT: When using createTrigger() for Click to Pay,
// you must create a custom UI. See Click to Pay UI Guidelines.

client.createTrigger('PANENTRY', { displayMode: 'sidebar' });   // Manual card
 entry
client.createTrigger('CLICKTOPAY', { displayMode: 'sidebar' }); // Click to Pay


// ============================================================
// CAPTURE CONTEXT REQUIREMENT
// ============================================================

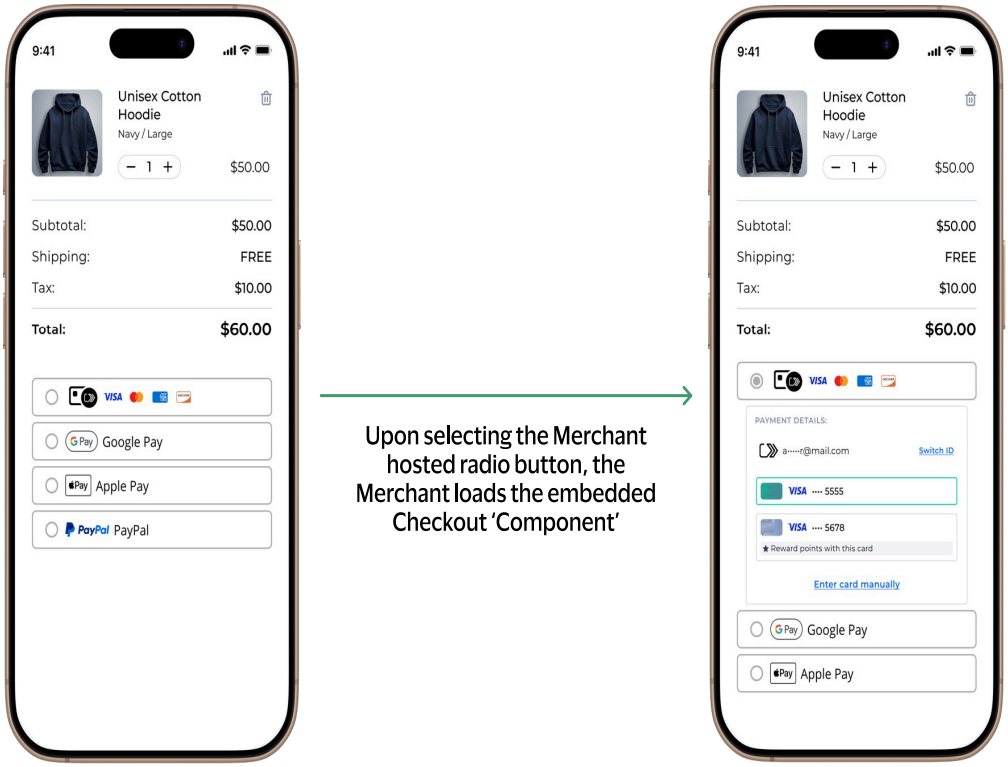
// The capture context allowedPaymentTypes must include the payment type
// used in createTrigger(). For PANENTRY, ensure your session request includes:
//
// "allowedPaymentTypes": ["PANENTRY", ...]
//
// Without PANENTRY in allowedPaymentTypes, createTrigger('PANENTRY') will
// fail silently and the sidebar will not open.
```

Components

Components UI

This image shows the Unified Checkout components UI.

Unified Checkout Components UI



Components Integration JavaScript Example

```
// ============================================================
// COMPONENTS API - USAGE GUIDE
// ============================================================

// ============================================================
// FLOW 1: AUTO-PROCESSING (autoProcessing: true)
// Mount returns complete result
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);

// Create component with auto-processing enabled
const component = client.createComponent('PANENTRY', {
    autoProcessing: true
});

// Mount - automatically processes payment and returns complete result
const paymentResult = await component.mount('#payment-container');
console.log('Payment complete:', paymentResult);

// Clean up
```

```
component.destroy();


// ============================================================
// FLOW 2: MANUAL COMPLETE (autoProcessing: false)
// Mount returns transient token, then the merchant can manually complete payment
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);

// Create component with auto-processing disabled
const component = client.createComponent('PANENTRY', {
    autoProcessing: false
});

// Mount - returns transient token only
const token = await component.mount('#payment-container');
console.log('Received token:', token);

// Manually complete the payment
const paymentResult = await component.complete(token);
console.log('Payment complete:', paymentResult);

// Clean up
component.destroy();


// ============================================================
// FLOW 3: TRANSIENT TOKEN ONLY (autoProcessing: false, no complete)
// Mount returns transient token only.
// ============================================================

// Initialize the Unified Checkout client
const client = await window.VAS.UnifiedCheckout(sessionJWT);

// Create component with auto-processing disabled
const component = client.createComponent('PANENTRY', {
    autoProcessing: false
});

// Mount - returns transient token only
const token = await component.mount('#payment-container');
console.log('Received token:', token);

// Clean up
component.destroy();
```

```
// ============================================================
// LIFECYCLE METHODS (available in all flows)
// ============================================================

// Mount - display the payment form
const result = await component.mount('#payment-container');

// Unmount - hide the form without destroying it
component.unmount();

// Remount - can mount again after unmounting
await component.mount('#payment-container');

// Check mounted state
const mounted = component.isMounted();  // true or false

// Destroy - permanently destroy the component
component.destroy();

// Check destroyed state
const destroyed = component.isDestroyed();  // true or false


// ============================================================
// EVENT HANDLING (optional for all flows)
// ============================================================

// Subscribe to events
component.on('mounted', () => console.log('Form displayed'));
component.on('unmounted', () => console.log('Form hidden'));
component.on('destroyed', () => console.log('Component destroyed'));

// ============================================================
// SUPPORTED PAYMENT TYPES
// ============================================================

client.createComponent('PANENTRY');    // Manual card entry
client.createComponent('CLICKTOPAY');  // Click to Pay
client.createComponent('CHECK');       // Check payment (US & CA Only)
```

When `CLICKTOPAY` is included in **allowedPaymentTypes**, use `client.createComponent('CLICKTOPAY')` and not `client.createComponent('PANENTRY')`. `CLICKTOPAY` renders the full Click to Pay lookup flow and automatically displays an option to pay with a new card for users without a Click to Pay profile. This option ensures that the PAN entry flow is also included. Using only `PANENTRY` bypasses Click to Pay, even if it is included in your capture context.

This table provides an overview of how to use the components integration method:

Components Integration Method

allowedPaymentTypes Method	Components Use
`CLICKTOPAY` with or without `PANENTRY`	`createComponent('CLICKTOPAY', …)`
`PANENTRY` only	`createComponent(PANENTRY, …)`
Multiple wallets + `CLICKTOPAY`	`createCheckout(…)` The Checkout API renders all checkout options automatically.

Important: There is no error if you load `PANENTRY` when `CLICKTOPAY` is included in your capture context. The component will be mounted successfully, but Click to Pay is not shown in the UI. This may result in one-time password emails on the server-side but no Click to Pay UI displaying on the client side.

Server-Side Set Up

This section contains the information you need to set up your server. Initializing Unified Checkout within your webpage begins with a server-to-server call to the Sessions API. This step authenticates your merchant credentials, and establishes how the Unified Checkout frontend components will function. The Sessions API request contains parameters that define how Unified Checkout performs.

The server-side component provides this information:

- A transaction-specific public key is used by the customer's browser to protect the transaction.
- An authenticated context description package that manages the payment experience on the client side. It includes available payment options such as card networks, payment interface styling, and payment methods.

The functions are compiled in a JSON Web Token (JWT) object referred to as the *capture context*. For information JSON Web Tokens, see JSON Web Tokens (on page 266).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Capture Context

This section contains the information you need to set up your server. Initializing Unified Checkout within your webpage begins with a server-to-server call to the Sessions API. This step authenticates your merchant credentials, and establishes how the frontend components will function. The Sessions API request contains parameters that define how Unified Checkout performs.

The server-side component provides this information:

- A transaction-specific public key is used by the customer's browser to protect the transaction.
- An authenticated context description package that manages the payment experience on the client side. It includes available payment options such as card networks, payment interface styling, and payment methods.

The functions are compiled in a JSON Web Token (JWT) object referred to as the *capture context*.

For information on JWTs see JSON Web Tokens (on page 266).

The capture context request is a signed JSON Web Token (JWT) that includes all of the merchant-specific parameters. This request tells the frontend JavaScript library how to behave within your payment experience. The request provides authentication, one-time keys, the target origin to the Unified Checkout integration in addition to allowed card networks and payment types.

Browser Support

Unified Checkout supports these browser versions:

- Safari 16
- Firefox 121
- Google Chrome/Chium-based browsers 118
- Microsoft Edge 118

Capture Context Example

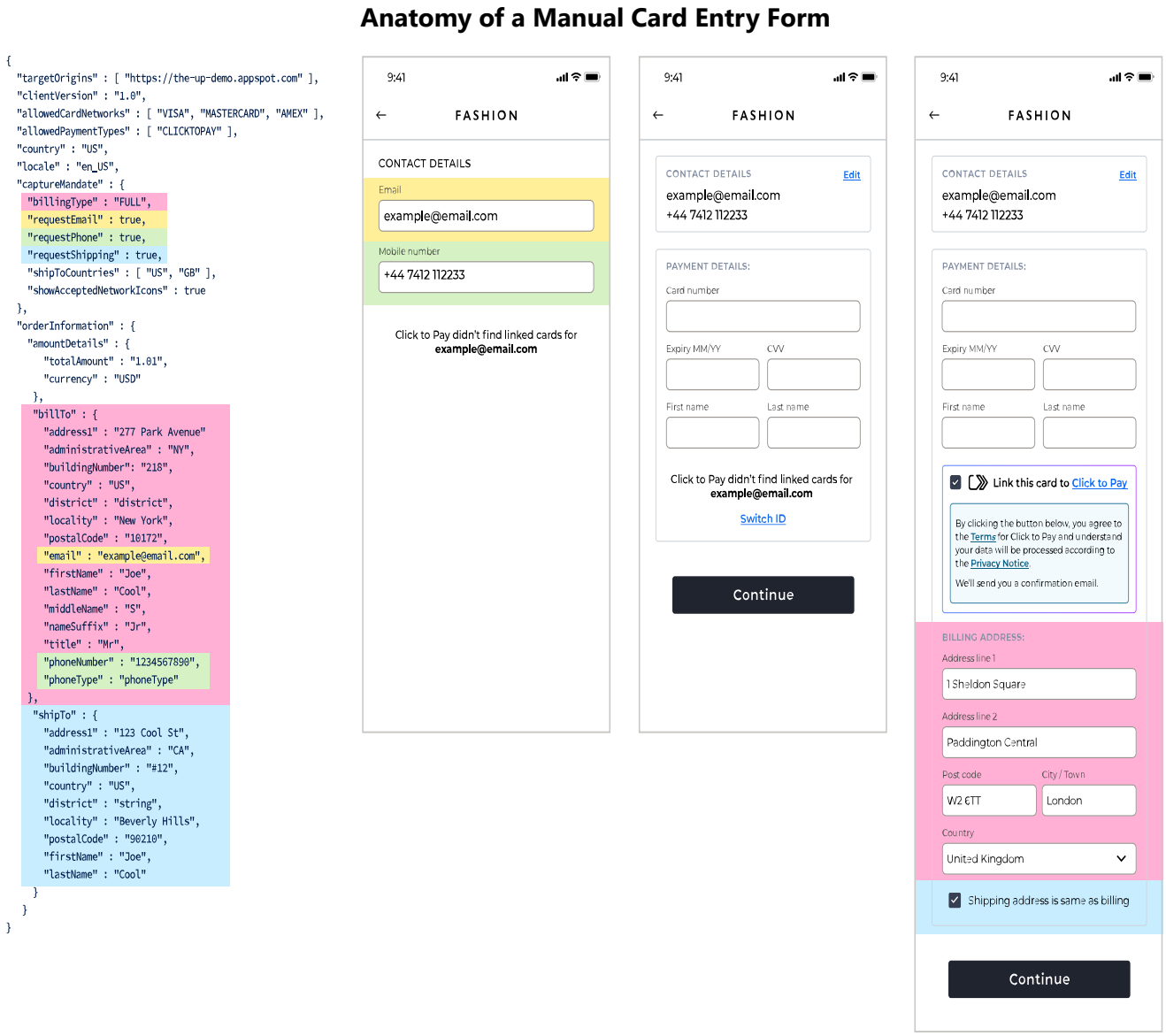
Use the **targetOrigins** and the **allowedPaymentTypes** fields to define the target origin and the accepted digital payment methods in your capture context. Use the **completeMandate** to orchestrate follow-on services such as Payments, Decision Manager, Payer Authentication, and TMS. The data that is included in the capture context are configured in the merchant experience. For information about how to configure these fields, see Configure Payment Options (on page 152) Configure Customer Data and Payment Flow (on page 165). For information about what fields can be overridden by what is included in the capture context, see Capture Context Fields in the Business Center (on page 167).

This example shows a capture context with the minimum required fields:

```
{
  "targetOrigins": ["https://merchant.com", "https://reseller.com:8443"],
  "locale": "en_US",
  "country": "US",
  "data": {
 "orderInformation": {
     "amountDetails": {
       "totalAmount": "21.00",
       "currency":  "USD"
          }
      }
  }
}
```

Card Entry Form

This diagram shows how elements of the capture context request appear in the card entry form.



Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Versioning

Unified Checkout uses Semantic Versioning (SemVer). Version numbers use the `MAJOR.MINOR.PATCH` format:

- **MAJOR**: breaking changes that require code modifications
- **MINOR**: new features that are backwards compatible
- **PATCH**: bug fixes and improvements that are backwards compatible

The server controls which SDK version is loaded for each session that you request. When your server creates a session, the response JWT includes a **clientLibrary** field that contains the full URL to the correct version of the SDK. Your server parses the JWT, extracts the URL, and passes it to the frontend to load dynamically.

Important: The `clientVersion` field in the session request is optional. When you do not include this field, the server automatically resolves the appropriate version for every session. This ensures that the client-side library and server-side features are compatible.Cybersource recommends that you do not include the `clientVersion` field in your request and that you use the most recent version. When you do this, your integration benefits from new features, payment methods, and improvements and there are no code changes required.

Pin to a Version

If you must set your integration to a specific version, you can set the `clientVersion` field to a `MAJOR` version such as `1`, or a `MAJOR.MINOR` version such as `1.2`. The server uses the latest compatible patch release within that range. This ensures that you continue to receive security fixes and bug fixes.

Important: You cannot pin to a specific patch version (`MAJOR.MINOR.PATCH`). This ensures that all integrations receive critical patches. Cybersource recommends omitting **clientVersion** from your request unless you have a specific need for pinned behavior.

For information about the latest releases, see Client Version History (on page 133).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Client Version History

Below is a list of client versions and the features that are included in each version.

Important: Cybersource recommends that you use the most recent client version in your integration.

0.23

Accepts these card networks in the **allowedCardNetworks** field for manual card entry:

- Carnet
- Cartes Bancaires
- China UnionPay with card verification value (CVV)
- EFTPOS
- ELO
- JCrew PLCC
- mada
- Meeza

Ordering controls for the **allowedPaymentTypes** button.

De-coupling of PANENTRY from other payment types in the **allowedPaymentTypes** field.

0.24

Support for enabling combo cards in the capture context.

Support for eight-digit BINs.

Support for enabling card save in the capture context.

0.25

Addition of **Skip Verification next time** in the Click to Pay payment flow.

Support for CPF in the capture context.

0.26

Support for auto-lookup in Click to Pay when an email is included in the capture context.

Inclusion of the **cardDetails** field object in the transient token response.

Support for the **cardholderAuthenticationStatus** field object in the transient token response.

Support for the complete mandate.

0.28

Complete mandate enhancement to support Payer Authentication for manual card entry for Visa, Mastercard, American Express, Discover, JCB, Cartes Bancaires, China UnionPay, and ELO card brands.

Support for Afterpay as an **allowedPaymentType**.

Support for PayPak as an **allowedCardNetwork**.

Auto-enrollment for Click to Pay in supported markets.

Removal of the confirm or continue screen for specific use cases.

Static button for Click to Pay flows.

0.30

Support for iDeal, Multibanco, and Przelewy24|P24.

Complete mandate enhancement to support Payer Authentication for Google Pay and Click to Pay.

Support for Pakistan locales (en_PK and ur_PK).

New look and feel of Unified Checkout in line with EMVCO best practices.

0.31

Addition of the **data** object of the **orderInformation** field object and pass-through fields.

Support for **tokenCreate** in the complete mandate.

Support of pass-through fields, including challenge codes and data only, for Payer Authentication.

Support for Jaywan as an **allowedCardNetwork**.

Updated the payment details response to return detected card types. Multiple card types are shown when more than one card type is detected.

Support for Bancontact, Dragonpay, MyBank, and Tink Pay By Bank.

0.32

Support for KCP and UATP in the **allowedCardNetwork** field.

Support for Konbini as a post-pay reference payment method.

A radio button in the UI for Cartes Bancaires dual-branded cards.

0.33

Support for mobile as identity for Click to Pay accounts.

Japanese language translation updates.

UX captures billing and shipping information when they are not included in the capture context.

0.34

Iframes are used instead of pop-ups to reduce pop-up blocking and streamlining mobile deployment.

Additional BIN range for Jaywan card types.

0.35

Look and feel customization.

1.0

Configure payment options.

Configure customer data and payment flow.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Client-Side Set Up

This section contains the information you need to set up the client side. You use the Unified Checkout JavaScript library to add the payment interface to your e-commerce site. It has two primary components:

- The button widget, which lists the payment methods available to the customer.
- The payment acceptance page, which captures payment information from the cardholder. You can set up the payment acceptance page to be embedded with your webpage or added as a sidebar.

Follow these steps to set up the client:

1. Load the JavaScript library.
2. Initialize the accept object, the capture context JWT. For information JSON Web Tokens, see JSON Web Tokens (on page 266).
3. Initialize the unified payment object with optional parameters.
4. Show the button list or payment acceptance page or both.
5. Process the payment request using the instructions included within the capture mandate.

The response to these interactions is a transient token that you can use to retrieve the payment information captured by the UI.

For information about handling the errors that may occur on the client-side, see Handle Errors (on page 231).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Loading the JavaScript Library

Use the client library asset path and client library integrity value that is returned by the capture context response to invoke Unified Checkout on your page.

You must retrieve these values from the **clientLibrary** and **clientLibraryIntegrity** fields that are returned in the JWT from `https://apitest.cybersource.com/uc/v1/sessions`. You can use these values to create your script tags.

You must perform this process for each transaction, as these values are unique for each transaction. You must avoid hard-coding values for the **clientLibrary** and **clientLibraryIntegrity** fields to prevent client-side errors.

For example, a response from `https://apitest.cybersource.com/uc/v1/sessions` would include:

```
"data": {
    "clientLibrary":"[EXTRACT clientLibrary VALUE from here]",
    "clientLibraryIntegrity": "[EXTRACT clientLibraryIntegrity VALUE from here]"
}
```

Below is an example script tag:

```
<script src="[INSERT clientLibrary VALUE HERE]"
    integrity=”[INSERT clientLibraryIntegrity VALUE HERE]”
    crossorigin=”anonymous”></script>
```

Important: Use the **clientLibrary** and **clientLibraryIntegrity** parameter values in the capture context response to obtain the Unified Checkout JavaScript library URL and the integrity value. This ensures that you are always using the most up-to-date library and protects against fraud. Do not hard-code the Unified Checkout JavaScript library URL or integrity value.

When you load the library, the capture context from your initial server-side request is used to invoke the accept function.

For information about the client-side API, see JavaScript API Reference (on page 244).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

JavaScript Example: Initializing the SDK

```
const client = await VAS.UnifiedCheckout(sessionJWT);
```

In this example, `sessionJWT` refers to the capture context JWT.

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

JavaScript Example: Displaying the Button List

After you initialize the Unified Checkout object, you can add the payment application and payment acceptance pages to your webpage. You can attach the embedded Unified Checkout tool and payment acceptance pages to any named element within your HTML. Typically, they are attached to explicit named components that are replaced with Unified Checkout's iframes.

```
// Sidebar
const result = await checkout.mount('#buttons');

// Embedded
const result = await checkout.mount({
  paymentSelection: '#buttons',
  paymentScreen: '#form'
});
```

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

JavaScript Example: Client-Defined Trigger for Click to Pay or PAN Entry

When you display `CLICKTOPAY` or `PANENTRY` as allowed payment types, you can load the UI without displaying the Unified Checkout checkout button. You can do this by creating a trigger that defines what event loads the UI.

You can create a trigger only for `CLICKTOPAY` or `PANENTRY` payment methods:

```
//PAN Entry

const trigger = client.createTrigger('PANENTRY');

//Click to Pay

const trigger = client.createTrigger('CLICKTOPAY');
```

Important: When you use the `client.createTrigger()` method for Click to Pay, you must create a custom UI. See Click to Pay UI Guidelines (on page 60).

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

JavaScript Example: Processing a Payment

Payment is initiated when Unified Checkout captures the customer's payment information by calling the `client.createCheckout()`. When `autoProcessing` is set to `true`, the payment is completed. When `autoProcessing` is set to `false`, you can manually complete the payment using `checkout.complete(token)`. See Authorizations with a Transient Token (on page 220).

```
// Automatic (default when completeMandate is in session)
const checkout = await client.createCheckout({ autoProcessing: true });
const result = await checkout.mount('#buttons');
// result is the completed transaction – no need to call complete()

// Manual - similar to v0
const checkout = await client.createCheckout({ autoProcessing: false });
const token = await checkout.mount('#buttons');
const result = await checkout.complete(token);
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

JavaScript Example: Authorization

Collect payment information and process an authorization. You must initiate a separate capture request to move funds and complete the transaction.

```
async function launchCheckout() {
  try {
    const client = await VAS.UnifiedCheckout(sessionJWT);
    const checkout = await client.createCheckout();
    const result = await checkout.mount('#payment-buttons');

    // result contains the completed payment result JWT
    // Send result to your server for verification
    sendToServer(result);
  } catch (error) {
    if (error.name === 'UnifiedCheckoutError') {
      handleError(error.reason, error.message);
    }
  } finally {
    checkout.destroy();
    client.destroy();
  }
}

launchCheckout();
```

Because the session includes **completeMandate**, `autoProcessing` defaults to `true` and `mount()` returns the completed payment result directly.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

JavaScript Example: Sale

Collect payment information and process a sale. A sale is a combined authorization and capture in a single step.

```
async function launchCheckout() {
  try {
    const client = await VAS.UnifiedCheckout(sessionJWT);
    const checkout = await client.createCheckout();
    const result = await checkout.mount('#payment-buttons');

    // result contains the completed payment result JWT
    // Send result to your server for verification
    sendToServer(result);
  } catch (error) {
    if (error.name === 'UnifiedCheckoutError') {
      handleError(error.reason, error.message);
    }
  } finally {
    checkout.destroy();
    client.destroy();
  }
}

launchCheckout();
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

JavaScript Example: Sale with Decision Manager

Collect payment information and process a sale while running Decision Manager fraud screening before the payment is initiated.

```
async function launchCheckout() {
  try {
    const client = await VAS.UnifiedCheckout(sessionJWT);
    const checkout = await client.createCheckout();
    const result = await checkout.mount('#payment-buttons');

    // result contains the completed payment result JWT
    // Send result to your server for verification
    sendToServer(result);
  } catch (error) {
    if (error.name === 'UnifiedCheckoutError') {
      handleError(error.reason, error.message);
    }
  } finally {
    checkout.destroy();
    client.destroy();
  }
}

launchCheckout();
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

JavaScript Example: No Service Orchestration

Collect payment information and receive a transient token. Your server handles payment authorization and any follow-on services directly.

```
async function launchCheckout() {
  try {
    const client = await VAS.UnifiedCheckout(sessionJWT);
    const checkout = await client.createCheckout({ autoProcessing: false });
    const transientToken = await checkout.mount('#payment-buttons');

    // transientToken is a JWT — send it to your server
    // Your server uses the transient token to authorize the payment
    sendToServer(transientToken);
  } catch (error) {
    if (error.name === 'UnifiedCheckoutError') {
      handleError(error.reason, error.message);
    }
  } finally {
    checkout.destroy();
    client.destroy();
  }
}

launchCheckout();
```

Without a **completeMandate**, `autoProcessing` defaults to `false` and the `mount()` call returns a transient token that you pass to your server for payment authorization. For information about how to use the transient token in an authorization request, see Transient Tokens (on page 214).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

JavaScript Example: Setting Up with Full Sidebar

```
<html>
<head>
  <script
    src="[INSERT clientLibrary VALUE HERE]"
    integrity="[INSERT clientLibraryIntegrity VALUE HERE]”
    crossorigin=”anonymous"
  ></script>
</head>
```

```
<body>
  <h1>Unified Checkout Integration</h1>
  <input
    type="hidden"
    name="sessionJWT"
    value="[INSERT sessionJWT HERE]"
  />
  <script type="text/javascript">
    const sessionJWT = document.getElementById("sessionJWT").value;

    async function launchCheckout() {
      try {
        const client = await VAS.UnifiedCheckout(sessionJWT);
        const checkout = await client.createCheckout();
        const result = await checkout.mount('#payment-buttons');

        // result contains the completed payment result JWT
        // Send result to your server for verification
        sendToServer(result);
      } catch (error) {
        if (error.name === 'UnifiedCheckoutError') {
          handleError(error.reason, error.message);
        }
      } finally {
        checkout.destroy();
        client.destroy();
      }
    }

    launchCheckout();
  </script>
</body>
</html>
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

JavaScript Example: Setting Up with the Embedded Component

The main difference between using an embedded component and the sidebar is that the `VAS.UnifiedCheckout(sessionJWT)` object is set to `false`, and the location of the payment screen is passed in the containers argument.



Important: If you do not specify a location for the payment acceptance page, it is placed in the side bar.

```
<html>
<head>
  <script
    src="[INSERT clientLibrary VALUE HERE]"
    integrity="[INSERT clientLibraryIntegrity VALUE HERE]"
    crossorigin="anonymous"
  ></script>
</head>
<body>
  <h1>Unified Checkout Integration</h1>
  <input
    type="hidden"
    id="sessionJWT"
    name="sessionJWT"
    value="[INSERT sessionJWT HERE]"
  />
  <script type="text/javascript">
    const sessionJWT = document.getElementById("sessionJWT").value;

    async function launchCheckout() {
      let client;
      let checkout;

      try {
        client = await VAS.UnifiedCheckout(sessionJWT);
        checkout = await client.createCheckout();
        const result = await checkout.mount('#payment-buttons');

        // result contains the completed payment result JWT
        // Send result to your server for verification
        sendToServer(result);
      } catch (error) {
        if (error.name === 'UnifiedCheckoutError') {
          handleError(error.reason, error.message);
        }
      } finally {
        if (checkout) {
```

```
          checkout.destroy();
        }

        if (client) {
          client.destroy();
        }
      }
    }

    launchCheckout();
  </script>
</body>
</html>
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Configuration

This section contains the information required to configure Unified Checkout.

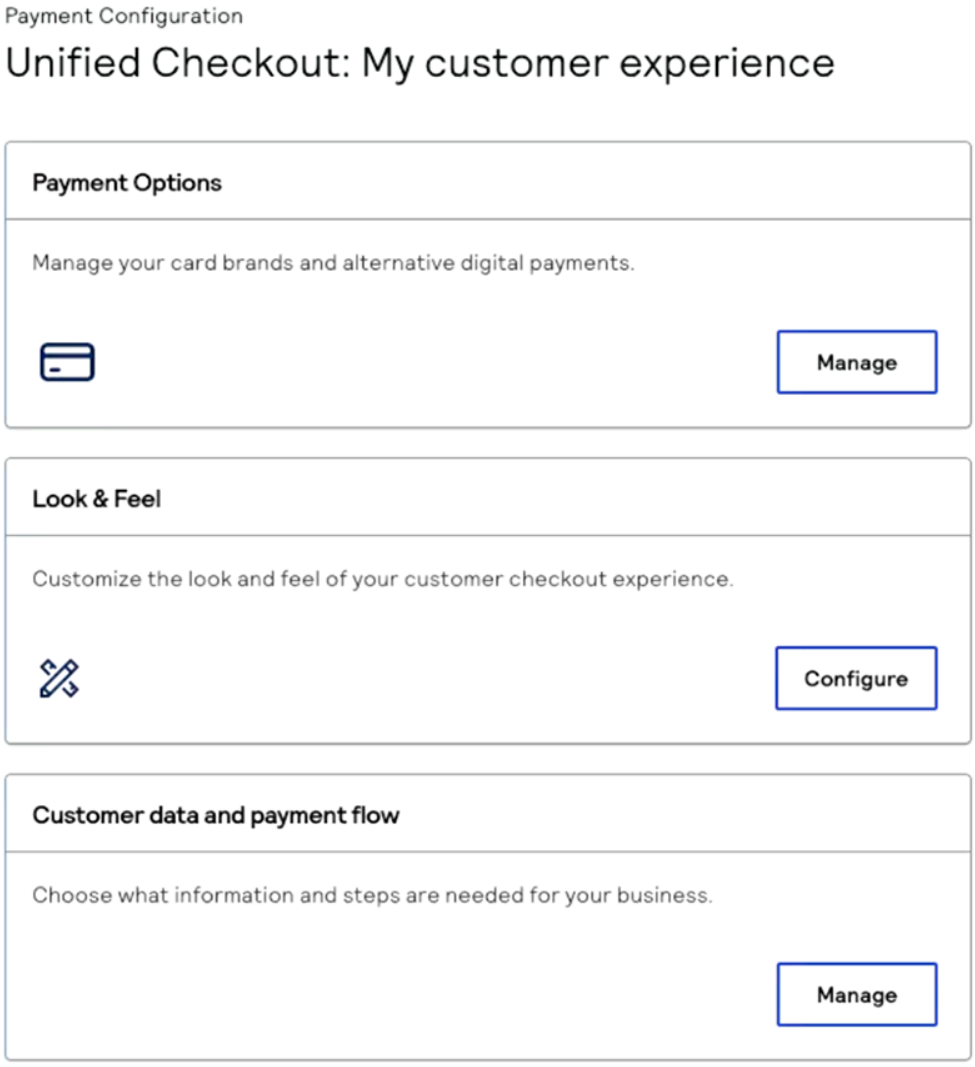
Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Configure the Unified Checkout Merchant Experience

The Unified Checkout merchant experience interface provides complete control over your checkout experience by using dedicated configuration screens accessible through the Business Center:

Unified Checkout Customer Experience



This option is low-code and provides a user-friendly approach to customization of your Unified Checkout UI.

Configure each of these components of the checkout experience:

- **Payment Options**: Add, remove, and arrange payment methods. For information about configuring payment methods, see Configure Payment Options (on page 152) and Process Payments with Unified Checkout (on page 174).
- **Look & feel**: Configure visual appearance and branding. For information about configuring the look and feel, see Customize the Unified Checkout Look and Feel (on page 155).
- **Customer information and payment flow**: Control data collection and manage payment processing service. For information about configuring customer data, see Configure Customer Data and Payment Flow (on page 165).

You can also manage permissions as a direct merchant or as a portfolio administrator. For information about managing permissions, see Manage Permissions (on page 171).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

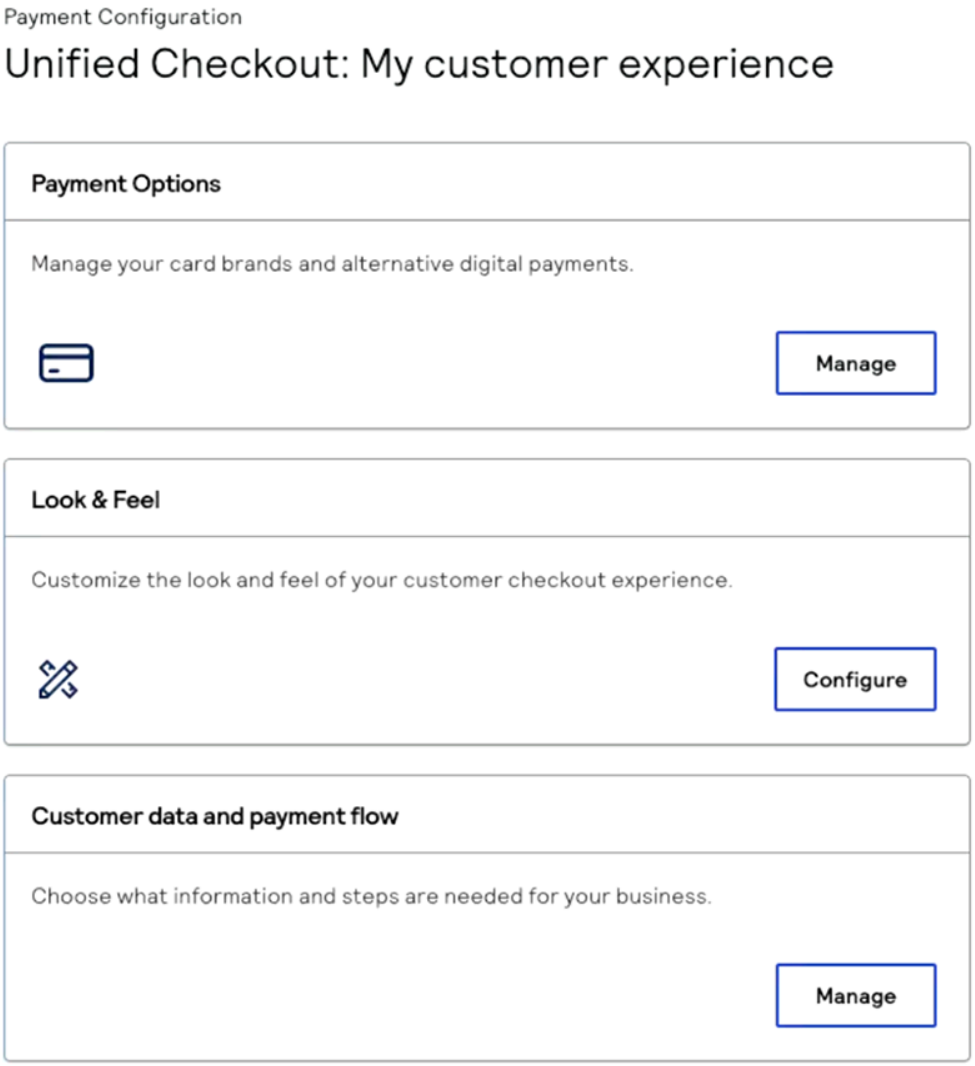
Enable Unified Checkout

To begin using Unified Checkout, you must first ensure that your merchant ID (MID) is configured to use the service and that any payment methods you intend to use are properly set up.

1. Log in to the Business Center:
   Test URL: `https://businesscentertest.cybersource.com/ebc2`
   Production URL: `https://businesscenter.cybersource.com`
   If you are unable to access this page, contact your sales representative.

2. In the Business Center, go to the left navigation panel and choose **Payment Configuration** > **Unified Checkout**. The Unified Checkout customer experience page appears:

Unified Checkout Customer Experience



3. The Unified Checkout configuration interface provides complete low-code control over your checkout experience by using dedicated configuration screens accessible through the Business Center. The configuration interface is organized into separate screens, each accessible from the *My customer experience* page. Configure each of these components of the checkout experience:

- **Payment Options**: Add, remove, and arrange payment methods. For information about configuring payment methods, see Configure Payment Options (on page 152).
- **Look & feel**: Configure visual appearance and branding. For information about configuring the look and feel, see Customize the Unified Checkout Look and Feel (on page 155).
- **Customer information and payment flow**: Control data collection and manage payment processing service. For information about configuring customer data, see Configure Customer Data and Payment Flow (on page 165).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

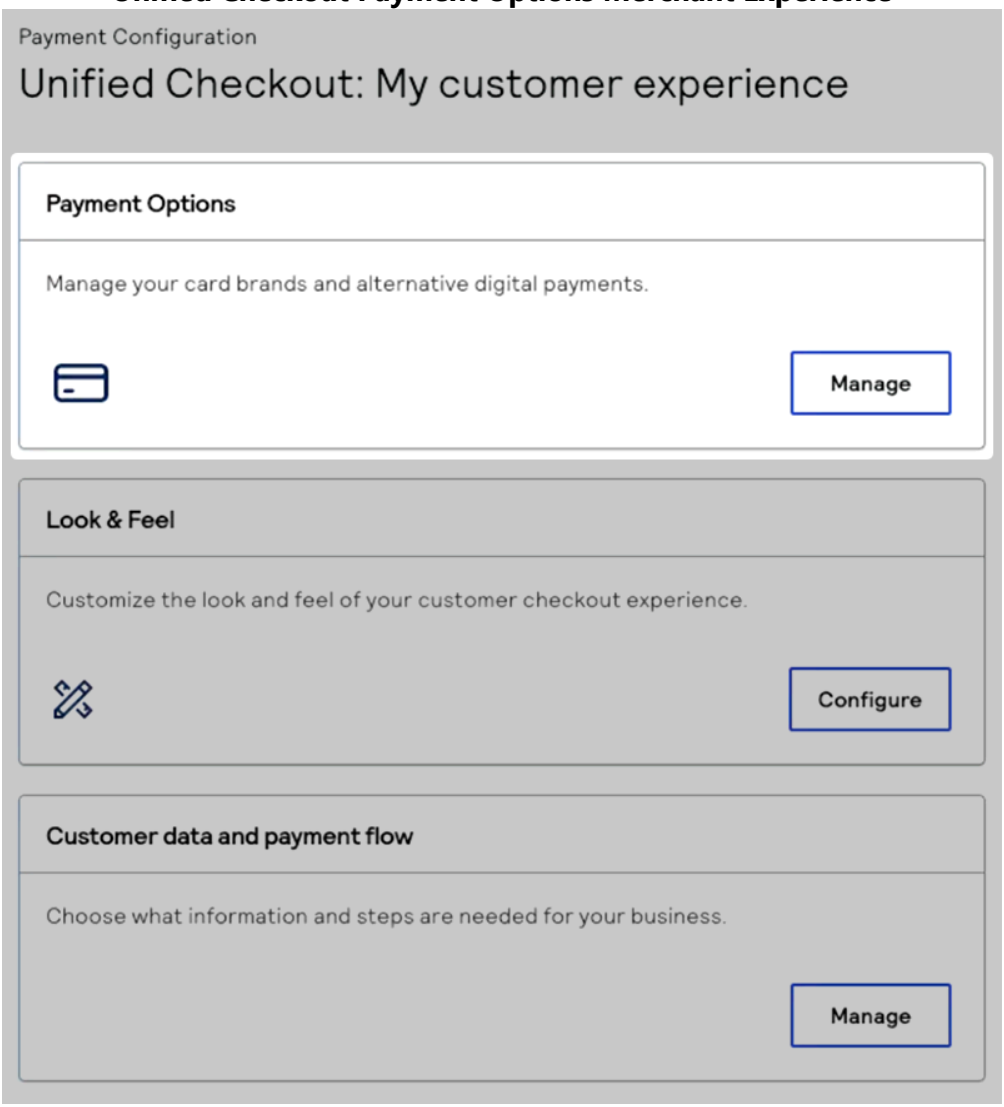
Configure Payment Options

Payment Options in the Business Center enable you to control which payment methods appear in your checkout and in what order. Follow these steps to customize the available payment options in Unified Checkout in the Business Center:

1. Log in to the Business Center:
   Test URL: `https://businesscentertest.cybersource.com/ebc2`
   Production URL: `https://businesscenter.cybersource.com`
   If you are unable to access this page, contact your sales representative.

2. In the Business Center, go to the left navigation panel and choose **Payment Configuration** > **Unified Checkout**. The Unified Checkout customer experience page appears:

Unified Checkout Payment Options Merchant Experience



3. In the Payment Options section, click **Manage**. The Payment Options page appears.

4. Under Payment options, click the checkbox next to each payment method that you want to display in your checkout UI. Click the drag icon (⠿) to rearrange the order of the payment options.

Important: Some payment options are set at the portfolio level and cannot be reordered. When you see a pin icon next to a payment option, you cannot move that payment option in the list of available methods. You must contact your portfolio administrator if you want to reorder the list of available payment options.

These payment options are supported by Unified Checkout:

- Apple Pay (on page 45)
- Click to Pay (on page 53)
- Google Pay (on page 48)
- eCheck (on page 40)
- Konbini (on page 95)
- Bancontact (on page 74)
- DragonPay (on page 75)
- iDeal (on page 78)
- MyBank (on page 80)
- Multibanco (on page 81)
- Przelewy24|P24 (on page 82)
- Tink Pay By Bank (on page 76)
- Afterpay (on page 88)

5. Click **Manage** next to each payment type that you want to configure. The configuration page for the selected payment type appears.
6. Under card brands, slide the toggle ( ) to display the card brand logos in your button list. Click the checkbox next to each card brand that you want to display in your checkout UI. Click the drag icon ( ) to rearrange the order of the card brands.
7. Click **Save and publish** to save your payment options configuration settings.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

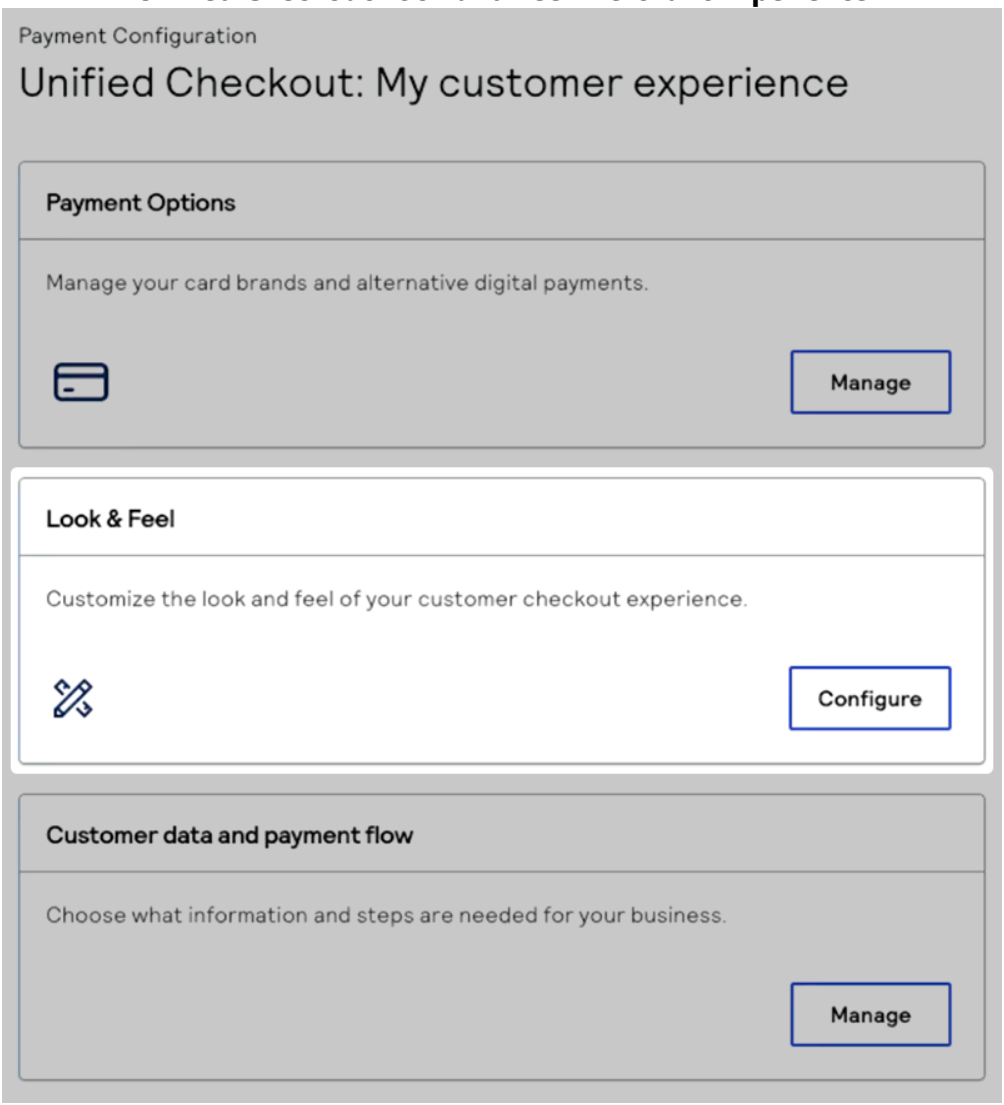
Customize the Unified Checkout Look and Feel

Follow these steps to customize the appearance of Unified Checkout in the Business Center:

1. Log in to the Business Center:
   Test URL: `https://businesscentertest.cybersource.com/ebc2`
   Production URL: `https://businesscenter.cybersource.com`
   If you are unable to access this page, contact your sales representative.

2. In the Business Center, go to the left navigation panel and choose **Payment Configuration** > **Unified Checkout**. The Unified Checkout customer experience page appears:

Unified Checkout Look and Feel Merchant Experience



3. In the Look & Feel section, click **Configure**. The Look and feel page appears.

4. If you want to use AI to determine the look and feel, under AI brand studio, click **Browse** the upload a screenshot of your website. These components are updated using the colors from the screenshot you provide:

- Button list background color
- Card checkout outline and text colors
- Header and checkout font background colors

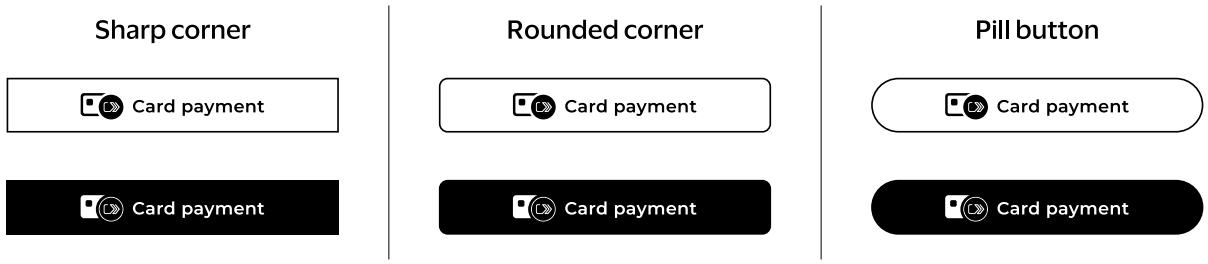
Proceed to the next steps to manually customize these and other components of the Unified Checkout appearance.

5. Under Button customizations, select the options for your button list. These customizations are available:

Universal Button Shape

Use the Button shape drop-down menu to select if you want a sharp corner, rounded corner, or pill button:

Unified Checkout Button Shapes



Button List

Use the color selector or enter a HEX code in the Button list background color text box to customize the background color of your button list. To review the button list preview, see Button List (on page 159).

If you uploaded a screenshot in the AI brand studio section, a color is entered in this space based on the colors present in your screenshot.

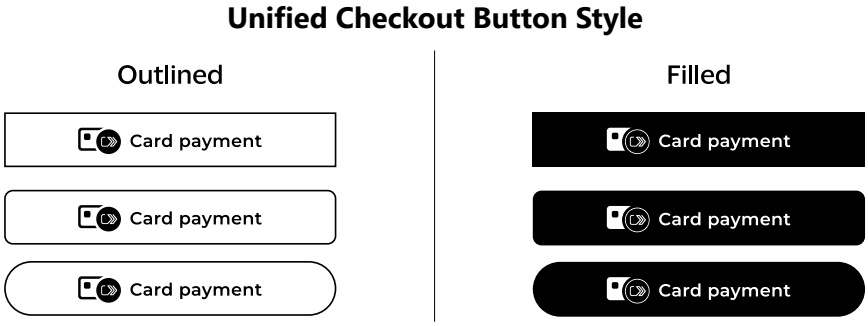
Card Checkout and Click to Pay Button

Use the card checkout button label drop-down menu to select the text that appears on the checkout button. These text options are available:

- Pay with card
- Card payment
- Checkout with card (default)
- Debit/Credit payment
- Donate with card
- Subscribe with card

Card Checkout Buttons

Use the button style drop-down menu to select if you want an outlined or filled checkout button. Select the button fill and text colors using the color selector or HEX code:



If you uploaded a screenshot in the AI brand studio section, a color is entered in this space based on the colors present in your screenshot.

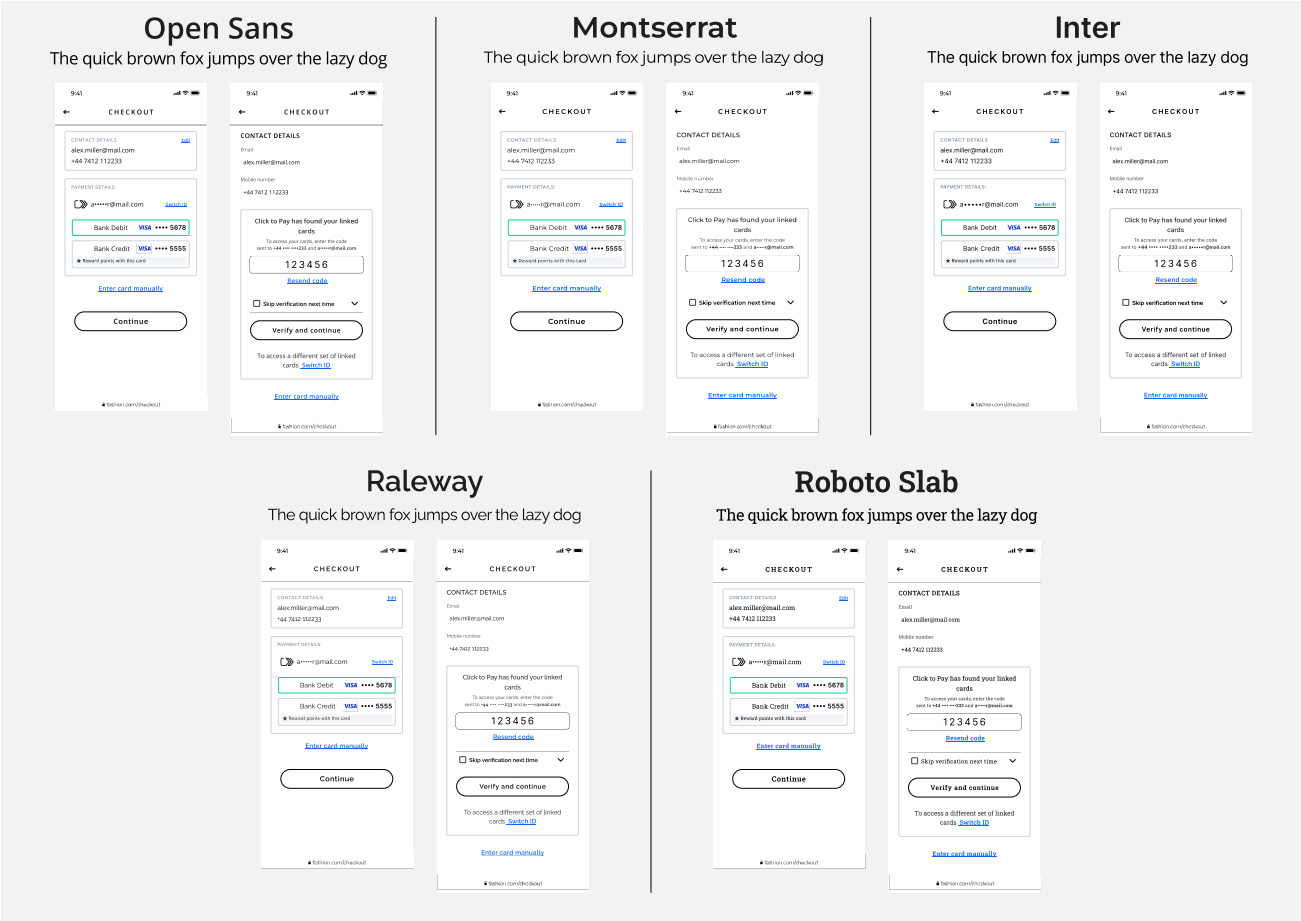
6. Under Checkout customizations, select the options for your checkout experience. These customizations are available:

Font

Use the font drop-down menu to select the font you want to use. These fonts are available:

- Inter
- Monserrat
- Open Sans
- Raleway
- Roboto Slab

Unified Checkout Fonts



Select the background color using the color selector or HEX code.

If you uploaded a screenshot in the AI brand studio section, a color is entered in this space based on the colors present in your screenshot.

Header Customization

Select the color you want to use in your header using the color selector or HEX code.

If you uploaded a screenshot in the AI brand studio section, a color is entered in this space based on the colors present in your screenshot.

7. Click **Save and publish** to save your look and feel customization. The Ready to publish popup window appears.

8. If you are done editing, click **Publish now** to publish your changes. If you need to make more changes, click **Keep editing** to return to the Look and feel page. To review your changes, click through the example screens. For more information about the UI previews, see Look and Feel UI Examples (on page 159).

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Look and Feel UI Examples

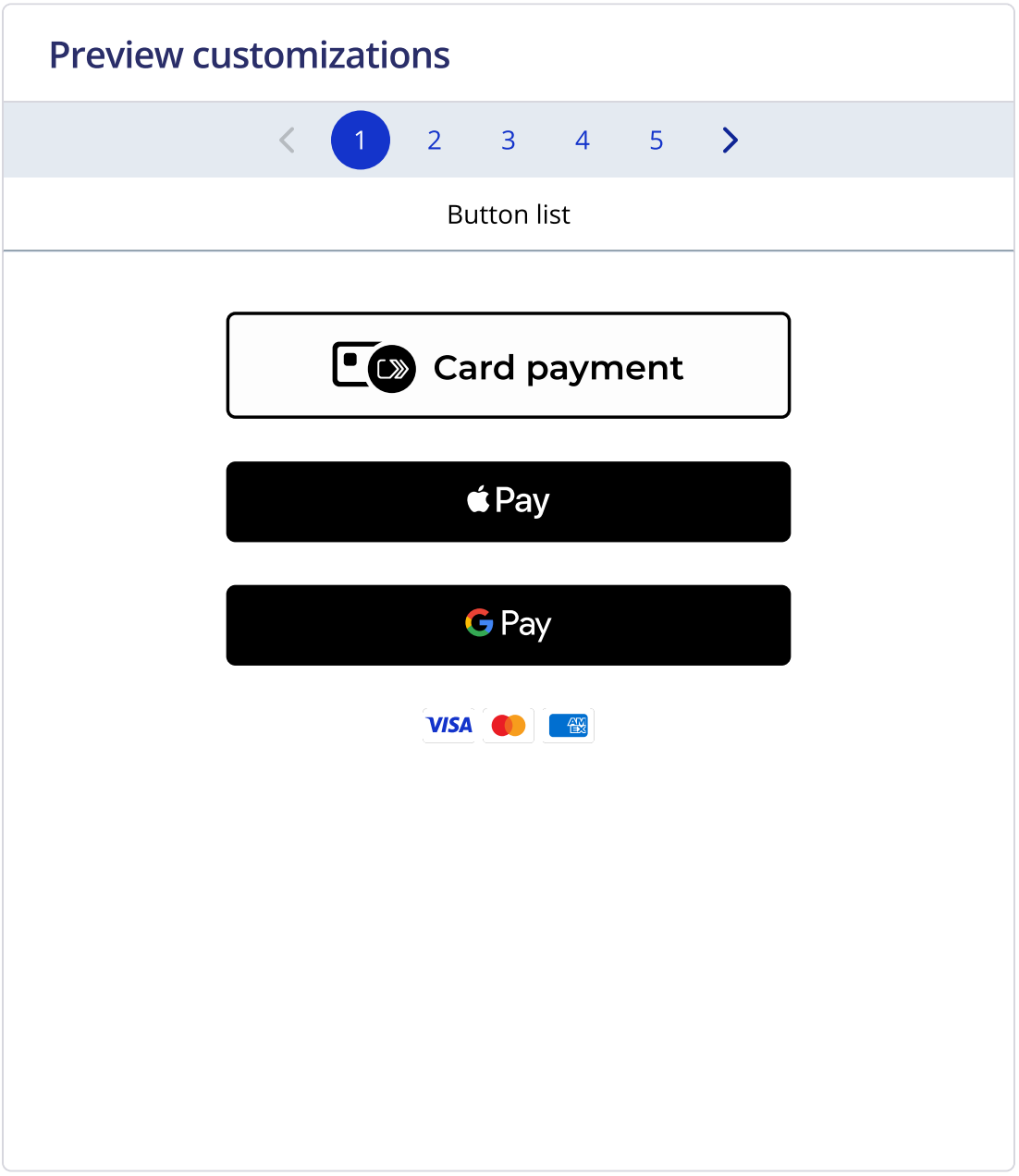
These examples show the different preview screens for each look and feel customization feature:

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Button List

This example shows the button list:

Screen 1: Button List



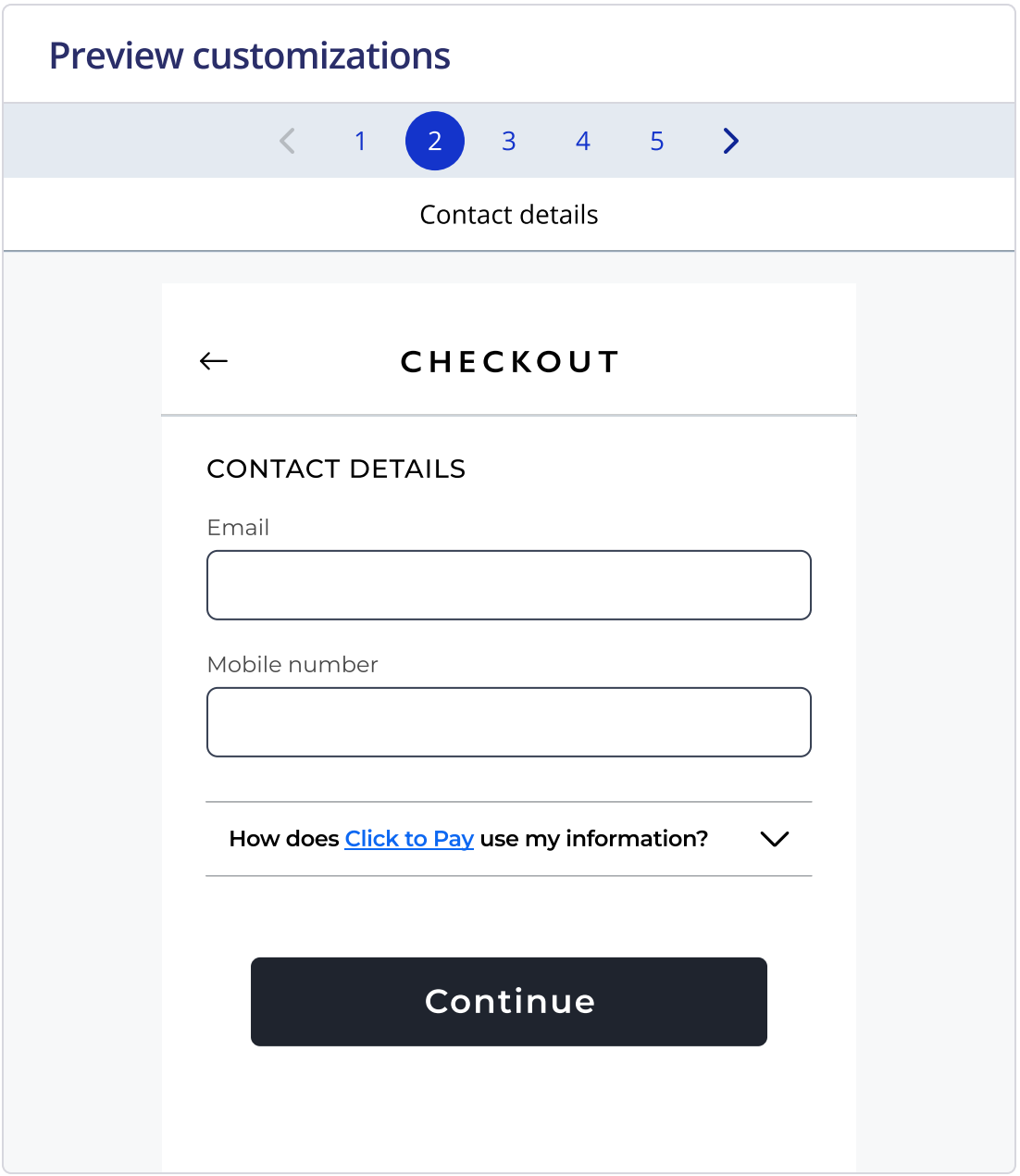
Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Contact Details

This example shows the contact details that appear during checkout:

Screen 2: Contact Details



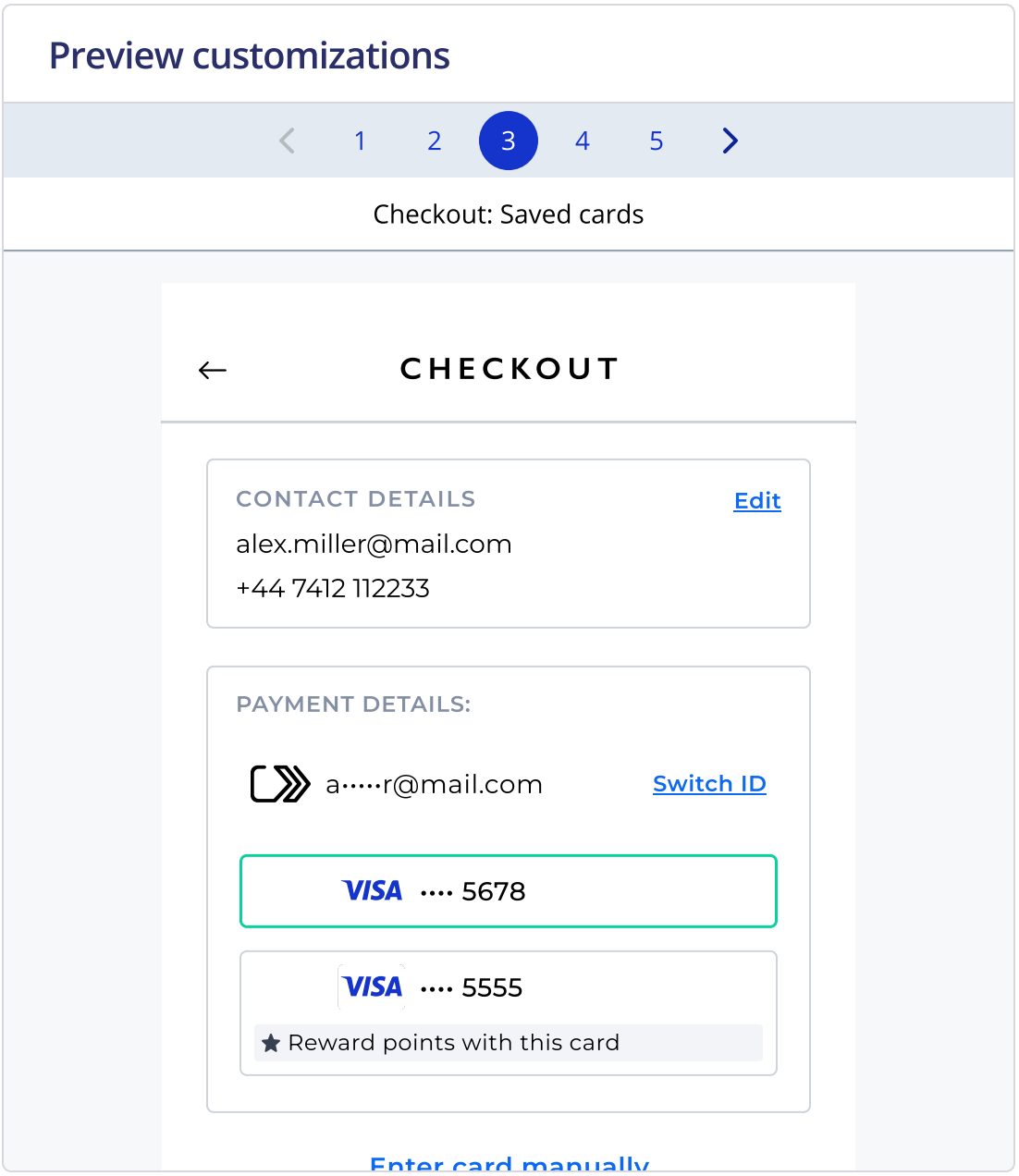
Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Saved Cards

This example shows the saved card payment details that appear during checkout:

Screen 3: Saved Cards



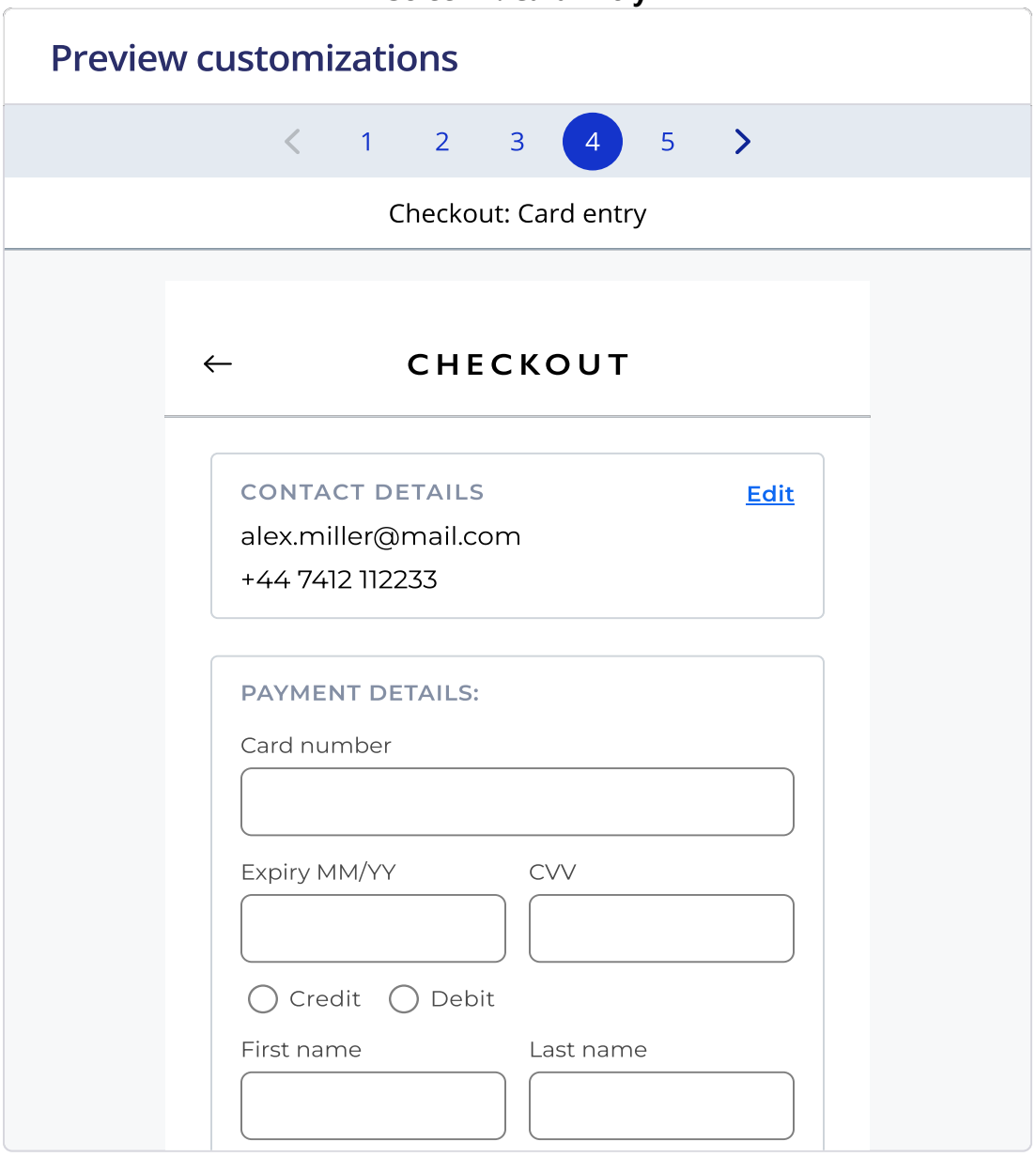
Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Card Entry

This example shows the card entry payment details that appear during checkout:

Screen 4: Card Entry



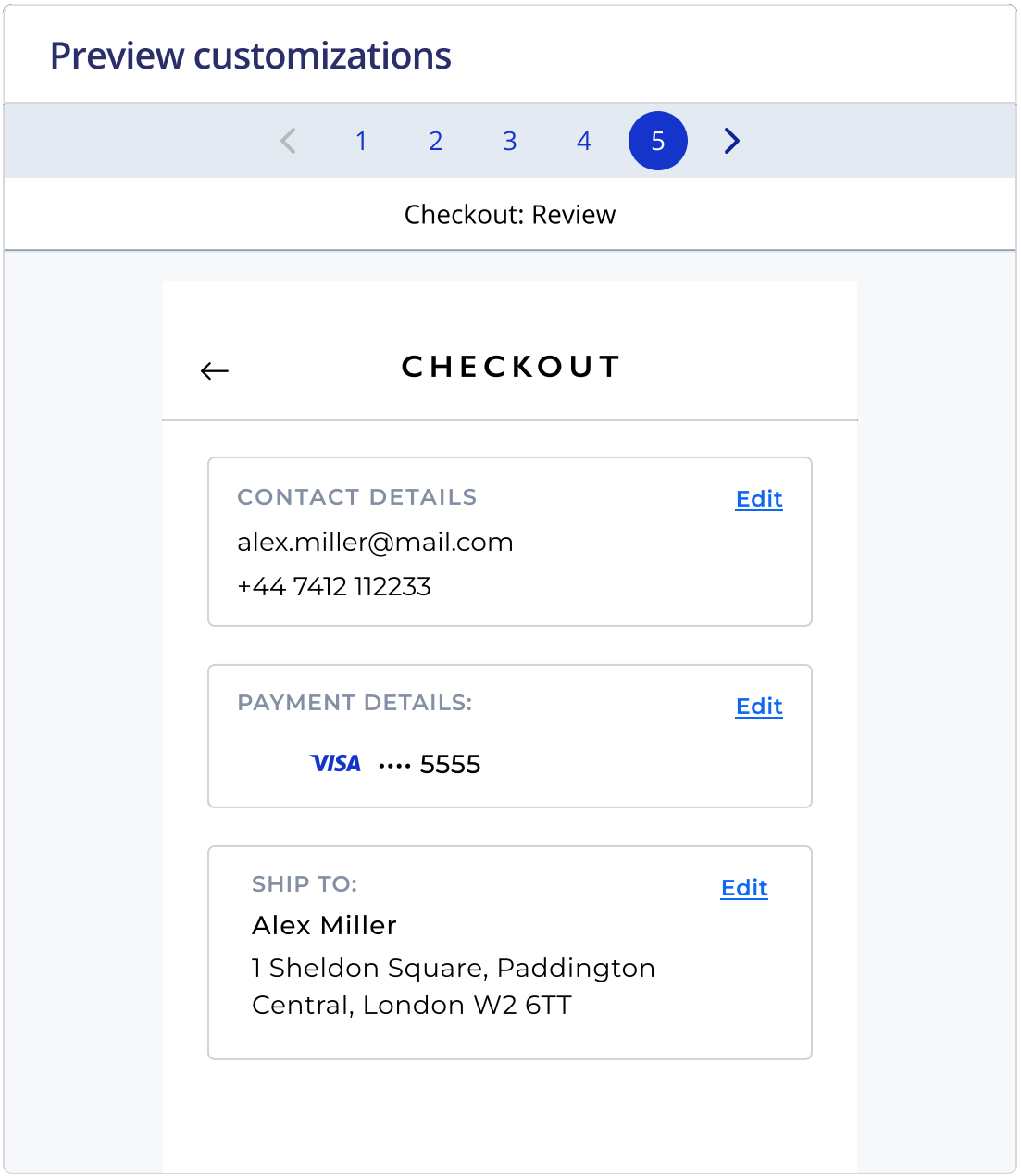
Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Review Details

This example shows the review contact details that appear at the end of checkout:

Screen 5: Review



Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

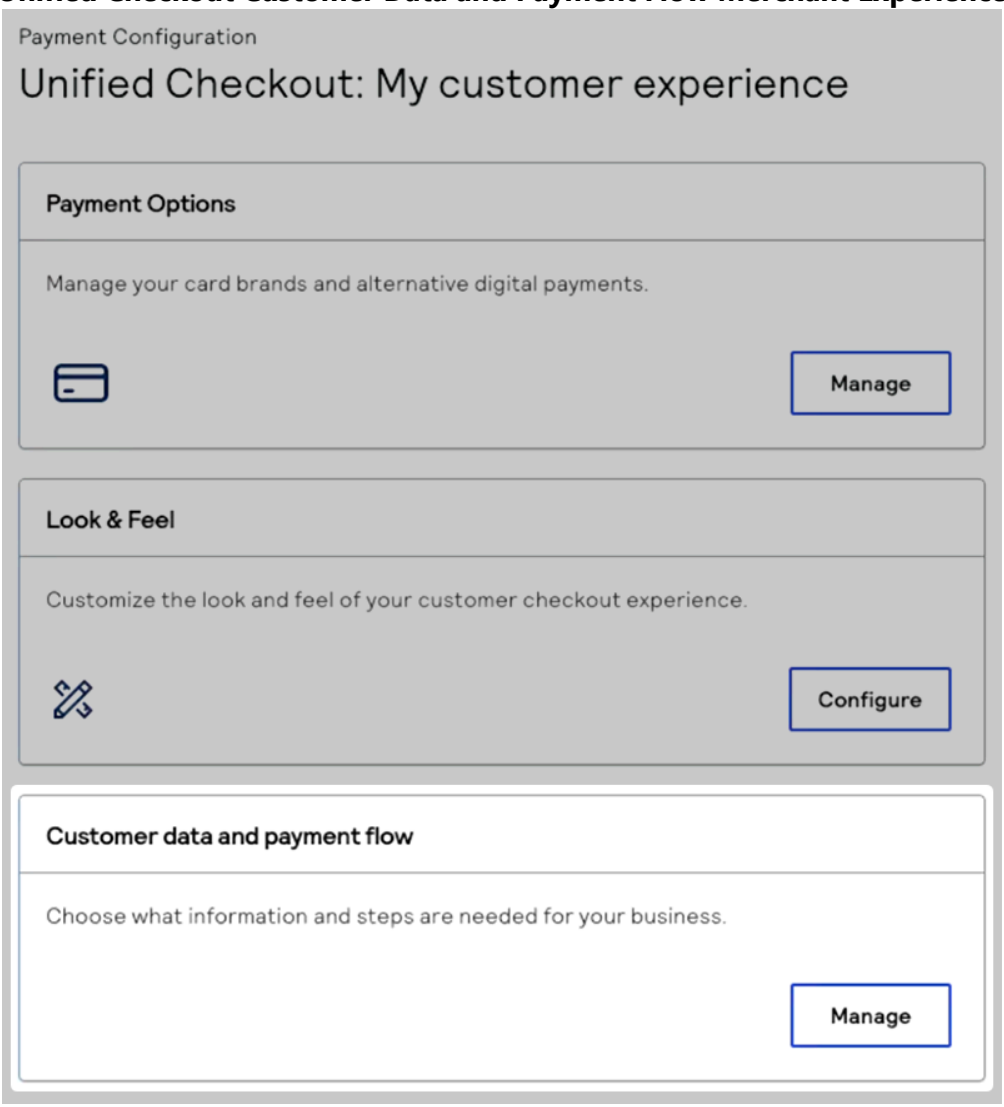
Configure Customer Data and Payment Flow

Use the Customer data and payment flow section of the Business Center to configure the information that you want to collect during checkout. Follow these steps to customize the appearance of Unified Checkout in the Business Center:

1. Log in to the Business Center:
   Test URL: `https://businesscentertest.cybersource.com/ebc2`
   Production URL: `https://businesscenter.cybersource.com`
   If you are unable to access this page, contact your sales representative.

2. In the Business Center, go to the left navigation panel and choose **Payment Configuration** > **Unified Checkout**. The Unified Checkout customer experience page appears:

Unified Checkout Customer Data and Payment Flow Merchant Experience



3. In the Customer data and payment flow section, click **Manage**. The Customer information and payment flow page appears.

4. Under Contact details, slide the toggle ( ) next to Email address and Mobile phone number to collect the customer email address and phone number during checkout.

5. Under Payment details, slide the toggle () next to Billing address to collect the cardholder billing address.

6. Under Payment details, in the Billing address drop-down menu, select the billing address information for payment verification:

    - **Full address**: Request the full cardholder billing address during checkout.

    - **Zip code only**: Request only the zip code of the cardholder billing address during checkout.

7. Under Payment details, slide the toggle () next to Shipping address to collect the cardholder shipping address.

8. Under Payment details, in the Shipping address drop-down menu, select the countries that you ship to.

9. Under Checkout review step, slide the toggle () to display a review page for the customer to confirm the payment and shipping address.

10. Under payment processing and & service orchestration, slide the toggle () in the Payment processing section to control how to process transactions:
    You can turn payment processing on or off:

    - **Payment Processing ON**: Unified Checkout handles the complete payment for you automatically. When payment processing is on, you can select how transactions are processed:

        - **Preferred auth**: Choose this option to authorize the payment when possible.

        - **Sale**: Choose this option to capture the funds immediately.

    - **Payment Processing OFF**: You must complete the payment independently using VISA or another selected gateway.

    For information about enabling or disabling payment processing in Unified Checkout, see Process Payments with Unified Checkout (on page 174).

11. Under save customer information in payment processing and & service orchestration, slide the toggle to the right to prompt the user to save their payment information for future use.

    Saving customer payment details can significantly improve the checkout experience for returning customers. There are two approaches that you can use:

    - **Ask for consent to save payment information for future use**: This option displays a checkbox during checkout that asks customers if they want their payment details saved. You can enable this option regardless of if payment processing is enabled at any time.

    - **Store payment details securely in your TMS vault**: This option saves payment information using secure encryption and links directly to your TMS vault. This enables faster checkout for returning customers by using tokens rather than raw card data.

Important: If you already have cardholder consent (for example, if it was obtained during account creation or through another verified process) or consent is not required for your flow, you should not display or request consent again in the UI.

12. Under Fraud detection in payment processing and & service orchestration, use the drop-down menu to turn fraud detection with Decision Manager or Fraud Management Essentials **On** or **Skip**.
13. Under Localized payment settings, slide the toggle ( ) to enable these features:
    - **Combo cards**: Enable cardholders to decide how to process their payments.
    - **Brazil tax ID**: Collect a customer's CPF or CNPJ at checkout.

Important: Combo cards and tax IDs are only available in Brazil.

14. Click **Save and publish** to save your customer payment settings.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Capture Context Fields in the Business Center

There are some components of Unified Checkout that are available in the API, the Business Center or both. This section describes which components of the Sessions API capture context object can be configured using the Business Center. For a complete list of fields that are available, see the API Reference in the Cybersource Developer Center.

API Parameters

Parameter	Business Center	API	Notes
allowedCardNetworks	Yes	Override	Optional. This is the override priority order:

API Parameters (continued)

Parameter	Business Center	API	Notes
			1. API 2. Merchant experience profile 3. Portfolio profile
allowedPaymentTypes	Yes	Override	Optional. This is the priority order: 1. API (payment types that are not enabled are removed) 2. Merchant experience profile (payment types that are not enabled are removed) 3. Portfolio profile (all payment types are enabled at the profile level) **Important:** `SRCVISA`, `SRCMASTERCARD`, and `SRCAMEX` are not supported. You must use `CLICKTOPAY`.
paymentConfigurations	Partial	Yes	Allows per-transaction payment type configuration overrides. This is available only in **clientVersion** `1.0` or later.
paymentConfigurations.CLICKTOPAY	Yes	Override	Default values can be set in the Business Center.
paymentConfigurations.CLICKTOPAY. autoCheckEnrollment	Yes	Override	This payment configuration is part of the merchant experience.

API Parameters (continued)

Parameter	Business Center	API	Notes
paymentConfigurations.GOOGLE PAY	Yes	Override	Default values can be set in the Business Center.
paymentConfigurations.GOOGLEPAY. allowedAuthMethods	Yes	Override	This payment configuration is part of the merchant experience.

Capture Mandate

Capture Mandate Parameters

Parameter	Business Center	API	Default Value
showConfirmationStep	Yes	Override	
billingType	Yes	Override	
requestEmail	Yes	Override	
requestPhone	Yes	Override	
requestShipping	Yes	Override	
shipToCountries	Yes	Override	
showAcceptedNetworkIcons	Yes	Override	
comboCard	Yes	Override	
requestSaveCredentials	Yes	Override	
cpf	Yes	Override	
cpf.required	Yes	Override	

Complete Mandate

Complete Mandate Parameters

Parameter	Business Center	API	Default Value
type	Yes	Override	`SALE`
decisionManager	Yes	Override	`true`
consumerAuthentication	Yes	Override	`NONE`
tms	Yes	Optional	

Complete Mandate Parameters (continued)

Parameter	Business Center	API	Default Value
tms.tokenCreate	Yes	Override	`true`

Appearance Variables

Top-Level Appearance Parameters and Variables

Parameter / Variable	Business Center	API	Default Value	Example
buttonType	Yes	Override	`CHECKOUT`	"CHECKOUT"
variables	Partial	Override		
backgroundColor	Yes	Override		"#FFFFFF"
textColor	Yes	Override		"#000000"
headerBackground	Yes	Override		"#1A237E"
headerForeground	Yes	Override		"#FFFFFF"
buttonBackground	Yes	Override		"#E0E0E0"
buttonForeground	Yes	Override		"#333333"
buttonBorderRadius	Yes	Override		"4px"
fontFamily	Yes	Override		"Roboto Slab, serif"
paymentSelectionBackground	Yes	Override		"#F5F5F5"

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Manage Permissions

Portfolio administrators can set permissions for new or existing Business Center user roles for Unified Checkout. Administrators retain full read and write permissions. They enable you to regulate access to specific pages and specify who can access, view, or amend digital products within Unified Checkout.

Portfolio administrators must apply the appropriate user role permission for any existing or newly created Business Center user roles for Unified Checkout. For information on managing permissions as a portfolio administrator, see Managing Permissions as a Portfolio Administrator (on page 173).

If you are a transacting merchant, you might find that your permissions are restricted. If your permissions are restricted, a message appears indicating that you do not have access, or buttons might appear gray. To make changes to your digital products within Unified Checkout that have restricted permissions, contact your portfolio administrator's customer support representative. For more information, see Managing Permissions as a Direct Merchant (on page 172).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Managing Permissions as a Direct Merchant

Follow these steps to configure and manage user permissions in the Business Center for Unified Checkout as a direct merchant:

1. On the left navigation panel, navigate to **Account Management**.
2. Click **Roles** to display a list of your user roles.
3. Click the pencil icon next to the user role that you want to update.
4. Click **Payment Configuration Permission**.
5. Select the relevant permission for the specific user role you are editing. You can select from these Unified Checkout permissions:
    - Unified Checkout View
    - Unified Checkout Manage

> **Important:**
>
> If you are a transacting merchant without view permissions, Unified Checkout will still appear on the navigation bar, however, a *no access* message appears when you access Unified Checkout.
>
> If you are a transacting merchant with view permissions but not management permissions, you can access the Unified Checkout screens and view the different payment methods configurations, however, you cannot edit or enroll new products.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Managing Permissions as a Portfolio Administrator

Follow these steps to configure and manage user permissions in the Business Center for Unified Checkout as a portfolio administrator:

1. On the left navigation panel, navigate to **Account Management**.
2. Click **Roles** to see a list of your user roles.
3. Click the pencil icon next to the user role that you want to update.
4. Click **Payment Configuration Permission**.
5. Select the relevant permission for the specific user role you are editing. You can choose from these Unified Checkout permissions:
    - Unified Checkout View
    - Unified Checkout Manage
    - Unified Checkout Portfolio View (available for portfolio users only)
    - Unified Checkout Portfolio Manage (available for portfolio users only)

> **Important:**
>
> If all permissions are left unselected, the user has restricted permission. A *no access* message appears when the user tries to access the Unified Checkout digital product enablement pages. The user is advised to contact a customer representative.
>
> If a portfolio user has view permissions and does not have a management role, they can access the Unified Checkout pages, but they cannot modify toggles for different digital payments.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Process Payments with Unified Checkout

Payment processing is a payment completion option in your merchant configuration. Your configuration determines if your checkout system automatically handles and finalizes customer payments. When payment processing is enabled, Unified Checkout handles the complete payment for you automatically. When payment processing not enabled, you complete payments independently using your selected gateway.

Payment Processing Enabled

Cybersource recommends that you enable payment processing if you meet these requirements:

- Your integration is configured with the payment completion step. For more information about **completeMandate()** see Complete Mandate (on page 188).
- You do not have a designated technical team to process payments.
- Your integration includes any of these features:
    - Fraud checks with Decision Manager or Fraud Management Essentials
    - 3-D Secure/ Payer Authentication
    - Stored customer credentials with the Token Management Service (TMS)
    - Multiple payment methods

When payment processing is enabled, you can choose how payments should be handled:

- **Preferred Authorization**: The payment is authorized first and captured at a later time. This method works well for businesses that ship products to their customers. For example, you authorize the payment when the customer places and order and capture the payment when you ship it.
- **Sale**: The payment is captured immediately. This method works well for service businesses, digital products, and immediate purchases.

Important: Not all payment methods are compatible with all processing types. Different payment methods work with different payment processing types. You must configure your payment processing settings to be compatible with your integration and review which payment methods are compatible. For information about payment methods and their processing compatibility, see Payment Methods (on page 32).

Payment Processing Disabled

Cybersource recommends that you disable payment processing if you meet these requirements:

- You want to process payments on our platform using your own API requests instead of relying on the automatic payment completion step.
- You want full control over your checkout and payment orchestration flow.
- You only use Unified Checkout to collect encrypted payment information and you handle the completion phase through our platform APIs or your custom back-end. The completion phase includes authorization, sale, fraud checks, 3-D Secure and card storage.

When disable payment processing, the checkout system will collect payment information from your customers but you will need to process the actual payment on your end. When payment processing is disabled, you must consider the following:

- Customer payments are not completed automatically.
- Fraud checks and 3-D Secure using automatic orchestration are disabled.
- Saved credentials and token management using automatic processing are disabled.
- You must handle payment completion yourself.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Webhooks Support

Unified Checkout supports webhooks. You can use webhooks to obtain the complete response from the **completeMandate** call. To receive a webhook notification, you must first subscribe to the webhook.

Prerequisite

Webhook payloads are encrypted. In order to receive a Unified Checkout webhook notification, you must enabled message-level encryption (MLE). For information about enabling MLE, see Enable Message-Level Encryption in the *Getting Started with REST Developer Guide*.

Integration

Follow these steps to set up your system to support the Webhooks REST API. Some of these steps are dependent on your system's security policy.

1. Set up a server with a URL to receive webhook notifications.

2. Configure your server security to receive webhooks notifications. For more information, see *Set Up Your Security* in the Webhooks Implementation Guide.

3. Create a REST API security key that is compliant with your security policy. Security keys are used to authenticate the requests you send to Cybersource. You must create separate keys for the testing and production environments. For more information, see *Create REST API Keys* in the Webhooks Implementation Guide.

4. Request a digital signature key from Cybersource. For more information, see *Create a Digital Signature Key* in the Webhooks Implementation Guide.

5. Implement message-level encryption for those events. See *Message-Level Encryption* in the Webhooks Implementation Guide.

6. If your system uses the *OAuth* or *OAuth with JWT* security policy, you must provide your OAuth credentials to Cybersource. OAuth is not required and Mutual Trust is the default. If you are not using OAuth, skip this step. For more information, see *(Optional) Provide Your OAuth Credentials* in the Webhooks Implementation Guide.

7. Request a list of the products for which your organization is enabled to receive webhook notifications. For more information, see *Retrieve a Lit of Products and Events* in the Webhooks Implementation Guide.

   > **Important:** You must enable message-level encryption (MLE) in order to access the webhook response. See *Message-Level Encryption* in the Webhooks Implementation Guide.

8. Create your webhook subscription event notifications. For more information, see *Create a Webhook Subscription* in the Webhooks Implementation Guide.

Optional Set Up Tasks

You can complete these optional tasks after creating a webhook subscription.

- Include a health check URL to enable Cybersource to monitor your server's status for reliability. For more information, see *Webhook Health Check URL and Automatic Revalidation* in the Webhooks Implementation Guide.
- Customize the retry policy for unresponsive webhook and health check URLs. For more information, see *Configure the Retry Policy* in the Webhooks Implementation Guide.
- Validate your digital signature. For more information, see *Validating a Notification with the Digital Signature Key* in the Webhooks Implementation Guide.

Webhook Events

Unified Checkout Webhook Events

Product ID	Event Types	Description
`unifiedCheckout`	`uc.orders.transactionresults`	Full payload response from the payment service call made by Unified Checkout

Set Up Webhook Subscriptions

For information on setting up a webhook for the `unifiedCheckout` product, see the How to Set Up Webhook Subscriptions section of the *Webhooks Implementation Guide*.

Example Webhook Payload

Example: Webhooks Request for Unified Checkout Events

```
{
  "organizationId": "your_merchant_id",
  "webhookId": "2d55e648-d96c-d727-e063-3cb8d30a938e",
  "productId": "unifiedCheckout",
  "eventType": "uc.orders.transactionresults",
  "eventDate": "2025-03-27T08:44:55",
  "payload": {
    "id": "743518889935640500 3091",
    "status": "AUTHORIZED",
    "outcome": "AUTHORIZED",
    "details": {
      "processorInformation": {
        "transactionId": "201601180815391001 1808153AUTH"
      },
      "paymentInformation": {
        "card": {
          "type": "001"
        }
      },
      "riskInformation": {
        "score": {
          "result": "42"
        }
      }
    }
  }
}
```


The `payload` field object contains the same fields as the response from a direct payment authorization request. Use the `id` field for capture requests or to look up a transaction.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Sessions API

Use the Sessions API to generate a capture context. The capture context contains all of the merchant-specific parameters that tell the front-end JavaScript library what to do within your payment experience.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Capture Context

The capture context is a signed JSON Web Token (JWT) containing this information:

- Merchant-specific parameters that dictate the customer payment experience for the current payment transaction.
- A one-time public key that secures the information flow during the current payment transaction.

There are some features of Unified Checkout that are available in the API, the Business Center or both. For information about how to configure Unified Checkout using the Business Center, see Configure the Unified Checkout Merchant Experience (on page 147). For information about which fields are available using the API or the Business Center, and when one is overridden by the other, see Capture Context Fields in the Business Center (on page 167). For a full capture context with all possible fields, see Example: Unified Checkout Complete Capture Context (on page 202).

Use these required fields to request the capture context:

allowedPaymentTypes

clientVersion

country

locale

data.orderInformation.amountDetails.currency

data.orderInformation.amountDetails.totalAmount

Include this field for values greater than or equal to `0.00`.

targetOrigins

The URL in this field value must contain `https`.

This example shows the minimum fields that must be included in the capture context:

```
{
  "targetOrigins": [
    "http://localhost:8080"
  ],
  "country": "US"
  "locale": "en_US"
  "data": {
    "orderInformation": {
      "amountDetails": {
        "totalAmount": "21.00",
        "currency": "USD"
      }
    }
  }
}
```

For information on JSON Web Tokens, see JSON Web Tokens (on page 266).

Important:

Cybersource recommends that you dynamically parse the response for the fields that you are looking for when you integrate with Cybersource APIs. Cybersource may add additional fields in the future.

You must ensure that your integration can handle new fields that are returned in the response. Even though the underlying data structures do not change, you must also ensure that your integration can handle changes to the order in which the data is returned. Cybersource uses semantic versioning practices, which enables you to retain backwards compatibility as new fields are introduced in minor version updates.

Related Information

- API field reference guide for the REST API

Endpoint

Production: `POST https://api.cybersource.com/uc/v1/sessions`

Test: `POST https://apitest.cybersource.com/uc/v1/sessions`

Production in Saudi Arabia: `POST https://api.sa.cybersource.com/uc/v1/sessions`

Test in Saudi Arabia: `POST https://apitest.sa.cybersource.com/uc/v1/sessions`

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Allowed Card Networks

Use the **allowedCardNetworks** field to define the card types.

These card networks are available for card entry:

- American Express
- Cartes Bancaires
- Carnet
- China UnionPay
- Diners Club
- Discover
- EFTPOS
- ELO
- Jaywan
- JCB
- JCrew

- KCP
- mada
- Maestro
- Mastercard
- Meeza
- PayPak
- UATP
- Visa

To support dual-branded or co-badged cards, you must list your supported card type values for the **allowedCardNetworks** field based on your preference for processing card numbers. For example, if a card is dual-branded as Visa and Cartes Bancaires, and Cartes Bancaires is listed first, the card type is set to Cartes Bancaires after the card number is entered in your Unified Checkout card collection form. For information on dual-branded or co-badged cards, see Dual-Branded Cards (on page 218).

Important:

Some card types, such as KCP and UATP, do not have security codes (CVV or CVN). If you include only card types that do not have security codes in the **allowedCardNetworks** field, Unified Checkout does not display the security code field in the UI.

If you include card types that do not have security codes and cards types that do have security codes in the **allowedCardNetworks** field, Unified Checkout displays the security code field in the UI. The field is disabled in the UI when the cardholder enters a card number for a card type with no security code

Domestic Scheme Rules

For markets where domestic routing, network preference, or co-badging regulations apply, you are responsible for ensuring that domestic payment schemes are prioritized appropriately within the **allowedCardNetworks** configuration.

In these scenarios, domestic schemes must be listed ahead of international networks such as Visa or Mastercard where required by local regulations or scheme rules.

If network preference varies by transaction, the order of networks should be dynamically configured when you create the capture context request using the Sessions API.

You are responsible for reviewing and complying with all applicable local regulatory, scheme, and acquirer requirements relating to domestic routing and network selection.

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Target Origins

The target origin is defined by the scheme (protocol), hostname (domain), and port number (if used).

You must use the https:// protocol. Sub domains must also be included in the target origin.

Any valid top-level domains, such as .com, .co.uk, and .gov.br, are supported. Wildcards are not supported.

For example, if you are launching Unified Checkout on example.com, the target origin could be any of the following:

- https://example.com
- https://subdomain.example.com
- https://example.com:8080

When you use Unified Checkout in an iframe, you must include the domain for the URL that loads the iframe and the iframe URL in the **targetOrigins** field.

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Allowed Payment Types

You can specify the type of Unified Checkout digital payment methods that you want to accept in the capture context.

Use the **allowedPaymentTypes** field to define the payment type:

- `AFTERPAY`
- `APPLEPAY`
- `BANCONTACT`
- `CHECK`
- `CLICKTOPAY`
- `DRAGONPAY`
- `GOOGLEPAY`
- `IDEAL`
- `KONBINI`
- `MULTIBANCO`
- `MYBANK`
- `P24`
- `PANENTRY`
- `PAZE`
- `TINKPAYBYBANK`



Important: Click to Pay accepts American Express, Mastercard, and Visa for saved cards. Visa and Mastercard tokenize payment credentials using network tokenization for all Click to Pay requests. Click to Pay uses Click to Pay Token Requester IDs (TRIDs) rather than your existing TRIDs to generate network tokens.

For more information on enabling and managing these digital payment methods, see these topics:

- Enrolling in Apple Pay (on page 45)
- Enabling Click to Pay in the Business Center (on page 53)
- Enrolling in Google Pay (on page 48)
- Alternative Payment Methods (on page 71)

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Client Version

This field is used to specify the Unified CheckoutAPI version that your integration should use.

Cybersource recommends that you do no include this field in your `uc/v1/sessions` API request. When you do not include this field, Unified Checkout automatically uses the latest available version. This ensures access to the most recent enhancements and updates without requiring integration changes.

When you include this field, the value must be provided in `MAJOR.MINOR` format (for example, `1.1` or `1.2`). For information about semantic versioning, see Versioning (on page 132).

Important: This field cannot be configured through the merchant experience screens in the Business Center.

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Auto-check Enrollment

You can have the Click to Pay box pre-checked when a user is manually entering their card details and Click to Pay is enabled. The customer can uncheck the box if necessary, which means the request is processed as a one-time manual PAN transaction. This is available when you set the **billingType** field to `PARTIAL` or `FULL` in the capture context. This ensures that the customer's billing country can be validated in the UI.

Click to Pay enrollment pre-check is available in these countries:

- Argentina
- Brazil
- Chile
- Colombia
- Kuwait
- Mexico
- Peru
- Qatar
- Saudi Arabia
- South Africa
- Ukraine
- United Arab Emirates

```
"paymentConfigurations": {
    "CLICKTOPAY": {
      "autoCheckEnrollment": true
    }
  }
```

Related information

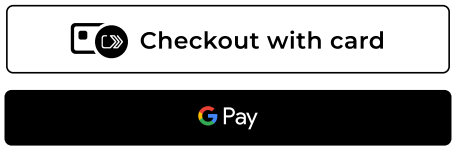
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Button Type

When Unified Checkout loads, the payment buttons displayed are based on what you include in the **allowedPaymentTypes** object in the capture context. Unified Checkout enables you to customize the text on the payment buttons. You can do this by setting the **buttonType** field object in the capture context to one of these values:

- `ADD_CARD`
- `CARD_PAYMENT`
- `CHECKOUT_AND_CONTINUE`
- `DEBIT_CREDIT`
- `DONATE`
- `PAY`
- `PAY_WITH_CARD`
- `SUBSCRIBE_WITH_CARD`

If you do not include the **buttonType** field in your request, the payment button text defaults to **Checkout with card**. For example:



Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Customize Button Text

Use the **buttonType** field to customize the text on payment buttons:

Button Text Options

buttonType Value	Button Display Text
`ADD_CARD`	Add card
`CARD_PAYMENT`	Card payment
`CHECKOUT_AND_CONTINUE`	Checkout and continue

Button Text Options (continued)

buttonType Value	Button Display Text
`DEBIT_CREDIT`	Debit or credit
`DONATE`	Donate
`PAY`	Pay
`PAY_WITH_CARD`	Pay with card
`SUBSCRIBE_WITH_CARD`	Subscribe with card

When you do not include this field in your request, the default button text is “Checkout with card.”

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Complete Mandate

The complete mandate feature provides service orchestration within Unified Checkout and simplifies your integration. Service orchestration enables Unified Checkout to orchestrate services on your behalf. The complete mandate feature provides instructions to the **unifiedPayment.complete()** method in the JavaScript. You must include both the **unifiedPayment.complete()** object in the Javascript and the **completeMandate** field object in your capture context to enable Unified Checkout to initiate services on your behalf from the browser.

Important: If you are updating an existing Unified Checkout configuration to use the complete mandate, you must update your JavaScript to include the **unifedPayment.complete()** function.

Important: When the **billingType** field is set to `NONE` you must include the required fields within the capture context request to ensure that the required fields are included for payment processing. For information about the fields that are required for payment services, see the Payments Developer Guide.

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

completeMandate.type

This field is required to run the complete mandate and is used to indicate how a payment should be processed.

Possible values:

- `AUTH`: Authorize the payment and capture the funds at a later date.
- `CAPTURE`: Perform a sale. A sale is a combined authorization and capture in a single request.
- `PREFER_AUTH`: Perform an authorization if possible. If a payment method requires the funds to be captured immediately, then Unified Checkout captures the payment.

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

completeMandate.decisionManager

This field determines whether Decision Manager is run. Set this field to `true` and include **completeMandate.type** in your request to run Decision Manager and device fingerprinting services. When Decision Manager runs, it uses the associated Decision Manager configuration based on the merchant ID that is included in the request.

When this field is set to `false` or is not included in the request, Decision Manager and device fingerprinting services do not run.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

completeMandate.consumerAuthentication

This field determines how a consumer is authenticated. You can authenticate a consumer using 3-D Secure.

You can set this field to one of these values:

- `NONE`: Consumer authentication does not run.
- `3DS`: 3-D Secure authentication through Payer Authentication runs

To initiate a passkey request for Unified Checkout, you must include these additional fields in your request:

- **completeMandate.consumerAuthentication** set to `PASSKEY`
- **data.merchantInformation.merchantDescriptor.name**
- **data.acquirerInformation.country**

When you use Unified Checkout with Payer Authentication, device data is collected through Payer Authentication setup and Unified Checkout completes all calls that are associated with Payer Authentication.

For information about challenge codes, see **consumerAuthenticationInformation.challengeCode** in the REST API Field Reference. Unified Checkout supports 3-D Secure data only when your payment processor supports it. For information see Visa Data Only in the *Payer Authentication Developer Guide*.

To test Payer Authentication, you must use Payer Authentication test cards. See Test Cases for 3-D Secure 2.x in the *Payer Authentication Developer Guide*.

Consumer authentication is available for these card types:

- American Express
- Cartes Bancaires
- China UnionPay
- Diners Club
- Discover
- EFTPOS
- ELO
- Jaywan
- JCB
- mada
- Maestro
- Mastercard
- Visa

Consumer authentication runs for these payment methods when they are supported in your Unified Checkout configuration:

- `PANENTRY`
- `CLICKTOPAY` when the transaction is not authenticated with Click to Pay.
- `GOOGLEPAY` when the transaction is not authenticated with Google Pay.

Unified Checkout does not attempt to authenticate for Click to Pay and Google Pay if the transaction has already been authenticated when it is received by Unified Checkout. For information about testing authentication, see Test Authentication (on page 229).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

completeMandate.tms

completeMandate.tms.tokenCreate: This field determines if a TMS token is created for the customer's selected payment method. When this field is set to `true`, a token is created. When this field is set to `false` or not included in the request, a token is not created.

Important: To make a new payment instrument or instrument identifier under an existing customer during the complete mandate, you must meet these requirements:

- You must include the customer token ID in the **paymentConfigurations** field object.
- `TMS_TOKEN` must be included in the **allowedPaymentTypes** field object.
- **tokenCreate** must be set to `true` and `paymentInstrument` and `instrumentIdentifier` must be included as values in the **tms.tokenTypes** field array. For example:

```
"tms": {
      "tokenCreate": true,
      "tokenTypes": [
        "paymentInstrument",
        "instrumentIdentifier",
      ]
    }
```

When you meet these requirements, a new payment instrument or instrument identifier is created under the specified customer token.

completeMandate.tms.tokenTypes: This is an optional field that you can use to indicate the token type for the token that is created. When this field is not included in the request, a token is created based on your TMS vault configuration. You can set this field to these values:

- `customer`
- `instrumentIdentifier`
- `paymentInstrument`
- `shippingAddress`

If you want Unified Checkout to capture the cardholder's consent to save the card before a request to create a token is completed, then you must set **captureMandate.requestSaveCredentials** to `true`. When this field is set to `true`, Unified Checkout presents a **Save card for future payments** checkbox within the UI and enables the cardholder to give consent. Do not include **captureMandate.requestSaveCredentials** in your request if you have already gained cardholder consent to create a TMS token or do not require consent.

This table indicates if a token is created given the requested payment method:

Payment Method	Capture Context	Result
PAN Entry and Click to Pay	**completeMandate.tms.token Create** = `true`	TMS token is created at the token level(s) specified in the request or based on the default for the token vault.
	completeMandate.tms.token Create = `true` and **captureMandate.requestSave Credentials** = `true`	Cardholder can check **Save Payment Information** in Unified Checkout. The request to create a token is made when the cardholder checks this field in the UI. When it is not checked, ni token is created.
Apple Pay, Google Pay, and Paze	**completeMandate.tms.token Create** = `true`	TMS token is created at the token level(s) specified in the request or based on the default for the token vault.
	completeMandate.tms.token Create = `true` and **captureMandate.requestSave Credentials** = `true`	Unified Checkout cannot obtain consent to create a token and no token is created when the customer completes the payment.
Echeck	**completeMandate.tms.token Create** = `true`	TMS token is created at the token level(s) specified in the request or based on the default for the token vault.
	completeMandate.tms.token Create = `true` and **captureMandate.requestSave Credentials** = `true`	Unified Checkout cannot obtain consent to create a token and no token is created when the customer completes the payment.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Capture Mandate

The capture mandate enables you to define which fields are captured within Unified Checkout. You must include the fields and set the values in the capture context based on the information that you want Unified Checkout to collect. This enables the cardholder to review and edit their details where the UI includes these fields. When the UI is used to capture cardholder information, all captured information is available within the Payment Details API response. When you want the cardholder to review existing address data, you can include the known customer data in the capture context and this information is pre-filled in the Unified Checkout UI. For information about the Payment Details API, see Payment Details API (on page 241).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

captureMandate.comboCard

A combo card is a single card in Brazil that functions as both a debit and a credit card. Unified Checkout enables the cardholder to choose whether to pay for a transaction using a debit or credit card. The cardholder can choose the card that they want to use when they enter their card details or when they choose a stored Visa card from their Click to Pay wallet during checkout. While in the card details section of the payment form, the cardholder is prompted for a debit or credit card. Credit is the default option.

To enable combo cards during checkout, you must include the **comboCard** field in your capture context request and set the field value to `true`. When the **comboCard** field value is set to `true`, the option to use a debit or credit card appears for all Visa cards that are entered in Unified Checkout and for all cards that are already stored in Click to Pay. If you do not want to offer a combo card at checkout, do not include the **comboCard** field in your capture context request:

```
"captureMandate" : {
      "comboCard": true
}
```

Important: This feature is available only in Brazil.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

captureMandate.CPF

The Cadastro de Pessoas Físicas (CPF) Brazilian tax ID feature is for customers in Brazil and provides your customers with a way to include their Consumer National Identifier when it is requested at checkout. Include this field in the capture context to display this field within the flow for manual card entry and Click to Pay transactions:

```
"captureMandate" : {
    "CPF": {
        "required": true
    }
}
```

Important: This feature is available only in Brazil.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

captureMandate.requestSaveCredentials

This feature enables you to display a consent option in the Unified Checkout UI for the cardholder to save their payment details for future use. If you use the complete mandate to create a token, see Sessions API (on page 179).

When you use this field without using the complete mandate, the transient token payload includes the **consumerPreference.saveCard** field with the value set to `true` when the cardholder has checked to save the payment information for future purchases:

```
"captureMandate" : {
      "requestSaveCredentials": true
}
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

captureMandate.showConfirmationStep

When **showConfirmstionStep** is set to `false`, you can remove the final summary confirmation screens from the checkout experience. When the UI displays cardholder data, the cardholder can review and, if necessary, edit their payment details before checkout is complete.

```
{
  "captureMandate": {
    "showConfirmationStep": false
  }
}
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

captureMandate.billingType

`PARTIAL`: Only the billing postal code and billing country are collected in the UI. Set to this value when you use relaxed address verification services (AVS). This includes markets where postal code and billing country are enough for successful payment processing.

`NONE`: No fields are shown in the UI to capture cardholder billing details. If you are using the Complete Mandate, you must provide billing details in the capture context. All information that is collected from these fields is tokenized in the transient token and sent for payment processing. For information about which fields are required for payment processing, see the Payments Developer Guide.

`FULL`: These fields are shown in the UI to capture cardholder billing details. When you include the billing details in the capture context, these details are pre-filled in the Unified Checkout UI. All information that is collected from these fields are tokenized in the transient token and sent for payment processing where the Complete Mandate is used.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

captureMandate.requestEmail

`false`: No email address is shown in the UI. If you are using Click to Pay, this email address is used to find the cardholder's Click to Pay account and it appears in the UI when **requestEmail** is set to `false`.

`true`: The email address is shown and captured in the UI. If you are using Click to Pay, this email address is used to find the cardholder's Click to Pay account.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

captureMandate.requestPhone

`false`: No phone number is shown or captured in the UI.

`true`: The phone number is shown and captured in the UI.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

captureMandate.requestShipping

`false`: No shipping information is captured in the UI. When shipping details are required for payment processing and are used for follow on services such as Decision Manager, you can include these fields in the capture context. These details are tokenized and passed through.

`true`: Shipping fields are shown in the UI and are collected by Unified Checkout. When you include the shipping details in the capture context, the information appears prefilled in the UI.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

captureMandate.shipToCountries

When the **requestShipping** field is set to `true`, only the countries that are included in this field can be selected by the cardholder for their shipping address.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide

API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Include Card Prefix

You can control the length of the card number prefix to be received in the response to the capture context `/sessions` request:

- Six digits
- Eight digits
- No prefix

To specify your preferred card number prefix length, include or exclude the **transientTokenResponseOptions.includeCardPrefix** field in the capture context `/sessions` request.

To receive a six-digit card number prefix in the response, follow this step:

Do not include the **transientTokenResponseOptions.includeCardPrefix** field in the capture context `/sessions` request.

This example shows how a six-digit card number prefix `411111` is returned in the transient token response:

```
"maskedValue" : "XXXXXXXXXXXX1111”,
                        "bin" : "411111"
```

To receive an eight-digit card number prefix in the response, follow this step:

Include the **transientTokenResponseOptions.includeCardPrefix** field in the capture context request, and set the value to `true`.

Important: This PCI DSS requirement applies only to card numbers longer than 15 digits and only for Discover, JCB, Mastercard, UnionPay, and Visa brands.

- If the card type entered is not part of these brands, a six-digit card number prefix is returned instead.
- If the card type entered is not part of these brands but is *co-branded* with these brands, an eight-digit card number prefix is returned.

This example shows how an eight-digit card prefix `41111102` is returned in the transient token response:

```
"maskedValue" : "XXXXXXXXXXXX1111”,
                        "prefix" : "41111102"
```

To not receive a card number prefix in the response, follow this step:

Include the **transientTokenResponseOptions.includeCardPrefix** field in the capture context request, and set the value to `false`.

This example shows how a card number is returned without a card number prefix in the transient token response:

```
"maskedValue" : "XXXXXXXXXXXX1111"
```

Best practice: If your application does not require card number prefix information for routing or identification, Cybersource recommends that you include the **transientTokenResponseOptions.includeCardPrefix** field in the capture context request and set its value to `false`. Doing so limits the exposure of payment data to only what is necessary for your processing needs.

For more information about PCI DSS, see *Frequently Asked Questions* on the PCI Security Standards Council site.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Email Autolookup

When you include Click to Pay as an **allowedPaymentType**, an automatic email lookup occurs when an email address is included in the capture context request. If the user has a Click to Pay account but is not on a recognized device, a one-time password (OTP) screen appears and the user is prompted to enter their OTP. If the user does not have a Click to Pay account, the user must enter their card information manually. They will have the option to create a Click to Pay account.

To enable email autolookup, you must include `CLICKTOPAY` as a value in the **allowedPaymentTypes** field and include an email address in the capture context.

Important: When the **captureMandate.requestEmail** and **captureMandate.requestPhone** fields are set to `false`, the contact details section is not displayed in the UI.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Mobile as Identity for Click to Pay

Click to Pay supports mobile numbers as way to identify a user. This enables cardholders to use their mobile number instead of their email address in certain markets for Visa and Mastercard transactions.

When the **requestEmail** field is set to `false` and the **requestPhone** field is set to `true`, the cardholder is identified using the provided mobile number. When the **requestEmail** field is set to `true` and the **requestPhone** field is set to `false`, the cardholder is identified using the provided email address. When the **requestEmail** field is set to `true` and the **requestPhone** field is also set to `true`, the cardholder is identified using the provided email address first and then the mobile number if there is no match.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

Example: Unified Checkout Complete Capture Context

Capture Context Request

```
{
  "country": "US",
  "locale": "en_GB",
  "targetOrigins": [
    "https://merchant.com",
    "https://reseller.com:8443"
  ],
  "clientVersion": "1.0",
  "allowedCardNetworks": [
    "VISA",
    "MASTERCARD",
    "AMEX",
    "JCB",
    "DISCOVER",
    "DINERSCLUB",
    "CARTESBANCAIRES",
    "EFTPOS",
    "JCREW",
    "MEEZA",
    "CUP",
    "CARNET",
    "MADA",
    "ELO",
    "MAESTRO",
    "PAYPAK",
    "JAYWAN",
    "KCP",
    "UATP"
  ],
  "allowedPaymentTypes": [
    "PANENTRY",
    "GOOGLEPAY",
    "CLICKTOPAY",
    "APPLEPAY",
    "PAZE",
    "CHECK",
```

```
    "AFTERPAY",
    "IDEAL",
    "MULTIBANCO",
    "PRZELEWY24",
    "MYBANK",
    "KONBINI",
    "DRAGONPAY",
    "BANCONTACT",
    "TINKPAYBYBANK"
  ],
  "appearance": {
    "theme": "LIGHT",
    "variables": {
      "primaryColor": "#007bff",
      "secondaryColor": "#6c757d",
      "fontFamily": "Arial, sans-serif",
      "fontSize": "14px",
      "borderRadius": "4px"
    }
  },
  "buttonType": "CHECKOUT",
  "captureMandate": {
    "showConfirmationStep": true,
    "billingType": "FULL",
    "requestEmail": true,
    "requestPhone": true,
    "requestShipping": true,
    "shipToCountries": [
      "US",
      "GB",
      "CA"
    ],
    "showAcceptedNetworkIcons": true,
    "comboCard": true,
    "requestSaveCredentials": true,
    "CPF": {
      "required": true
    }
  },
  "completeMandate": {
    "type": "CAPTURE",
    "decisionManager": true,
    "consumerAuthentication": "3DS",
    "tms": {
      "tokenCreate": true,
      "tokenTypes": [
        "customer",
        "paymentInstrument",
        "instrumentIdentifier",
```

```
      "shippingAddress"
    ]
  }
},
"paymentConfigurations": {
  "PANENTRY": {
    "customer": "existing_customer_token_123"
  },
  "GOOGLEPAY": {
    "allowedAuthMethods": [
      "PAN_ONLY",
      "CRYPTOGRAM_3DS"
    ]
  },
  "CLICKTOPAY": {
    "autoCheckEnrollment": true
  }
},
"transientTokenResponseOptions": {
  "includeCardPrefix": true
},
"data": {
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "102.21",
      "currency": "USD",
      "surcharge": {
        "amount": "2.50"
      },
      "discountAmount": "2.00",
      "serviceFeeAmount": "5.00",
      "taxAmount": "10.00",
      "taxDetails": [
        {
          "taxId": "1234",
          "type": "N"
        },
        {
          "taxId": "5678",
          "type": "S"
        }
      ]
    },
    "billTo": {
      "address1": "123 Main Street",
      "address2": "Apt 4B",
      "address3": "Building C",
      "address4": "Floor 3",
      "administrativeArea": "CA",
```

```
      "buildingNumber": "123",
      "country": "US",
      "district": "Downtown",
      "locality": "San Francisco",
      "postalCode": "94105",
      "email": "jane.doe@visa.com",
      "firstName": "John",
      "middleName": "Michael",
      "lastName": "Doe",
      "phoneNumber": "+1-123456789",
      "phoneType": "night",
      "nameSuffix": "Mr",
      "title": "Software Engineer",
      "company": {
        "name": "Visa Inc",
        "address1": "900 Metro Center Blvd",
        "address2": "Suite 200",
        "country": "US",
        "administrativeArea": "CA",
        "postalCode": "94404",
        "locality": "Foster City"
      }
    },
    "shipTo": {
      "address1": "456 Oak Avenue",
      "address2": "Suite 100",
      "address3": "Building A",
      "address4": "Level 2",
      "administrativeArea": "NY",
      "buildingNumber": "456",
      "country": "US",
      "district": "Midtown",
      "locality": "New York",
      "postalCode": "10001",
      "firstName": "Jane",
      "lastName": "Smith"
    },
    "lineItems": [
      {
        "productCode": "WIDGET-001",
        "productName": "Premium Widget",
        "productSku": "WID-PRE-001",
        "quantity": 2,
        "unitPrice": "45.50",
        "unitOfMeasure": "EA",
        "totalAmount": "91.00",
        "taxAmount": "7.28",
        "taxRate": "0.08",
        "taxAppliedAfterDiscount": "y",
```

```
          "taxStatusIndicator": "N",
          "taxTypeCode": "1234",
          "amountIncludesTax": true,
          "typeOfSupply": "12",
          "commodityCode": "COMM-001",
          "discountAmount": "5.00",
          "discountApplied": true,
          "discountRate": "0.05",
          "invoiceNumber": "INV-2024-001",
          "taxDetails": [
            {
              "type": "STATE",
              "amount": "3.64",
              "rate": "0.04",
              "code": "1234",
              "taxId": "TAX-001",
              "applied": true,
              "exemptionCode": "1"
            },
            {
              "type": "LOCAL",
              "amount": "3.64",
              "rate": "0.04",
              "code": "5678",
              "taxId": "TAX-002",
              "applied": true,
              "exemptionCode": "2"
            }
          ],
          "fulfillmentType": "SHIP",
          "weight": "500",
          "weightIdentifier": "N",
          "weightUnit": "mg",
          "referenceDataCode": "REF-001",
          "referenceDataNumber": "REF-NUM-001",
          "unitTaxAmount": "3.64",
          "productDescription": "High-quality premium widget with extended
warranty",
          "giftCardCurrency": "USD",
          "shippingDestinationTypes": "residential",
          "gift": false,
          "passenger": {
            "type": "ADT",
            "status": "confirmed",
            "phone": "+1-123456789",
            "firstName": "Robert",
            "lastName": "Johnson",
            "id": "PASS-001",
            "email": "jane.doe@visa.com",
```

```
            "nationality": "US"
          }
        },
        {
          "productCode": "GADGET-002",
          "productName": "Digital Gadget",
          "productSku": "GAD-DIG-002",
          "quantity": 1,
          "unitPrice": "29.99",
          "unitOfMeasure": "EA",
          "totalAmount": "29.99",
          "taxAmount": "2.40",
          "taxRate": "0.08",
          "taxAppliedAfterDiscount": "n",
          "taxStatusIndicator": "Y",
          "taxTypeCode": "5678",
          "amountIncludesTax": false,
          "typeOfSupply": "11",
          "commodityCode": "COMM-002",
          "discountAmount": "3.00",
          "discountApplied": true,
          "discountRate": "0.10",
          "invoiceNumber": "INV-2024-002",
          "taxDetails": [
            {
              "type": "FEDERAL",
              "amount": "2.40",
              "rate": "0.08",
              "code": "9012",
              "taxId": "TAX-003",
              "applied": true,
              "exemptionCode": "0"
            }
          ],
          "fulfillmentType": "DIGITAL",
          "weight": "0",
          "weightIdentifier": "Y",
          "weightUnit": "g",
          "referenceDataCode": "REF-002",
          "referenceDataNumber": "REF-NUM-002",
          "unitTaxAmount": "2.40",
          "productDescription": "Advanced digital gadget with cloud sync",
          "giftCardCurrency": "EUR",
          "shippingDestinationTypes": "commercial",
          "gift": true,
          "passenger": {
            "type": "CHD",
            "status": "pending",
            "phone": "+1-123456789",
```

```
            "firstName": "Emily",
            "lastName": "Williams",
            "id": "PASS-002",
            "email": "jane.doe@visa.com",
            "nationality": "GB"
          }
        }
      ],
      "invoiceDetails": {
        "invoiceNumber": "INV-MAIN-2024-001",
        "productDescription": "Multiple items including widgets and gadgets"
      }
    },
    "buyerInformation": {
      "personalIdentification": [
        {
          "type": "CPF",
          "id": "01234567890"
        }
      ],
      "merchantCustomerId": "CUST-12345",
      "companyTaxId": "123456789",
      "dateOfBirth": "19901215",
      "language": "en"
    },
    "clientReferenceInformation": {
      "code": "TAGX001",
      "partner": {
        "developerId": "DEV-1234",
        "solutionId": "SOL-4567"
      }
    },
    "consumerAuthenticationInformation": {
      "challengeCode": "01",
      "messageCategory": "01",
      "acsWindowSize": "01"
    },
    "merchantInformation": {
      "merchantDescriptor": {
        "name": "Jane Sales",
        "alternateName": "BIG SALES INC",
        "locality": "New York",
        "phone": "+1-123456789",
        "country": "US",
        "postalCode": "170056",
        "administrativeArea": "NY",
        "address1": "123 47TH STREET"
      }
    },
```

```
    "processingInformation": {
      "reconciliationId": "01234567",
      "authorizationOptions": {
        "aftIndicator": true,
        "authIndicator": "Y",
        "ignoreCvResult": true,
        "ignoreAvsResult": true,
        "initiator": {
          "credentialStoredOnFile": true,
          "merchantInitiatedTransaction": {
            "reason": "1"
          }
        }
      },
      "businessApplicationId": "AA",
      "commerceIndicator": "recurring",
      "processingInstruction": "ORDER_SAVED_EXPLICITLY"
    },
    "recipientInformation": {
      "firstName": "John",
      "middleName": "A",
      "lastName": "Buyer",
      "country": "GB",
      "accountId": "acc0123567",
      "administrativeArea": "GB",
      "accountType": "01",
      "dateOfBirth": "19901215",
      "postalCode": "170056"
    },
    "merchantDefinedInformation": [
      {
        "key": "promo_code",
        "value": "DISCOUNT20"
      },
      {
        "key": "customer_tier",
        "value": "gold"
      }
    ],
    "deviceInformation": {
      "ipAddress": "192.168.1.100"
    },
    "paymentInformation": {
      "card": {
        "typeSelectionIndicator": "0"
      }
    }
  }
}
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Validating the Capture Context

The capture context that you generate is a JSON Web Token (JWT) data object. The JWT is digitally signed using a public key and confirms the validity of the JWT and that it comes from Cybersource. When you do not have a key in the JWT header, Cybersource recommends that you follow cryptography best practices and validate the capture context signature.

To validate a JWT, you must obtain its public key. This public RSA key is in JSON Web Key (JWK) format. The public key is associated with the capture context on the Cybersource domain.

To get the public key of a capture context from the header of the capture context itself, you must retrieve the key ID associated with the public key and then pass the key ID to the `/flex/v2/public-keys` endpoint:

1. From the header of the capture context, get the key ID (**kid**):

```
{
    "kid": "3g",
    "alg": "RS256"
}
```

2. Send a GET request to the `/flex/v2/public-keys` endpoint and include the key ID. For example:

   - **Test:** `GET https://apitest.cybersource.com/flex/v2/public-keys/{3g}`
   - **Production:** `GET https://api.cybersource.com/flex/v2/public-keys/{3g}`
   - **Production in Saudi Arabia:** `GET https://api.sa.cybersource.com/flex/v2/public-keys/{3g}`
   - **Test in Saudi Arabia:** `GET https://apitest.sa.cybersource.com/flex/v2/public-keys/{3g}`

   Depending on the cryptographic method you use to validate the public key, you might need to convert the key to privacy-enhanced mail (PEM) format.

3. The resource returns the public key:

```
eyJmbHgiOnsicGF0aCI6Ii9mbGV4L3YyL3Rva2VucyIsImRhdGEiOiI2bUFLNTNPNVpGTUk5Y3Rob
WZmd2doQUFFRGNqNU5QYzcxelErbm8reDN6WStLOTVWQ2c5bThmQWs4czlTRXBtT21zMmVhbEx5Nk
hHZ29oQ0JEWjVlN3ZUSGQ5YTR5a2tNRDlNVHhqK3ZoWXVDUmRDaDhVY1dwVUNZWlZnbTE1UXVFMkE
iLCJvcmlnaW4iOiJodHRwczovL3Rlc3RmbGV4LmN5YmVyc291cmNlLmNvbSIsImp3ayI6eyJrdHki
OiJSU0EiLCJlIjoiQVFBQiIsInVzZSI6ImVuYyIsIm4iOiJyQmZwdDRjeGlkcVZwT0pmVTlJQXcwU
1JCNUZqN0xMZjA4U0R0VmNyUjlaajA2bEYwTVc1aUpZb3F6R3ROdnBIMnFZbFN6LVRsSDdybVNTUE
ZIeTFJQ3BfZ0I3eURjQnJ0RWNEanpLeVNZSTVCVjNsNHh6Qk5CNzRJdnB2Smtqcnd3QVZvVU4wM1R
aT3FVc0pfSy1jT0xpYzVXV0ZhQTEyOUthWFZrZFd3N3c3LVBLdnMwNmpjeGwyV05STUIzTS1ZQ0xO
b3FCdkdCSk5oYy1uM1lBNU5hazB2NDdiYUswYWdHQXRfWEZ0ZGItZkphVUVUTW5WdW9fQmRhVm90d
1NqUFNaOHFMOGkzWUdmemp2MURDTUM2WURZRzlmX0tqNzJjTi1OaG9BRURWUlZyTUtiZ3QyRDlwWk
J1d2gzZlNfS3VRclFWTVdPelRnT3AzT2s3UVFGZ1EiLCJraWQiOiIwOEJhWXMxbjdKTUhjSDh1bkc
xc1NDUVdxN2VveWQ1ZyJ9fSwiY3R4IjpbeyJkYXRhIjp7InRhcmdldE9yaWdpbnMiOlsiaHR0cHM6
Ly93d3cudGVzdC5jb20iXSwibWZPcmlnaW4iOiJodHRwczovL3Rlc3RmbGV4LmN5YmVyc291cmNlL
mNvbSJ9LCJ0eXBlIjoibWYtMC4xMS4wIn1dLCJpc3MiOiJGbGV4IEFQSSIsImV4cCI6MTYxNjc3OT
A5MSwiaWF0IjoxNjE2Nzc4MTkxLCJqdGkiOiJ6SG1tZ25uaTVoN3ptdGY0In0.GvBzyw6JKl3b2Pz
tHb9rZXawx2T817nYqu6goxpe4PsjqBY1qeTo19R-CP_DkJXov9hdJZgdlzlNmRY6yoiziSZnGJdp
nZ-pCqIlC06qrpJVEDob3O_efR9L03Gz7F5JlLOiTXSj6nVwC5mRlcP032ytPDEx5TMI9Y0hmBadJ
YnhEMwQnn_paMm3wLh2v6rfTkaBqd8n6rPvCNrWMOwoMdoTeFxku-
```

Use this public RSA key to validate the capture context.

4. Parse the JWT capture context to get the **kid** from its header:

```
{
    "kid": "3g",
    "alg": "RS256"
}
```

5. Send a GET request to retrieve the public key from `/flex/v2/public-keys/3g`:

```
{
    "kty":"RSA",
    "use":"enc",
    "kid":"3g",
    "n":"ir7Nl1Bj8G9rxr3co5v_JLkP3o9UxXZRX1LIZFZeckguEf7Gdt5kGFFfTsymKBesm3Pe
     8o1hwfkq7KmJZEZSuDbiJSZvFBZycK2pEeBjycahw9CqOweM7aKG2F_bhwVHrY4YdKsp
     _cSJe_ZMXFUqYmjk7D0p7clX6CmR1QgMl41Ajb7NHI23uOWL7PyfJQwP1X8HdunE6ZwK
     DNcavqxOW5VuW6nfsGvtygKQxjeHrI-gpyMXF0e_PeVpUIG0KVjmb5-em_Vd2SbyPNme
     nADGJGCmECYMgL5hEvnTuyAybwgVwuM9amyfFqIbRcrAIzclT4jQBeZFwkzZfQF7MgA6QQ",
     "e":"AQAB"
}
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Session Validation

The session JWT is digitally signed using RS256. You must confirm that it was issued by Cybersource and has not been tampered with. Follow these steps to validate the signature:

1. Parse the session JWT header to extract the key ID (`kid`):

```
{
  "kid": "3g",
  "alg": "RS256"
}
```

2. Retrieve the public key by sending a request to the `/flex/v2/public-keys/{kid}` endpoint:
   - **Test**: GET `apitest.cybersource.comflex/v2/public-keys/{kid}`
   - **Production**: GET `api.cybersource.comflex/v2/public-keys/{kid}`
3. Use the returned RSA public key in JSON Web Key format to verify the JWT signature.

> **Important:** Depending on the cryptographic library that tou use, you may need to convert the key to Privacy-Enhanced Mail (PEM) format.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

Transient Tokens

The response to a successful customer interaction with Unified Checkout is a transient token. This is returned in the response from the **checkout.mount()** function. The transient token is a reference to the payment data collected on your behalf. Transient tokens allow secure card payments to occur without risk of exposure to sensitive payment information. The transient token is a short-term token that expires after 15 minutes. This reduces your PCI burden/responsibility and ensures that sensitive information is not exposed to your back-end systems.

Transient tokens can be included requests sent to the Payment Details API for the customer payment data that is collected.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Transient Token Format

The transient token is issued as a JSON Web Token (JWT) (RFC 7519). For information on JSON Web Tokens, see JSON Web Tokens (on page 266).

The payload portion of the token is a Base64URL-encoded JSON string and contains various claims. For more information, see JSON Web Tokens (on page 266).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Example: Transient Token Format

Transient Token Payload

Important: The empty field values in the transient token indicate which fields were captured by the application without exposing you to personally identifiable information directly.


```
{
  "metadata" : {
    "sequenceNumber" : "1",
    "cardholderAuthenticationStatus" : false,
    "paymentType" : "PANENTRY"
  },
  "iss" : "Flex/00",
  "exp" : 1762870464,
  "type" : "gda-0.10.0",
  "iat" : 1762869564,
  "jti" : "1D4Q8FJSSZ9ASKQ9ZCJ7E13IFOITOOH2GGHY6TRZ3O28TUQ1BN8H691344C098CA",
  "content" : {
    "deviceInformation" : {
      "fingerprintSessionId" : { }
    },
    "orderInformation" : {
      "billTo" : {
        "country" : { },
        "lastName" : { },
        "firstName" : { },
        "phoneNumber" : { },
        "address1" : { },
        "postalCode" : { },
        "locality" : { },
        "buildingNumber" : { },
        "company" : {
          "name" : { }
        },
        "administrativeArea" : { },
        "email" : { }
      },
      "amountDetails" : {
        "totalAmount" : { },
        "currency" : { }
      },
      "shipTo" : {
        "firstName" : { },
```

```
        "lastName" : { },
        "country" : { },
        "address1" : { },
        "postalCode" : { },
        "locality" : { },
        "buildingNumber" : { },
        "administrativeArea" : { }
      }
    },
    "paymentInformation" : {
      "card" : {
        "expirationYear" : {
          "value" : "2027"
        },
        "number" : {
          "maskedValue" : "XXXXXXXXXXXX1111",
          "bin" : "411111"
        },
        "securityCode" : { },
        "expirationMonth" : {
          "value" : "03"
        },
        "typeSelectionIndicator" : {
          "value" : "1"
        },
        "type" : {
          "value" : "001"
        }
      }
    }
  }
}
```

PAN BIN in `metadata` Object

The `cardDetails` object, including the PAN BIN, is included in the transient token `metadata` when a Click to Pay network token is used as a payment method. This allows you to display information about the card on invoices and see the BIN details that are linked to the underlying card.

```
"metadata": {
  "cardDetails": {
    "suffix": "9876",
    "prefix": "123456",
    "expirationMonth": "MM",
    "expirationYear": "YYYY"
  }
}
```

Authentication Status in **metadata** Object

The `cardholderAuthenticationStatus` object is included in the `metadata` and enables you to determine if the payload is fully authenticated. When `cardholderAuthenticationStatus` is set to `true`, the payload is fully authenticated. When `cardholderAuthenticationStatus` is set to `false`, the transaction is not authenticated.

If you are using Unified Checkout with **unifiedPayment.complete()** and **consumerAuthentication** is set to `3DS` or `PASSKEY` in the complete mandate request, then Payer Authentication is called automatically if it is available for the selected payment method and card network. If you use a transient token to request follow-on services directly, the value of this field indicates if the transaction has been authenticated.

```
"metadata": {
  "cardholderAuthenticationStatus": "true"
  }
}
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Token Verification

When you receive the transient token, you should cryptographically verify its integrity using the public key embedded within the capture context. Doing so verifies that Cybersource issued the token and that the data has not been tampered with in transit. Verifying the transient token JWT involves verifying the signature and various claims within the token. Programming languages each have their own specific libraries to assist. For an example in Java, see: Java Example in Github.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

Dual-Branded Cards

Unified Checkout accepts dual-branded cards. To use this feature, you must include the card networks that have overlapping BIN ranges in the capture context request. For example:

```
"allowedCardNetworks": ["VISA", "MASTERCARD", "AMEX", "CARTESBANCAIRES"]
```

When a card number within an overlapping BIN range is entered, the network that is listed first in the value array for the **allowedCardNetworks** field is used. Based on the previous example, if the card number 403550XXXXXXXXXX is entered, the payment network for payment processing is Visa.

During the transaction, the card type is populated with the first network in the list, and the **detectedCardTypes** field returned in the transient token includes all of the detected card types in the transient token.

The **detectedCardTypes** field is returned in the transient token response only when more than one card type is detected.

If you include Cartes Bancaires as a supported dual-branded card type, Unified Checkout displays a radio button with Visa and Mastercard options at checkout. This enables the customer to select which payment scheme they want to use to process the payment. The radio button defaults to the card type that you specify in the capture context request, but the payment is processed using the option selected by the customer during checkout.

Domestic Scheme Rules

For markets where domestic routing, network preference, or co-badging regulations apply, you are responsible for ensuring that domestic payment schemes are prioritized appropriately within the **allowedCardNetworks** configuration.

In these scenarios, domestic schemes must be listed ahead of international networks such as Visa or Mastercard where required by local regulations or scheme rules.

If network preference varies by transaction, the order of networks should be dynamically configured when you create the capture context request using the Sessions API.

You are responsible for reviewing and complying with all applicable local regulatory, scheme, and acquirer requirements relating to domestic routing and network selection.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Authorizations with a Transient Token

This section provides the information required in order to perform a successful authorization with a Unified Checkout transient token. You can use this method to construct more complex payment scenarios that are not supported by the `unifiedPayments.complete()` payment method.

Important: When you process payments through Unified Checkout using **unifiedPayments.complete()**, Unified Checkout invokes service orchestration directly. When you send an authorization request using a transient token, you must request the follow-on services that you want to use. For information about the required fields for the payment services that you request, see the Payments Developer Guide.

The transient token is a short-term token that expires after 15 minutes. Doing so eliminates the need to send sensitive payment data along with the request. For more information on transient tokens, see Transient Tokens (on page 214).

To send the transient token with a request, use the **tokenInformation.transientTokenJwt** field.

This example shows a transient token in the context of an authorization request:

```
"tokenInformation": {
    "transientTokenJwt":
 "eyJraWQiOiIwOG4zUnVsRTJGQXJDRktycVRkZFlkWGZSWFhMNXFoNSIsImFsZyI6IlJTMjU2In0.eyJp
c3MiOiJGbGV4LzA3IiwiZXhwIjoxNTk3MDg0ODk3LCJ0eXBlIjoiZ2RhLTAuMS4xIiwiaWF0IjoxNTk3MD
gzOTk3LCJqdGkiOiIxQzI2VlpSkVJUU1PTzVIMDUwNEtINDdJMEFNMklaRkM0M1Y1TDU0MUhCTE45Q09JM
0w3NUYzMTk0RTE5NkExIn0.SNm1VZaZr3DkTqUg9CdV0F5arRe-uQU9oUWPKfWIpbIzIPZutRokv5DSDcM
7asZIKNJyNIBx5DLsl_yQPrKgzhwQxZ8qbhto7cu3t-v8DHG2yO951plPQVQnj7x-vEDcXkLUL1F8sqY23
R5HW-xSDAQ3AFLawCckn7Q2eudRGeuMhLWH742Gflf9Hz3KyKnmeNKA3o9yW2na16nmeVZaYGqbUSPVITd
l5cMA0o9lEob8E3OQH0HHdmIsu5uMA4x7DeBjfTKD1rQxFP3JBNVcv30AIMLkNcw0pHbtHDVzKBWxUVxvn
m3zFEdiBuSAco2uWhC9zFqHrrp64ZvzxZqoGA"
}
```

To retrieve non-sensitive data from a Unified Checkout transient token, use the `payment-details` endpoint. This data includes cardholder name and billing and shipping details. For more information, see Payment Details API (on page 241).

Important: Fields supplied directly in an API request supersede those that are also present in the transient token. For example, in the request below, the total amount might have been overridden because of a tax calculation.

Endpoint

Production: `POST https://api.cybersource.com/pts/v2/payments`

Test: `POST https://apitest.cybersource.com/pts/v2/payments`

Production in Saudi Arabia: `POST https://api.sa.cybersource.com/pts/v2/payments`

Test in Saudi Arabia: `POST https://apitest.sa.cybersource.com/pts/v2/payments`

Required Field for an Authorization with a Transient Token

tokenInformation.transientTokenJwt

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

REST Example: Requesting an Authorization with a Transient Token

```
{
  "clientReferenceInformation": {
    "code": "TC50171_3"
  },
  "processingInformation": {
    "commerceIndicator": "internet"
  },
  "tokenInformation": {
    "transientTokenJwt":
 "eyJraWQiOiIwOG4zUnVsRTJGQXJDRktycVRkZFlkWGZSWFhMNXFoNSIsImFs
ZyI6IlJTMjU2In0.eyJpc3MiOiJGbGV4LzA3IiwiZXhwIjoxNTk3MDg0ODk3LCJ0eXBlIjoiZ2RhLTAuMS
4xIi
wiaWF0IjoxNTk3MDgzOTk3LCJqdGkiOiIxQzI2VlpSRkVJUU1PTzVIMDUwNEtINDdJMEFNMklaRkM0M1Y1
TDU0
MUhCTE45Q09JM0w3NUYzMTk0RTE5NkExIn0.SNm1VZaZr3DkTqUg9CdV0F5arRe-uQU9oUWPKfWIpbIzIP
ZutR
okv5DSDcM7asZIKNJyNIBx5DLsl_yQPrKgzhwQxZ8qbhto7cu3t-v8DHG2yO951plPQVQnj7x-vEDcXkLU
L1F8
sqY23R5HW-xSDAQ3AFLawCckn7Q2eudRGeuMhLWH742Gflf9Hz3KyKnmeNKA3o9yW2na16nmeVZaYGqbUS
PVIT
dl5cMA0o9lEob8E3OQH0HHdmIsu5uMA4x7DeBjfTKD1rQxFP3JBNVcv30AIMLkNcw0pHbtHDVzKBWxUVxv
nm3z
FEdiBuSAco2uWhC9zFqHrrp64ZvzxZqoGA"
```

```
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "21.00",
      "currency": "USD"
    },
    "billTo": {
      "firstName": "John",
      "lastName": "Doe",
      "address1": "1Market St",
      "address2": "Address 2",
      "locality": "san francisco",
      "administrativeArea": "CA",
      "postalCode": "94105",
      "country": "US",
      "email": "test@cybs.com",
      "phoneNumber": "4158880000"
    }
  }
}
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Test Your Configuration

Use these test payment credentials to test your Unified Checkout configuration:

- Unified Checkout Test Cards (on page 223)
- Visa and Mastercard Click to Pay Test Cards (on page 225)
- Test Cards for Authentication by Click to Pay (on page 227)
- Echeck Test Values (on page 228)

You can handle errors and test your configuration using these topics:

- Handle Errors (on page 231)
- Reason Codes (on page 237)

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Unified Checkout Test Cards

Use these test card numbers to test your Unified Checkout configuration.

Combine the BIN with the card number when sending to Unified Checkout.

To test Payer Authentication, you must use Payer Authentication test cards. See Test Cases for 3-D Secure 2.x in the *Payer Authentication Developer Guide*.

Test Card Numbers

Card Brand	BIN	Card Number	Expiration Date	CVV
Visa	411111	1111111111	12/2026	123
Mastercard	555555	5555554444	02/2026	265

Test Card Numbers (continued)

Card Brand	BIN	Card Number	Expiration Date	CVV
American Express	378282	246310005	03/2026	7890
Cartes Bancaires	436000	0001000005	04/2040	123
Carnet	506221	0000000009	04/2026	123
China UnionPay	627988	6248094966	04/2040	123
Diners Club	305693	09025904	04/2040	123
Discover	644564	4564456445	04/2040	123
JCB	353011	13333 0000	04/2040	123
Jaywan	679009	0000002009	04/2040	123
Jaywan	669000	0000000000	04/2040	123
KCP	949022	0011669217	04/2040	
Paypak	220543	0000003002	04/2040	123
Maestro	675964	9826438453	04/2040	123
mada	446404	0000000007	04/2040	123
ELO	451416	0000000003	04/2040	123
JCrew	515997	1500000005	04/2040	123
EFTPOS	401795	000000000009	04/2040	123
Meeza	507808	3000000002	04/2040	123
UATP	148512	345678905	04/2040	

Test Cards for Click to Pay Authentication by Unified Checkout

se these test cards to test when you use a Click to Pay card with authentication performed outside Click to Pay and the **consumerAuthentication** field is set to `3DS` or `PASSKEY` in the capture context.

Replace the X in the card number with 4.

To manage Visa test cards for customer authentication, contact your implementation consultant or technical account manager.

Important: These test cards are not valid for testing in production. To test in production, you must leverage production credentials.

Test Card Numbers for Authentication Outside Click to Pay Flow

Card Brand	Card Number	Expiration Date	CVV
Visa	46229431231X2X56	12/2026	432
	46229431231X232X	12/2026	581
Mastercard	512X35X1XXX64578	Any future date	Any
	512X35X1XXX64552	Any future date	Any

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Visa and Mastercard Click to Pay Test Cards

Visa Test Cards

These Visa test cards can be added to your Click to Pay wallet.

Replace the X in the card number with 4.

You can manage your Visa Click to Pay test cards and account here:

- **Production:** https://src.visa.com/login
- **Test:** https://sandbox.src.visa.com/login

Important: Visa Click to Pay may request additional verification details during testing, such as CVV confirmation.

To manage Visa test cards for customer authentication, contact your implementation consultant or technical account manager.

Important: These test cards are not valid for testing in production. To test in production, you must leverage production credentials.

Visa Test Card Numbers

Card Number	Expiration Date	CVV
x6229x3123123755	12/2029	728
x6229x3123123763	12/2029	605
x6229x3123123771	12/2029	694
x6229x3123123789	12/2029	881
x6229x3123123797	12/2029	678
x6229x3123123805	12/2029	084
x6229x3123123813	12/2029	127
x6229x3123123821	12/2029	218
x6229x3123123839	12/2029	114
x6229x31231238x7	12/2029	867
x6229x312312385x	12/2029	301

Mastercard Test Cards

Mastercard test cards can be added to your Click to Pay wallet. You must retrieve Mastercard test cards from their Click to Pay test page: https://developer.mastercard.com/mastercard-checkout-solutions/documentation/testing/test_cases/click_to_pay_case/#test-cards

For Mastercard Click to Pay, the user will see a loading Mastercard window where they will be prompted to enter additional information into this window

Mastercard has different test cards for retrieving tokenized and non-tokenized data. Cybersource recommends that you use these test cards as follows:

- Test cards to retrieve PAN data: Use these cards when the customer is completing checkout as a one-time guest and does not have a Click to Pay account or want to create one.
- Test cards to retrieve token data: Use these cards for tokenized Click to Pay transactions.
- Tokenize eligible test cards for token authentication: Use these test cards to attempt Click to Pay authentication when Click to Pay authentication is enabled.

You can manage your Mastercard Click to Pay test cards and account here:

- **Live:** https://src.mastercard.com/profile/enroll
- **SBX:** https://sandbox.src.mastercard.com/profile/enroll

Mastercard authentication test cards are available on the *Mastercard Checkout Solutions* page in the Mastercard Developer Center.

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Test Cards for Authentication by Click to Pay

Use these cards when authentication is performed by Click to Pay within the Click to Pay flow.

Replace the X in the card number with 4.

To manage Visa test cards for customer authentication, contact your implementation consultant or technical account manager.

Important: These test cards are not valid for testing in production. To test in production, you must leverage production credentials.

3-D Secure Friction

Card Brand	Card Number	Expiration Date	CVV
Visa	439584X300520271	12/2030	504
	439584X300600271	12/2030	184
	439584X300780271	12/2030	164
	439584X300860271	12/2030	668
	439584X300940271	12/2030	856

3-D Secure Frictionless

Card Brand	Card Number	Expiration Date	CVV
Visa	X3958X0000011019	12/2030	956
	X3958X0000011027	12/2030	550
	X3958X0000011035	12/2030	623
	X3958X00000110X3	12/2030	997
	X3958X0000011050	12/2030	790
	X3958X0000011068	12/2030	888
	X3958X0000011076	12/2030	924
	X3958X000001108X	12/2030	375
	X3958X0000011092	12/2030	865

Mastercard authentication test cards are available on the *Mastercard Checkout Solutions* page in the Mastercard Developer Center.

For information about testing authentication, see Test Authentication (on page 229). For information about enabling Visa customer authentication, see Set Up Customer Authentication for Click to Pay (on page 54).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Echeck Test Values

These eCheck test values can be used to process a test eCheck transactions:

- **Routing number:** Set to 071923284
- **Account number:** Set to any supported value. For example, 1234567890.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Test Authentication

Use this table to determine how to test your authentication method.

Authentication Testing by Product

Payment Method	Authentication	Minimum Follow-On Actions	Prerequisites	Test Cards	Details
PAN Entry	Payer Authentication through Unified Checkout	Authorization and Payer Authentication	The transacting MID must be enabled for Payer Authentication and the complete mandate is used with the **consumerAuthentication** field set to `3DS` or `PASSKEY`.	See Testing Payer Authentication in the *Payer Authentication Developer Guide.*	When the complete mandate is not used, Unified Checkout does not initiate authentication and you must perform authentication within your own environment.
Click to Pay	Payer Authentication through Unified Checkout	Authorization and Payer Authentication	The transacting MID must be enabled for Payer Authentication and the complete mandate is used with the **consumerAut**	See Unified Checkout Test Cards (on page 223).	When authentication is not enabled for Click to Pay or Click to Pay is not able to perform authentication for Click to Pay, Unified Checkout performs authentication

Authentication Testing by Product (continued)

Payment Method	Authentication	Minimum Follow-On Actions	Prerequisites	Test Cards	Details
			hentication field set to `3DS` or `PASSKEY`. Authentication for Click to Pay must not be configured.		using Payer Authentication when the complete mandate is used with the **consumerAuthentication** field set to `3DS` or `PASSKEY`.
Click to Pay	Visa Click to Pay	Authorization and Payer Authentication	You must configure the authentication for Click to Pay. Click to Pay performs authentication only if it is a tokenized Visa or Mastercard card.	See Test Cards for Authentication by Click to Pay (on page 227). Mastercard authentication test cards are available on the *Mastercard Checkout Solutions* page in the Mastercard Developer Center.	When authentication is enabled for Click to Pay, authentication is attempted for all Click to Pay transactions for Visa and Mastercard cards that are stored in Click to Pay. For information about setting up authentication for Visa Click to Pay, see Set Up Customer Authentication for Click to Pay (on page 54).
Google Pay	Google Pay	Authorization	A Google device must be used with biometric authentication for Google authentication.		A user authenticates themselves on a Google device with a tokenized Google Pay credential. The returned

Authentication Testing by Product (continued)

Payment Method	Authentication	Minimum Follow-On Actions	Prerequisites	Test Cards	Details
					payload from Google is authenticated.
Google Pay	Payer Authentication through Unified Checkout	Authorization and Payer Authentication	You must use a device, such as a web browser, that does not authenticate the cardholder as part of the authorization process.		Google returns an un-authenticated payload to Unified Checkout. Unified Checkout processes authentication via Payer Authentication when the complete mandate function is used with **consumerAuthentication**.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Handle Errors

The Unified Checkout SDK uses a structured error object for all error scenarios. Errors are returned as exceptions from asynchronous methods and are also returned as events for centralized handling.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

UnifiedCheckoutError

All SDK errors are instances of `UnifiedCheckoutError` with these properties:

`UnifiedCheckoutError` Properties

Property	Type	Description
`correlationId`	`string?`	The correlation ID from an underlying API call, when applicable.
`details`	`unknown?`	Additional error-specific information. This is often an array of objects.
`informationLink`	`string?`	The URL linked to the online documentation for this error.
`message`	`string`	This property is a human-readable description of the error.
`name`	`string`	The value is always `"UnifiedCheckoutError"`.
`reason`	`string`	This property is a machine-readable error code, such as `"CAPTURE_CONTEXT_INVALID"`.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Detect Errors

Errors may be serialized through `postMessage`. Cybersource recommends that you use the `name` property instead of `instanceof`:

```
try {
  const result = await checkout.mount('#buttons');
} catch (error) {
  if (error.name === 'UnifiedCheckoutError') {
    // Access error.reason, error.message, error.details
  }
}
```

You can also write a helper function that can be reused:

```
function isUnifiedCheckoutError(obj) {
  return (
    obj !== null &&
    typeof obj === 'object' &&
    obj.name === 'UnifiedCheckoutError' &&
    typeof obj.reason === 'string' &&
    typeof obj.message === 'string'
  );
}
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Error Handling Patterns

Try/Catch

```
try {
  const client = await VAS.UnifiedCheckout(sessionJWT);
  const checkout = await client.createCheckout();
  const result = await checkout.mount('#buttons');
} catch (error) {
  console.error(error.reason, error.message);
}
```

Promise .catch()

```
VAS.UnifiedCheckout(sessionJWT)
  .then(client => client.createCheckout())
  .then(checkout => checkout.mount('#buttons'))
  .catch(error => console.error(error.reason, error.message));
```

Centralized Error Logging via Events

Errors from all integrations are applicable up to the client level. Use `client.on('error')` for centralized logging:

```
const client = await VAS.UnifiedCheckout(sessionJWT);

client.on('error', (err) => {
  errorReporter.send({
    source: err.source,   // "checkout", "trigger", "button", or "client"
    code: err.code,
    message: err.message
  });
});
```

Cybersource recommends that you do this as it catches errors from all checkouts, triggers, and buttons created from this client instance.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Error Codes

Initialization Errors

These errors are returned during `VAS.UnifiedCheckout(sessionJWT)`:

Initialization Reason Values

Reason	Description
`CAPTURE_CONTEXT_EXPIRED`	The supplied JWT has expired. Generate a new session.
`CAPTURE_CONTEXT_INVALID`	The session JWT is not valid. For example, it has a bad signature or is malformed.
`UNUSED_TARGET_ORIGINS`	One or more `targetOrigins` in the session do not match the current page origin. The `details` array lists the unused origins.

Mount Errors

These errors are returned during `checkout.mount()` or `trigger.mount()`:

Mount Reason Values

Reason	Description
`CHECKOUT_ALREADY_MOUNTED`	The checkout or trigger is already mounted. Call `unmount()` first, or create a new instance.
`MOUNT_CONTAINER_SELECTOR`	The CSS selector does not match any Document Object Model (DOM) element. Check that the container exists before calling `mount()`.
`MOUNT_ERROR`	A problem occurred loading the payment iframe.
`MOUNT_INVALID_CONTAINER`	The supplied container parameter is not a valid CSS selector string or `HTMLElement`.
`MOUNT_PAYMENT_TIMEOUT`	A payment method timed out during initialization.
`MOUNT_PAYMENT_UNAVAILABLE`	No payment types could be presented to the customer. This may be due to browser or device support, or errors during checkout initialization.
`MOUNT_SIDEBAR_OPTIONS`	The supplied container parameter is invalid for sidebar mode.
`MOUNT_TOKEN_TIMEOUT`	Token creation timed out during mount. This may indicate a network issue.
`MOUNT_TOKEN_XHR_ERROR`	A network error occurred during token creation. Check the customer's connectivity.

Complete Errors

These errors are returned during `checkout.complete()` or `trigger.complete()`:

Complete Reason Values

Reason	Description
`COMPLETE_AUTHENTICATION_CANCELED`	The customer cancelled the 3-D Secure authentication step-up.
`COMPLETE_AUTHENTICATION_FAILED`	The 3-D Secure authentication step-up failed.
`COMPLETE_ERROR`	A general error occurred during transaction completion.
`COMPLETE_IN_PROGRESS`	**complete()** has already been called and has not yet finished. Wait for the current call to resolve.
`COMPLETE_NOT_ALLOWED`	Complete is not allowed for this transaction, such as when `autoProcessing` is set to `true`.
`COMPLETE_TRANSACTION_CANCELLED`	The customer cancelled the transaction.
`COMPLETE_TRANSACTION_FAILED`	The transaction failed during processing.
`COMPLETE_VALIDATION_ERROR`	The parameters supplied to **complete()** have a validation error. Check the `details` for specifics

Checkout Errors

Checkout Reason Values

Reason	Description
`CHECKOUT_ERROR`	A general checkout error occurred.
`CHECKOUT_PAYMENT_PARAMETERS`	One or more payment parameters have a validation error.
`CHECKOUT_VALIDATION_PARAMS`	One or more checkout parameters have a validation error.

Trigger Errors

Trigger Reason Values

Reason	Description
`TRIGGER_PAYMENT_TYPE_NOT_SUPPORTED`	The specified payment type cannot be used with a trigger. Only `PANENTRY` and `CLICKTOPAY` values are supported.

Payment-Specific Errors

Payment-Specific Reason Values

Reason	Description
`CLICK_TO_PAY_SDK_LOAD_ERROR`	The Click to PaySDK failed to load.
`ENCRYPT_CARD_FOR_SRC_ENROLMENT_ERROR`	Card encryption for Click to Pay enrollment failed.
`GOOGLEPAY_CHECKOUT_ERROR`	A Google Pay checkout error occurred.
`LAUNCH_SRC_CHECKOUT_ERROR`	Launching the Click to Pay checkout failed.
`TRIGGER_PAYMENT_TYPE_NOT_SUPPORTED`	The payment type is not supported for triggers.

General Errors

Reason Code	Description
`UNKNOWN_ERROR`	An unknown error has occurred.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Reason Codes

This section describes the server-side HTTP status codes and reason values that are returned when you send requests to Unified Checkout. For information about client-side SDK error codes, see Handle Errors (on page 231).

HTTP Status Codes

Code	Description
`200`	Request processed successfully
`201`	Session created
`400`	Bad request. The response body contains a reason value with details
`404`	Resource not found
`500`	Unexpected server error

Reason Values (HTTP `400`)

When the API returns a status code value of `400`, the response body includes the **reason** field. This section lists all possible values for the **reason** field:

Initialization Reason Values

Reason	Description
`CAPTURE_CONTEXT_EXPIRED`	The session JWT is expired. Generate a new session.
`CAPTURE_CONTEXT_INVALID`	The session JWT is not valid. For example, it has a bad signature or is malformed.
`INVALID_APIKEY`	The API key is not valid.

Checkout Reason Values

Reason	Description
`CHECKOUT_ERROR`	A general checkout error occurred.
`UNIFIED_PAYMENTS_ALREADY_SHOWN`	The checkout is already displayed.
`UNIFIED_PAYMENTS_PAYMENT_PARAMETERS`	One or more payment parameters have a validation error.
`UNIFIED_PAYMENTS_VALIDATION_FIELDS`	One or more fields have a validation error.
`UNIFIED_PAYMENTS_VALIDATION_PARAMS`	One or more checkout parameters have a validation error.

Mount Reason Values

Reason	Description
`SHOW_LOAD_CONTAINER_SELECTOR`	The CSS selector does not match any element.

Mount Reason Values (continued)

Reason	Description
`SHOW_LOAD_ERROR`	A problem occurred loading the payment iframe.
`SHOW_LOAD_INVALID_CONTAINER`	The container parameter is not valid.
`SHOW_LOAD_SIDEBAR_OPTIONS`	The container parameter is not valid for sidebar mode.
`SHOW_PAYMENT_TIMEOUT`	A payment method timed out during initialization.
`SHOW_PAYMENT_UNAVAILABLE`	No payment types could be presented to the customer.
`SHOW_TOKEN_TIMEOUT`	Token creation timed out during mount.
`SHOW_TOKEN_XHR_ERROR`	A network error occurred during token creation.

Complete Reason Values

Reason	Description
`COMPLETE_AUTHENTICATION_CANCELLED`	The customer cancelled 3-D Secure authentication.
`COMPLETE_AUTHENTICATION_FAILED`	3-D Secure authentication failed.
`COMPLETE_ERROR`	A general error occurred during completion.
`COMPLETE_IN_PROGRESS`	A complete call is already in progress.
`COMPLETE_NOT_ALLOWED`	Complete is not allowed, such as when auto-processing is enabled.
`COMPLETE_TRANSACTION_CANCELLED`	The customer cancelled the transaction.
`COMPLETE_TRANSACTION_FAILED`	The transaction failed during processing.
`COMPLETE_VALIDATION_ERROR`	Parameters supplied to complete have a validation error.

Tokenization Reason Values

Reason	Description
`CREATE_TOKEN_TIMEOUT`	Token creation request timed out.
`CREATE_TOKEN_XHR_ERROR`	A network error occurred during token creation.
`SDK_XHR_ERROR`	A general SDK network error occurred.
`TOKENIZATION_ERROR`	Tokenization of payment data failed.

Payment-Specific Reason Values

Reason	Description
`CLICK_TO_PAY_SDK_LOAD_ERROR`	The Click to PaySDK failed to load.
`ENCRYPT_CARD_FOR_SRC_ENROLMENT_ERROR`	Card encryption for Click to Pay enrollment failed.
`GOOGLEPAY_CHECKOUT_ERROR`	A Google Pay checkout error occurred.
`LAUNCH_SRC_CHECKOUT_ERROR`	Launching the Click to Pay checkout failed.
`TRIGGER_PAYMENT_TYPE_NOT_SUPPORTED`	The payment type is not supported for triggers.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Payment Details API

This section contains the information you need to retrieve the non-sensitive data associated with a Unified Checkout transient token and the Payment Details API. This API can be used to retrieve personally identifiable information, such as the cardholder name and billing and shipping details, without retrieving payment credentials, which helps ease the PCI compliance burden.

There are two methods of authentication, and they are described in the *Getting Started with REST Developer Guide*:

- Set Up a JSON Web Token Message
- Set Up HTTP Signature Message

Important:

Cybersource recommends that you dynamically parse the response for the fields that you are looking for when you integrate with Cybersource APIs. Cybersource may add additional fields in the future.

You must ensure that your integration can handle new fields that are returned in the response. Even though the underlying data structures do not change, you must also ensure that your integration can handle changes to the order in which the data is returned. Cybersource uses semantic versioning practices, which enables you to retain backwards compatibility as new fields are introduced in minor version updates.

Endpoint

Production: `GET https://api.cybersource.com/flex/v2/payment-details/{jti}`

Test: `GET https://apitest.cybersource.com/flex/v2/payment-details/{jti}`

Production in Saudi Arabia: `GET https://api.sa.cybersource.com/flex/v2/payment-details/{jti}`

Test in Saudi Arabia: `GET https://apitest.sa.cybersource.com/flex/v2/payment-details/{jti}`

The `{jti}` is the ID of the JWT within the transient token that is returned by Unified Checkout. The transient token is a JWT object that you retrieved as part of a successful capture of payment information from a cardholder.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

REST Example: Retrieving Transient Token Payment Details

Request

```
GET https://apitest.cybersource.com/flex/v2/payment-details/{jti}
```

Response to Successful Request

```
{
  "paymentInformation": {
    "card": {
      "expirationYear": "2026",
      "number": "XXXXXXXXXXXX1111",
      "expirationMonth": "05",
      "type": "001"
    }
  },
  "orderInformation": {
    "amountDetails": {
      "totalAmount": "21.00",
      "currency": "USD"
    },
    "billTo": {
      "lastName": "Lee",
      "country": "US",
      "firstName": "Tanya",
      "email": "tanyalee@example.com"
    },
    "shipTo": {
      "locality": "Small Town",
      "country": "US",
      "administrativeArea": "CA",
      "address1": "123 Main Street",
```

```
      "postalCode": "98765"
    }
  }
}
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

JavaScript API Reference

This reference provides details about the JavaScript API for creating the Unified Checkout v1 payment form.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

VAS.UnifiedCheckout(sessionJWT)

This is a factory function that initializes the SDK. It returns a frozen, immutable client interface.

VAS.UnifiedCheckout(sessionJWT) Parameters

Name	Type	Required?	Description
`sessionJWT`	`string`	Yes	Signed JSON Web Token (JWT) from the server-side session endpoint

Returns

`Promise<UnifiedCheckoutInterface>`

Errors

Returns `UnifiedCheckoutError` with reason `CAPTURE_CONTEXT_INVALID` if the JWT signature is invalid, or `UNUSED_TARGET_ORIGINS` if the current page origin is not in the JWT's `targetOrigins` list.

Example

```
const client = await VAS.UnifiedCheckout(sessionJWT);
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

UnifiedCheckoutInterface

The client object returned by `VAS.UnifiedCheckout()`. All methods throw an `Error` if called after `destroy()`.

client.createCheckout(options?)

client.createCheckout(options?) Parameters

Name	Type	Required?	Description
`options`	`CreateCheckoutOptions`	No	Configuration for the checkout

`CreateCheckoutOptions` Properties

Property	Type	Default	Description
`autoProcessing`	`boolean`	Inferred from capture context.	• `true`: `mount()` returns completed payment result. Defaults to `true` when **completeMandate** is included in the capture context. • `false`: `mount()` returns transient token.

Returns

`Promise<Checkout>`

Example

```
const checkout = await client.createCheckout({ autoProcessing:
 false });
```

client.createTrigger(paymentType, options?)

client.createTrigger(paymentType, options?) Parameters

Name	Type	Required?	Description
`paymentType`	`AllowedPaymentType`	Yes	Only `PANENTRY` and `CLICKTOPAY` are supported.
`options`	`CreateTriggerOptions`	No	The configuration for the trigger.

`CreateTriggerOptions` Properties

Property	Type	Default	Description
`autoProcessing`	`boolean`	Inferred from session	Same as checkout `autoProcessing`

Returns

`Trigger`

Errors

Returns `UnifiedCheckoutError` with reason `TRIGGER_PAYMENT_TYPE_NOT_SUPPORTED` when the payment type cannot be used with a trigger.

Example

```
const trigger = client.createTrigger('PANENTRY');
```

client.createButton(paymentType, options?) (Experimental)

Creates an individual payment method button.

client.createButton(paymentType, options?) Parameters

Name	Type	Required?	Description
`paymentType`	`AllowedPaymentType`	Yes	Payment type for the button. For example, `GOOGLEPAY` and `APPLEPAY`.
`options`	`CreateButtonOptions`	No	The configuration for the button.

`CreateButtonOptions` Properties

Property	Type	Default	Description
`autoProcessing`	`boolean`	Inferred from session	Same as checkout `autoProcessing`

Returns

`PaymentButton`

Example

```
const button = client.createButton('GOOGLEPAY');
```

client.on(event, callback)

Subscribes to a client-level event and returns an unsubscribe function.

client.on(event, callback) Parameters

Name	Type	Required?	Description
`event`	`string`	Yes	Event name. Possible values: • `*` • `created` • `destroyed` • `error`
`callback`	`function`	Yes	Handler function that receives event-specific payload.

`CreateButtonOptions` Properties

Property	Type	Default	Description
`autoProcessing`	`boolean`	Inferred from session	Same as checkout `autoProcessing`

Returns

`Unsubscribe`: A function that removes the handler when called.

Errors

Returns `Error` when `event` is not a valid event name with reason `TRIGGER_PAYMENT_TYPE_NOT_SUPPORTED` when the payment type cannot be used with a trigger.

Example

```
const unsubscribe = client.on('error', (err) => {
  console.error(err.source, err.code, err.message);
});

// Later
unsubscribe();
```

client.off(event, callback?)

Removes an event handler. This method is permissive — calling it with an unknown event or callback does not throw.

client.off(event, callback?) Parameters

Name	Type	Required?	Description
`event`	`string`	Yes	Event name to unsubscribe from
`callback`	`function`	No	Specific handler to remove. When this is not included, all handlers for the event are removed.

client.destroy()

Permanently destroys the client. Returns a `destroyed` event, clears all event listeners, and marks the instance as destroyed.

You can call `destroy()` multiple times.

client.isDestroyed()

Returns a value of `true` if **client.destroy()** is called.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Checkout

This field is returned by `client.createCheckout()` and manages the full checkout UI lifecycle.

checkout.mount(target)

Subscribes to a client-level event and returns an unsubscribe function.

checkout.mount(target) Parameters

Name	Type	Required?	Description
`target`	`string` or `CheckoutContainers`	No	CSS selector string for sidebar mode, or an object with `paymentSelection` and `paymentScreen` for embedded mode. Omit for full sidebar

`CheckoutContainers` Properties

Property	Type	Default	Description
`paymentSelection`	`string`	Yes	CSS selector for the button list container
`paymentScreen`	`string`	No	CSS selector for the payment form container. If omitted,

`CheckoutContainers` Properties (continued)

Property	Type	Default	Description
			payment screens appear in sidebar mode

Returns

`Promise<string>`: This is a transient token JWT when `autoProcessing: false` or completed payment result JWT when `autoProcessing: true`.

Errors

Returns `UnifiedCheckoutError`. For information about how to handle mount error codes, see Handle Errors (on page 231).

Example

```
// Sidebar
const result = await checkout.mount('#buttons');

// Embedded
const result = await checkout.mount({
  paymentSelection: '#buttons',
  paymentScreen: '#form'
});
```

checkout.unmount()

Removes the payment UI from the page. The checkout is not destroyed — you can call `mount()` again.

checkout.complete(transientToken)

Manually completes the payment flow. This is only available when `autoProcessing` is set to `false`.

checkout.complete(transientToken) Parameters

Name	Type	Required?	Description
`transientToken`	`string`	Yes	The transient token JWT that is returned by `mount()`.

`CheckoutContainers` **Properties**

Property	Type	Default	Description
`paymentSelection`	`string`	Yes	CSS selector for the button list container
`paymentScreen`	`string`	No	CSS selector for the payment form container. If omitted, payment screens appear in sidebar mode

Returns

`Promise<string>`: This is the completed payment result JWT.

Errors

Returns `UnifiedCheckoutError`. For information about how to handle mount error codes, see Handle Errors (on page 231).

Example

```
const token = await checkout.mount('#buttons');
const result = await checkout.complete(token);
```

checkout.isMounted()

Returns `true` when the checkout UI is mounted.

checkout.isDestroyed()

Returns `true` when `destroy()` is called.

checkout.on(event, handler)

Subscribes to a checkout-level event and returns an unsubscribe function.

Valid events:

- `mounted`
- `ready`
- `unready`

- `unmounted`
- `destroyed`
- `paymentMethodSelected`
- `paymentMethodCancelled`
- `paymentMethodUpdate"`
- `error`
- `*`

checkout.off(event, handler?)

Removes a checkout event handler.

checkout.destroy()

Permanently destroys the checkout. This field removes the payment UI, cleans up iframes, and emits a `destroyed` event.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Trigger

The trigger is returned by `client.createTrigger()` and programmatically launches a specific payment method.

trigger.mount(target?)

Launches the payment method UI.

trigger.mount(target?) Parameters

Name	Type	Required?	Description
`target`	`string`	No	CSS selector for embedded mode. Omit for sidebar mode.

Returns

`Promise<string>`: A transient token or completed payment result.

Example

```
const result = await trigger.mount('#payment-screen');
```

trigger.unmount()

Hides the payment method UI. The trigger is not destroyed.

trigger.complete(transientToken)

Manually completes the payment. Same interface as `checkout.complete()`.

trigger.isMounted()

Returns a boolean value.

trigger.isDestroyed()

Returns a boolean value.

trigger.on(event, handler)

Subscribes to trigger events. Same event names and payloads as checkout events.

trigger.off(event, handler?)

Removes a trigger event handler.

trigger.destroy()

Permanently destroys the trigger.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

PaymentButton

This is returned by `client.createButton()`. It renders an individual payment method button.

button.mount(container)

Mounts the button into a container.

button.mount(container) Parameters

Name	Type	Required?	Description
`container`	`string` or `HTMLElement`	Yes	CSS selector string or Document Object Model (DOM) element for the button container

Returns

`Promise<string>`: A transient token or completed payment result.

button.unmount()

Removes the button from the page. The button is not destroyed.

button.isMounted()

Returns a boolean value if the button is mounted (`true`) or not mounted (`false`).

button.isDestroyed()

Returns a boolean value if the button is destroyed (`true`) or not destroyed (`false`).

button.on(event, callback)

Subscribes to button events. Returns `void`. Use `button.off()` to remove handlers.

> **Important:** The button event system is under development. Event names and payloads may change in future releases.

button.off(event, callback?)

Removes a button event handler.

button.destroy()

Permanently destroys the button. Idempotent.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Events

Unified Checkout provides a type-safe event system for monitoring the payment lifecycle. Events are emitted at the client and integration levels.

Subscribe to Events

Use `on()` to subscribe to events. this returns an unsubscribe function:

```
const unsubscribe = checkout.on('ready', (data) => {
  console.log('Ready:', data.availablePaymentMethods);
});

// Later, remove the handler
unsubscribe();
```

You can use `off()` to remove a specific handler:

```
function onReady(data) { /* ... */ }

checkout.on('ready', onReady);
checkout.off('ready', onReady);
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Update to Unified Checkout Version 1

The version 1 (v1) SDK simplifies your integration with fewer lines of code, a streamlined API, and enhancements such as auto-processing and a full event system. The core flow is the same in v1, and migrating to v1 involves only straightforward method renames.

Summary of Changes

Summary of Version 1 Changes

Aspect	v0	v1
Initialization	`new Accept(session).unifiedPayments()`	`VAS.UnifiedCheckout(session)`
Display the payment UI	`up.show(options)`	`checkout.mount(target)`
Completing transactions	`up.complete(token)`	`checkout.complete(token)` or automatic using `autoProcessing`
Events	None	Full event system on client and checkout
Cleanup	`up.dispose()`	`checkout.destroy()` + `client.destroy()`
Hide UI	`up.hide()`	`checkout.unmount()`
Target Origin (Sessions API)	Multiple non-usable URLs can be included in the request.	If any origins are absent or mismatched, for example, they are not presented in Unified Checkout, the system prevents Unified Checkout from loading and displays a client-side error message.
consumerAuthentication within the Complete Mandate	**Data type:** Boolean	**Data type:** Enum Possible values: • `3DS` • `NONE` For information about **consumerAuthentication**, see completeMandate.consumerAuthentication (on page 190).

Summary of v0 and v1 Changes

Feature	Pre V1 Support	V1 Support	Description
Status	✓	Business Center	
Capture context endpoint	`/up/v1/capture-contexts`	`/uc/v1/sessions`	
Capture context management	API only	API only or API and Business Center	Business Center configuration is at the merchant level.
Unified Checkout Look and Feel in Business Center	✗	✓	Business Center configuration is at the merchant level.
Unified Checkout Look and Feel Using the API	✗	✓	Configure the look and feel in a Sessions API request.
Payment methods	API only	API or API and Business Center	Business Center configuration is at the merchant level.
Real-time preview in Business Center	✗	✓	Business Center configuration is at the merchant level.
Three-decimal currency support	✗	✓	
SDK	Legacy Unified Payments SDK supported (link)	New UC SDK (link)	
Payment Details API	`/up/v1/payment-details/{id}`	JTI used in place of transient token	JTI is located in the transient token
Future enhancements	Manual opt-in is required.	Automatic when the **clientVersion** is not included in the Sessions API request.	Legacy versions receive critical updates only.

Initialization

Initialization with Unified Checkout v1 is done in a single asynchronous factory call. There is no intermediate `Accept` object:

v0 Initialization

```
const accept = new Accept(sessionJWT);
const up = accept.unifiedPayments();
```

v1 Initialization

```
const client = await VAS.UnifiedCheckout(sessionJWT);
```

Unified Checkout v1 validates the JWT signature and target origins during initialization.

Display the Payment UI

Unified Checkout v1 passes your UI payment selectors directly to `mount()`.

When `autoProcessing` is enabled, `mount()` returns the completed payment result rather than a transient token.

v0 Display Payment UI with `show()`

```
// Sidebar
const token = await up.show({
  containers: {
    paymentSelection: '#buttons'
  }
});

// Embedded
const token = await up.show({
  containers: {
    paymentSelection: '#buttons',
    paymentScreen: '#form'
  }
});
```

v1 Display Payment UI with `mount()`

```
// Sidebar
const result = await checkout.mount('#buttons');

// Embedded
```

```
const result = await checkout.mount({
  paymentSelection: '#buttons',
  paymentScreen: '#form'
});
```

Completing Transactions

When `autoProcessing` is enabled in Unified Checkout v1, `mount()` returns the completed payment result and you do not need to send a separate **complete()** request. When `autoProcessing` is disabled in Unified Checkout v1, you must complete transactions manually.

v0 Complete Transactions Manually

```
const token = await up.show({
  containers: {
    paymentSelection: '#buttons'
  }
});
const result = await up.complete(token);
```

v1 Complete Transactions Manually or Automatically

```
// Automatic (default when completeMandate is in session)
const checkout = await client.createCheckout({ autoProcessing: true });
const result = await checkout.mount('#buttons');
// result is the completed transaction — no need to call complete()

// Manual - similar to v0
const checkout = await client.createCheckout({ autoProcessing: false });
const token = await checkout.mount('#buttons');
const result = await checkout.complete(token);
```

Events

Unified Checkout v0 does not include an event system, as the integration resolution or rejection from `show()` and **complete()**. v1 includes a full event system as the client and integration levels.

v1 Full Event System

```
// Client-level — centralized error tracking
client.on('error', (err) => {
  console.error(`[${err.source}] ${err.code}: ${err.message}`);
});

// Checkout-level — granular lifecycle events
checkout.on('ready', (data) => {
  console.log('Available methods:', data.availablePaymentMethods);
});

checkout.on('paymentMethodSelected', (data) => {
  console.log('Selected:', data.type);
});

checkout.on('error', (err) => {
  console.error('Checkout error:', err.code);
});
```

Cleanup

Unified Checkout v1 distinguishes between `unmount()`, which is reversible, and `destroy()`, which is permanent. Before a cleanup, `client.destroy()` sends a `destroyed` event.

v0 Cleanup

```
up.hide();     // Hide UI
up.dispose();  // Clean up resources
```

v1 Cleanup

```
checkout.unmount();   // Remove UI from page (can remount later)
checkout.destroy();   // Permanent cleanup

client.destroy();     // Destroy client and clear all event listeners
```

Handle Errors

The `UnifiedCheckoutError` class and its reason codes are the same in v0 and v1:

v0 Error Handling

```
try {
  const token = await up.show({
    containers: {
      paymentSelection: '#buttons'
    }
  });
} catch (err) {
  console.error(err.reason, err.message);
}
```

v1 Error Handling

```
// Same error class, same properties
try {
  const result = await checkout.mount('#buttons');
} catch (err) {
  console.error(err.reason, err.message);
}
```

Migrate Triggers

If your Unified Checkout v0 integration uses triggers, the migration is similar to checkout. In v1, triggers are created from the `client.createTrigger`, not from `UnifiedPayments` as in v0. In v1, `show()` is renamed to `mount()`.

v0 Triggers

```
const trigger = up.createTrigger('CLICKTOPAY', {
  containers: { paymentScreen: '#screen' }
});
const token = await trigger.show();
```

v1 Triggers

```
const trigger = client.createTrigger('CLICKTOPAY');
const result = await trigger.mount('#screen');
```

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Update Reason Codes

Some reason codes were renamed in v1. This table shows the v0 reason code name and the corresponding name in the v1 client-side SDK:

v0 Reason Code	v1 Reason Code
`SHOW_LOAD_CONTAINER_SELECTOR`	`MOUNT_CONTAINER_SELECTOR`
`SHOW_LOAD_ERROR`	`MOUNT_ERROR`
`SHOW_LOAD_INVALID_CONTAINER`	`MOUNT_INVALID_CONTAINER`
`SHOW_LOAD_SIDEBAR_OPTIONS`	`MOUNT_SIDEBAR_OPTIONS`
`SHOW_PAYMENT_TIMEOUT`	`MOUNT_PAYMENT_TIMEOUT`
`SHOW_PAYMENT_UNAVAILABLE`	`MOUNT_PAYMENT_UNAVAILABLE`
`SHOW_TOKEN_TIMEOUT`	`MOUNT_TOKEN_TIMEOUT`
`SHOW_TOKEN_XHR_ERROR`	`MOUNT_TOKEN_XHR_ERROR`
`UNIFIED_PAYMENTS_ALREADY_SHOWN`	`CHECKOUT_ALREADY_MOUNTED`
`UNIFIED_PAYMENTS_PAYMENT_PARAMETERS`	`CHECKOUT_PAYMENT_PARAMETERS`
`UNIFIED_PAYMENTS_VALIDATION_PARAMS`	`CHECKOUT_VALIDATION_PARAMS`

Important: The server-side API continues to return these v0 reason codes. The v1 reason codes listed here are used only in the client-side SDK. For all v1 client-side error codes, see Handle Errors (on page 231).

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

Version 1 Update Checklist

You must complete these tasks before you can complete your migration from Unified Checkout v0 to v1:

- Replace `new Accept(session).unifiedPayments()` with `await VAS.UnifiedCheckout(session)`.
- Replace `up.show(options)` with `checkout = await client.createCheckout(); checkout.mount(target)`.
- Update container options: `{ containers: { paymentSelection, paymentScreen } }` becomes direct arguments to `mount()`.
- Replace `up.complete(token)` with `checkout.complete(token)` or use `autoProcessing: true` to complete transactions automatically
- Replace `up.hide()` with `checkout.unmount()`.
- Replace `up.dispose()` with `checkout.destroy()` and `client.destroy()`.
- Add event listeners for observability. For example, `client.on('error')` and `checkout.on('ready')`.

Related information
Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Appendix

This section contains supplementary information for Unified Checkout.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

JSON Web Tokens

JSON Web Tokens (JWTs) are digitally signed JSON objects based on the open standard RFC 7519. These tokens provide a compact, self-contained method for securely transmitting information between parties. These tokens are signed with an RSA-encoded public/private key pair. The signature is calculated using the header and body, which enables the receiver to validate that the content has not been tampered with.

A JWT takes the form of a string, and consists of three parts separated by dots:

```
<Header>.<Payload>.<Signature>
```

The header and payload is **Base64-encoded JSON** and contains these claims:

- **Header**: The algorithm and token type. For example:

  ```
  {
    "kid": "zu",
    "alg": "RS256"
  }
  ```

- **Payload**: The claims of what the token represents. For example:

  ```
  {
    "sub": "1234567890",
    "name": "John Doe",
    "iat": 1516239022
  }
  ```

- **Signature**: The signature is computed from the header and payload using a secret or private key.

Important: When working with JWTs, Cybersource recommends that you use a well-maintained JWT library to ensure proper decoding and parsing of the JWT.

Important: When parsing the JWT's JSON payload, you must ensure that you implement a robust solution for transversing JSON. Additional elements can be added to the JSON in future releases. Follow JSON parsing best practices to ensure that you can handle the addition of new data elements in the future.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Supported Countries for Digital Payments

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Supported Countries for Digital Payments A-D

Supported Countries (A through D)

Country	Apple Pay	Click to Pay	eCheck	Google Pay
Afghanistan	✗	✗	✗	✓
Albania	✗	✗	✗	✓
Algeria	✗	✗	✗	✓
Andorra	✗	✗	✗	✓
Angola	✗	✗	✗	✓
Antigua and Barbuda	✗	✗	✗	✓
Argentina	✓	✓	✗	✓
Armenia	✓	✗	✗	✓
Australia	✓	✓	✗	✓
Austria	✓	✓	✗	✓
Azerbaijan	✓	✗	✗	✓
Bahamas	✗	✗	✗	✓
Bahrain	✓	✗	✗	✓
Bangladesh	✗	✗	✗	✓

Supported Countries (A through D) (continued)

Country	Apple Pay	Click to Pay	eCheck	Google Pay
Barbados	✗	✗	✗	✓
Belarus	✓	✗	✗	✓
Belgium	✓	✗	✗	✓
Brazil	✓	✓	✗	✓
Belize	✗	✗	✗	✓
Benin	✗	✗	✗	✓
Bhutan	✗	✗	✗	✗
Bolivia	✗	✗	✗	✓
Bosnia and Herzegovina	✗	✗	✗	✓
Botswana	✗	✗	✗	✓
Brunei Dar ussalam	✗	✗	✗	✓
Bulgaria	✓	✓	✗	✗
Burkina Faso	✗	✗	✗	✓
Burundi	✗	✗	✗	✓
Cambodia	✗	✗	✗	✓
Cameroon	✗	✗	✗	✓
Canada	✓	✓	✓	✓
Cape Verde	✗	✗	✗	✓
Central African Republic	✗	✗	✗	✓
Chad	✗	✗	✗	✓
Chile	✓	✓	✗	✓
China	✓	✓	✗	✗

Supported Countries (A through D) (continued)

Country	Apple Pay	Click to Pay	eCheck	Google Pay
Colombia	✓	✓	✗	✓
Comoros	✗	✗	✗	✓
Costa Rica	✓	✓	✗	✓
Côte d'Ivoire	✗	✗	✗	✓
Croatia	✓	✗	✗	✓
Cyprus	✓	✗	✗	✓
Czech Republic	✓	✓	✗	✓
Democratic Republic of the Congo	✗	✗	✗	✓
Denmark	✓	✓	✗	✓
Djibouti	✗	✗	✗	✓
Dominica	✗	✗	✗	✓
Dominican Republic	✗	✓	✗	✓

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Supported Countries for Digital Payments E-K

Supported Countries (E through K)

Country	Apple Pay	Click to Pay	Google Pay
Ecuador	✗	✓	✓

Supported Countries (E through K) (continued)

Country	Apple Pay	Click to Pay	Google Pay
Egypt	✗	✗	✓
El Salvador	✓	✓	✓
Equatorial Guinea	✗	✗	✓
Eritrea	✗	✗	✓
Estonia	✓	✗	✓
Eswatini	✗	✗	✓
Ethiopia	✗	✗	✓
Faroe Islands	✓	✗	✗
Fiji	✗	✗	✓
Finland	✓	✓	✓
France	✓	✓	✓
Gabon	✗	✗	✓
Gambia	✗	✗	✓
Georgia	✓	✗	✓
Germany	✓	✓	✓
Ghana	✗	✗	✓
Gibraltar	✗	✗	✗
Greece	✓	✓	✓
Greenland	✓	✗	✗
Guernsey	✓	✗	✗
Grenada	✗	✗	✓
Guatemala	✓	✓	✓
Guinea	✗	✗	✓

Supported Countries (E through K) (continued)

Country	Apple Pay	Click to Pay	Google Pay
Guinea-Bissau	✗	✗	✓
Guyana	✗	✗	✓
Haiti	✗	✗	✓
Honduras	✓	✓	✓
Hong Kong	✓	✓	✓
Hungary	✓	✓	✓
Iceland	✓	✗	✓
Indonesia	✗	✓	✗
Iraq	✗	✗	✓
Ireland	✓	✓	✓
Isle of Man	✓	✗	✗
Israel	✓	✗	✓
Italy	✓	✓	✓
Jamaica	✗	✗	✓
Japan	✓	✓	✓
Jersey	✓	✗	✗
Jordan	✓	✓	✓
Kazakhstan	✓	✗	✓
Kenya	✗	✗	✓
Kiribati	✗	✗	✓
Kuwait	✓	✓	✓
Kyrgyzstan	✗	✗	✓

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Supported Countries for Digital Payments L-R

Supported Countries (L through R)

Country	Apple Pay	Click to Pay	Google Pay
Laos	✗	✗	✓
Latvia	✓	✗	✓
Lebanon	✗	✗	✓
Lesotho	✗	✗	✓
Liberia	✗	✗	✓
Libya	✗	✗	✓
Liechtenstein	✓	✗	✓
Lithuania	✓	✗	✓
Luxembourg	✓	✗	✓
Macau	✓	✗	✓
Madagascar	✗	✗	✓
Malawi	✗	✗	✓
Malaysia	✓	✓	✓
Maldives	✗	✗	✓
Mali	✗	✗	✓
Malta	✓	✗	✓
Marshall Islands	✗	✗	✓
Mauritania	✗	✗	✓
Mauritius	✗	✗	✓
Mexico	✓	✓	✓
Micronesia, Federated States of	✗	✗	✓
Moldova	✓	✗	✓

Supported Countries (L through R) (continued)

Country	Apple Pay	Click to Pay	Google Pay
Monaco	✓	✗	✓
Mongolia	✗	✗	✓
Montenegro	✓	✗	✓
Morocco	✗	✗	✓
Mozambique	✗	✗	✓
Myanmar	✗	✗	✓
Namibia	✗	✗	✓
Nauru	✗	✗	✓
Nepal	✗	✗	✓
Netherlands	✓	✓	✓
New Zealand	✓	✓	✓
Nicaragua	✗	✓	✓
Niger	✗	✗	✓
Nigeria	✗	✗	✓
North Macedonia	✗	✗	✓
Norway	✓	✓	✓
Oman	✗	✗	✓
Pakistan	✗	✗	✓
Palau	✗	✗	✓
Palestinian Territories	✓	✗	✓
Panama	✓	✓	✓
Papua New Guinea	✗	✗	✓
Paraguay	✗	✓	✓

Supported Countries (L through R) (continued)

Country	Apple Pay	Click to Pay	Google Pay
Peru	✓	✓	✓
Philippines	✗	✗	✗
Poland	✓	✓	✓
Portugal	✓	✗	✓
Qatar	✓	✓	✓
Republic of the Congo	✗	✗	✓
Romania	✓	✓	✓
Rwanda	✗	✗	✓

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Supported Countries for Digital Payments S-Z

Supported Countries (S through Z)

Country	Apple Pay	Click to Pay	eCheck	Google Pay	Paze
Saint Kitts and Nevis	✗	✗	✗	✓	✗
Saint Lucia	✗	✗	✗	✓	✗
Saint Vin cent and the Grenadines	✗	✗	✗	✓	✗
Samoa	✗	✗	✗	✓	✗
San Marino	✓	✗	✗	✓	✗
Sao Tome and Principe	✗	✗	✗	✓	✗
Saudi Arabia	✓	✓	✗	✓	✗
Senegal	✗	✗	✗	✓	✗
Serbia	✓	✗	✗	✓	✗
Seychelles	✗	✗	✗	✓	✗
Sierra Leone	✗	✗	✗	✓	✗
Singapore	✓	✓	✗	✓	✗
Slovakia	✓	✓	✗	✓	✗
Slovenia	✓	✓	✗	✓	✗
Solomon Islands	✗	✗	✗	✓	✗
Somalia	✗	✗	✗	✓	✗
South Africa	✓	✓	✗	✓	✗
Korea, Republic of (South)	✓	✗	✗	✓	✗
South Sudan	✗	✗	✗	✓	✗

Supported Countries (S through Z) (continued)

Country	Apple Pay	Click to Pay	eCheck	Google Pay	Paze
Spain	✓	✓	✗	✓	✗
Sri Lanka	✗	✓	✗	✓	✗
Sudan	✗	✗	✗	✓	✗
Suriname	✗	✗	✗	✓	✗
Sweden	✓	✓	✗	✓	✗
Switzerland	✓	✓	✗	✓	✗
Switzerland -Italian	✗	✗	✗	✗	✗
Taiwan	✓	✗	✗	✓	✗
Tajikistan	✗	✗	✗	✓	✗
Tanzania	✗	✗	✗	✓	✗
Thailand	✗	✓	✗	✗	✗
Timor-Leste	✗	✗	✗	✓	✗
Togo	✗	✗	✗	✓	✗
Tonga	✗	✗	✗	✓	✗
Trinidad and Tobago	✗	✗	✗	✓	✗
Tunisia	✗	✗	✗	✓	✗
Turkey	✗	✗	✗	✓	✗
Turkmenistan	✗	✗	✗	✓	✗
Tuvalu	✗	✗	✗	✓	✗
Uganda	✗	✗	✗	✓	✗
Ukraine	✓	✓	✗	✓	✗
United Arab Emirates	✓	✓	✗	✓	✗

Supported Countries (S through Z) (continued)

Country	Apple Pay	Click to Pay	eCheck	Google Pay	Paze
United Kingdom	✓	✓	✗	✓	✗
United States	✓	✓	✓	✓	✓
Uruguay	✗	✓	✗	✓	✗
Uzbekistan	✗	✗	✗	✓	✗
Vanuatu	✗	✗	✗	✓	✗
Vatican City (Holy See)	✓	✗	✗	✓	✗
Venezuela	✗	✗	✗	✓	✗
Vietnam	✓	✓	✗	✓	✗
Yemen	✗	✗	✗	✓	✗
Zambia	✗	✗	✗	✓	✗
Zimbabwe	✗	✗	✗	✓	✗

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Supported Locales

The locale field within the capture context request consists of an ISO 639 language code, an underscore (_), and an ISO 3166 region code. Set the **locale** field in your session request to display the checkout UI in the customer's language.

Important: When the chosen language is Arabic, Unified Checkout supports right-to-left languages in the UI for this locale.

Unified Checkout supports these locales:

Supported Locales

Locale	ISO Language	ISO Region
`ar_AE`	Arabic	United Arab Emirates
`ar_JO`	Arabic	Jordan
`ar_QA`	Arabic	Qatar
`bg_BG`	Bulgarian	Bulgaria
`ca_ES`	Catalan	Spain
`zh_CN`	Chinese	China
`zh_HK`	Chinese	Hong Kong
`zh_MO`	Chinese	Macao
`zh_SG`	Chinese	Singapore
`zh_TW`	Chinese	Taiwan
`hr_HR`	Croatian	Croatia
`cs_CZ`	Czech	Czechia
`da_DK`	Danish	Denmark
`nl_NL`	Dutch	Netherlands
`en_AE`	English	United Arab Emirates
`en_AU`	English	Australia
`en_CA`	English	Canada
`en_GB`	English	United Kingdom
`en_IE`	English	Ireland
`en_NZ`	English	New Zealand
`en_PK`	English	Pakistan

Supported Locales (continued)

Locale	ISO Language	ISO Region
en_QA	English	Qatar
en_US	English	United States
en_LK	English	Sri Lanka
fi_FI	Finnish	Finland
fr_CA	French	Canada
fr_FR	French	France
de_AT	German	Austria
de_DE	German	Germany
el_GR	Greek	Greece
he_IL	Hebrew	Israel
hu_HU	Hungarian	Hungary
id_ID	Indonesian	Indonesia
it_IT	Italian	Italy
ja_JP	Japanese	Japan
km_KH	Khmer	Cambodia
ko_KR	Korean	South Korea
lo_LA	Lao	Laos
ms_MY	Malay	Malaysia
nb_NO	Norwegian Bokmål	Norway
pl_PL	Polish	Poland
pt_BR	Portuguese	Brazil
ro_RO	Romanian	Romania
ru_RU	Russian	Russia
si_LK	Sinhala	Sri Lanka
sk_SK	Slovak	Slovakia
sl_SI	Slovenian	Slovenia
es_AR	Spanish	Argentina
es_CL	Spanish	Chile
es_CO	Spanish	Colombia

Supported Locales (continued)

Locale	ISO Language	ISO Region
es_ES	Spanish	Spain
es_MX	Spanish	Mexico
es_PE	Spanish	Peru
es_US	Spanish	United States
sv_SE	Swedish	Sweden
tl_PH	Tagalog	Philippines
ta_LK	Tamil	Sri Lanka
th_TH	Thai	Thailand
tr_TR	Turkish	Türkiye
uk_UA	Ukrainian	Ukraine
ur_PK	Urdu	Pakistan
vi_VN	Vietnamese	Vietnam

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Security Recommendations

Unified Checkout is compliant with Payment Card Industry (PCI) Self-Assessment Questionnaire A (SAQ-A). Cybersource recommends that you consider these security policies so you can maintain a secure integration.

Content Security Policy

Implement a Content Security Policy (CSP) to mitigate cross-site scripting (XSS) attacks. Add these directives for Unified Checkout:

CSP Directives

Directive	Test	Production
`connect-src`	https://apitest.cybersource.com	https://api.cybersource.com
`frame-src`	https://apitest.cybersource.com	https://api.cybersource.com
`child-src`	https://apitest.cybersource.com	https://api.cybersource.com
`script-src`	https://apitest.cybersource.com	https://api.cybersource.com

These directives enable the SDK to load secure iframes and communicate with Cybersource services.

Important: When you use additional payment methods such as Google Pay or PayPal, you must also add their respective domains to your CSP directives.

Iframe Isolation

Unified Checkout renders all payment UI inside cross-origin iframes hosted by Cybersource. This architecture provides several security benefits:

- **Data isolation**: Your page cannot access payment data within the iframe due to the browser's same-origin policy.
- **Reduced attack surface**: Attackers cannot extract card data from the isolated iframe if your merchant page is compromised.
- **Origin verification**: The SDK validates that the hosting page origin matches the `targetOrigins` that is declared in the session before displaying any UI.

Do not attempt to access or manipulate the contents of the payment iframes. The browser blocks cross-origin access by design.

Token Security

A session is a signed JWT with a short lifespan. Cybersource recommends that you follow these practices:

- Generate a new session for each checkout. Do not reuse sessions across checkouts or customers.
- Keep the session server-side until needed. Pass it to the client only when the customer is ready to pay.
- Set `targetOrigins` to only the domains that host the SDK. Do not use wildcard origins or include domains that do not need access.

The transient token returned by `mount()` or **complete()** expires after 15 minutes. Cybersource recommends that you follow these practices:

- Send the transient token to your server immediately after receiving it.
- Verify the token signature using the public key from the session before authorizing the payment. For verification details, see Transient Tokens (on page 214).
- Do not store transient tokens in browser storage (`localStorage`, `sessionStorage`, or cookies). Process them server-side and discard.

Immutable API

The client interface that is returned by `VAS.UnifiedCheckout()` is frozen with `Object.freeze()`. This prevents runtime tampering, which means that no properties can be added, removed, or modified on the client, checkout, trigger, or button objects. Do not attempt to modify or extend the SDK objects. If you need custom behavior, use the event system to react to SDK state changes.

Cleanup

You must always call `destroy()` on the client when the payment flow is complete or the customer navigates away. This removes all iframes and clears internal state:

```
checkout.destroy();
client.destroy();
```

If you do not destroy the client, you could leave payment iframes in the page after they are no longer needed.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

PCI Compliance

The least burdensome level of Payment Card Industry (PCI) compliance is Self-Assessment Questionnaire A (SAQ-A). To be compliant with SAQ-A, you must securely capture sensitive payment data with a validated payment provider. Unified Checkout meets this requirement by rendering secure iframes hosted by Cybersource. Payment data is submitted directly to Cybersource and never touches your systems.

Security Architecture

Unified Checkout uses many layers of protection to be compliant with PCI SAQ-A guidelines:

- **Iframe isolation**: All payment UI renders inside cross-origin iframes hosted by Cybersource. Your page cannot access payment data within the iframe due to the browser's same-origin policy.
- **Origin verification**: The SDK validates that the hosting page origin matches the `targetOrigins` declared in the session.
- **Immutable API**: The client interface returned by `VAS.UnifiedCheckout()` is frozen with `Object.freeze()`. This prevents runtime tampering.
- **Closure-based privacy**: The internal SDK state is not accessible from outside the SDK. There are no public properties that expose session data or credentials.
- **Short-lived tokens**: The session and transient tokens expire after a short period, limiting the window for misuse

Because Unified Checkout handles payment data capture within secure iframes, your page never receives, processes, or stores cardholder data. This means that you qualify for SAQ-A over the more burdensome SAQ A-EP or SAQ D and your PCI audit scope is significantly reduced compared to direct API integrations.

Even with all that Unified Checkout handles, you must still do the following to remain SAQ-A compliant:

- All pages that load the SDK must use Transport Layer Security (TLS).
- You must restrict which domains can load scripts and frames. For information about the required directives, see Security Recommendations (on page 282).
- You must generate a new session for each checkout and restrict `targetOrigins` to only your domains.
- You must send transient tokens to your server over HTTPS and verify their signatures before authorizing payments.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox
Business Center Test
Business Center Production
Customer Support

Client Version History

Below is a list of client versions and the features that are included in each version.

Important: Cybersource recommends that you use the most recent client version in your integration.

0.23

Accepts these card networks in the **allowedCardNetworks** field for manual card entry:

- Carnet
- Cartes Bancaires
- China UnionPay with card verification value (CVV)
- EFTPOS
- ELO
- JCrew PLCC
- mada
- Meeza

Ordering controls for the **allowedPaymentTypes** button.

De-coupling of PANENTRY from other payment types in the **allowedPaymentTypes** field.

0.24

Support for enabling combo cards in the capture context.

Support for eight-digit BINs.

Support for enabling card save in the capture context.

0.25

Addition of **Skip Verification next time** in the Click to Pay payment flow.

Support for CPF in the capture context.

0.26

Support for auto-lookup in Click to Pay when an email is included in the capture context.

Inclusion of the **cardDetails** field object in the transient token response.

Support for the **cardholderAuthenticationStatus** field object in the transient token response.

Support for the complete mandate.

0.28

Complete mandate enhancement to support Payer Authentication for manual card entry for Visa, Mastercard, American Express, Discover, JCB, Cartes Bancaires, China UnionPay, and ELO card brands.

Support for Afterpay as an **allowedPaymentType**.

Support for PayPak as an **allowedCardNetwork**.

Auto-enrollment for Click to Pay in supported markets.

Removal of the confirm or continue screen for specific use cases.

Static button for Click to Pay flows.

0.30

Support for iDeal, Multibanco, and Przelewy24|P24.

Complete mandate enhancement to support Payer Authentication for Google Pay and Click to Pay.

Support for Pakistan locales (en_PK and ur_PK).

New look and feel of Unified Checkout in line with EMVCO best practices.

0.31

Addition of the **data** object of the **orderInformation** field object and pass-through fields.

Support for **tokenCreate** in the complete mandate.

Support of pass-through fields, including challenge codes and data only, for Payer Authentication.

Support for Jaywan as an **allowedCardNetwork**.

Updated the payment details response to return detected card types. Multiple card types are shown when more than one card type is detected.

Support for Bancontact, Dragonpay, MyBank, and Tink Pay By Bank.

0.32

Support for KCP and UATP in the **allowedCardNetwork** field.

Support for Konbini as a post-pay reference payment method.

A radio button in the UI for Cartes Bancaires dual-branded cards.

0.33

Support for mobile as identity for Click to Pay accounts.

Japanese language translation updates.

UX captures billing and shipping information when they are not included in the capture context.

0.34

Iframes are used instead of pop-ups to reduce pop-up blocking and streamlining mobile deployment.

Additional BIN range for Jaywan card types.

0.35

Look and feel customization.

1.0

Configure payment options.

Configure customer data and payment flow.

Related information

Getting Started with REST
Response Codes
API Field Reference Guide
API Reference Sandbox

Business Center Test
Business Center Production
Customer Support

Visa Platform Connect: Specifications and Conditions for Resellers/Partners

The following are specifications and conditions that apply to a Reseller/Partner enabling its merchants through Cybersource for Visa Platform Connect ("VPC") processing. Failure to meet any of the specifications and conditions below is subject to the liability provisions and indemnification obligations under Reseller/Partner's contract with Visa/Cybersource.

1. Before boarding merchants for payment processing on a VPC acquirer's connection, Reseller/Partner and the VPC acquirer must have a contract or other legal agreement that permits Reseller/Partner to enable its merchants to process payments with the acquirer through the dedicated VPC connection and/or traditional connection with such VPC acquirer.

2. Reseller/Partner is responsible for boarding and enabling its merchants in accordance with the terms of the contract or other legal agreement with the relevant VPC acquirer.

3. Reseller/Partner acknowledges and agrees that all considerations and fees associated with chargebacks, interchange downgrades, settlement issues, funding delays, and other processing related activities are strictly between Reseller and the relevant VPC acquirer.

4. Reseller/Partner acknowledges and agrees that the relevant VPC acquirer is responsible for payment processing issues, including but not limited to, transaction declines by network/issuer, decline rates, and interchange qualification, as may be agreed to or outlined in the contract or other legal agreement between Reseller/Partner and such VPC acquirer.

DISCLAIMER: NEITHER VISA NOR CYBERSOURCE WILL BE RESPONSIBLE OR LIABLE FOR ANY ERRORS OR OMISSIONS BY THE Visa Platform Connect ACQUIRER IN PROCESSING TRANSACTIONS. NEITHER VISA NOR CYBERSOURCE WILL BE RESPONSIBLE OR LIABLE FOR RESELLER/PARTNER BOARDING MERCHANTS OR ENABLING MERCHANT PROCESSING IN VIOLATION OF THE TERMS AND CONDITIONS IMPOSED BY THE RELEVANT Visa Platform Connect ACQUIRER.